

Vorlesungsskript zum Modul

Informatik 2

im Bachelor-Studiengang (B.Sc.) Medieninformatik

Prof. Dr. Nils Martini

Hochschule für Angewandte Wissenschaften Hamburg
Fakultät Informatik und Digitale Gesellschaft

Inhaltsverzeichnis

1 Einführung Linux.....	3
1.1 Grundlegende Kommandos und Funktionen.....	3
1.1.1 Dateien und Dateisystem.....	3
1.1.2 Anlegen einer Datei.....	3
1.1.3 Dateien.....	4
1.1.4 Weiteres.....	6
1.1.5 Standard-Quellen für Ein- und Ausgabe.....	7
1.2 Kurze Einführung in Shell-Skripte.....	9
2 Reguläre Ausdrücke.....	11
2.1 Grundlagen regulärer Ausdrücke.....	11
2.2 Suchen und Ersetzen mit sed.....	14
2.3 Grundlagen von awk.....	15
3 Shell-Programmierung.....	18
3.1 Grundlagen.....	18
3.2 Rechenoperationen.....	19
3.3 Shell-Skripte.....	21
3.3.1 Variablen und Argumente.....	21
3.3.2 Bedingte Anweisung if.....	22
3.3.3 case-Anweisung.....	23
3.3.4 Schleifen.....	24
3.3.5 Ein- und Ausgabe.....	25
4 Systemstart.....	27
4.1 Initialisierung der Hardware.....	27
4.1.1 BIOS.....	27
4.1.2 UEFI.....	27
4.1.3 MBR.....	27
4.1.4 GUID Partition Table.....	29
4.1.5 Boot-Loader GRUB.....	29
4.2 Kernel.....	30
4.3 Unix systemd.....	31
4.4 Prozesse.....	34
5 Dateisysteme.....	37
5.1 Grundlagen.....	37
5.2 Einbinden von Dateisystemen.....	37
5.3 Dateisystem ext2.....	38
5.4 Dateisysteme mit Journaling.....	41
5.5 Dateisystem ext4.....	42
5.6 B-Tree-Filesysteme.....	43
6 OSI-Modell.....	44
7 Vermittlungsschicht.....	46
7.1 Grundlagen.....	46
7.2 IPv4 im Internet.....	47
7.2.1 IPv4-Header.....	47
7.2.2 IPv4-Adressen.....	48
7.2.3 Subnetzmasken.....	50
7.2.4 Beispiel für IPv4-Adressen und Subnetzmasken.....	52
7.3 IPv6-Adressen.....	53
7.3.1 Adress-Typen und -Formate.....	53
7.3.2 Automatische Konfiguration für IPv6.....	54
7.3.3 Path MTU Discovery.....	55
7.4 Steuerprotokolle.....	55
7.4.1 ARP.....	55
7.4.2 ICMP.....	56
7.4.3 ICMPv6.....	57
7.5 Routing-Protokolle.....	58
7.5.1 Distance-Vector-Verfahren.....	58
7.5.2 RIP.....	60
7.5.3 Link-State-Verfahren.....	61
7.5.4 OSPF.....	62

7.5.5 BGP.....	65
8 Routing-Algorithmen.....	66
8.1 Dijkstra-Algorithmus.....	66
8.2 Bellman-Ford-Algorithmus.....	67
9 Algorithmen und Datenstrukturen.....	69
9.1 Feld/Array und verkettete Liste.....	69
9.2 Laufzeitabschätzung.....	70
9.3 Sortieren.....	70
9.3.1 Sortieren durch Auswahl.....	70
9.3.2 Sortieren durch Einfügen.....	71
9.3.3 Bubble-Sort.....	72
9.3.4 Quicksort (Sortieren durch rekursives Teilen).....	72
9.4 Baum-Strukturen.....	74
9.4.1 Binäres Suchen.....	74
9.4.2 Skiplisten.....	75
9.4.3 Binäre Suchbäume.....	76
9.4.4 Balancierte Bäume (AVL-Bäume).....	79
9.4.5 B-Bäume (Bäume mit extern gespeicherten Daten).....	83
Literatur.....	86

1 Einführung Linux

1.1 Grundlegende Kommandos und Funktionen

Viele Funktionen im Linux-Betriebssystem lassen sich über einfache Kommandos in einem sog. Terminal-Fenster ausführen. Diese Übung soll einige wenige wichtige Kommandos erläutern.

1.1.1 Dateien und Dateisystem

Die Verzeichnisstruktur unter Linux beginnt mit dem Wurzelverzeichnis „root“ (gekennzeichnet mit einem Schrägstrich /). Es enthält u.a. folgende wichtige Verzeichnisse:

- /bin grundlegende Linux-Kommandos, die von allen Benutzern ausgeführt werden können
- /sbin Systemverwaltungsprogramme, die nur root ausführt bzw. nur von ihm ausgeführt werden dürfen
- /lib und /usr/lib Bibliotheks-Routinen (Libraries)
- /usr/bin ausführbare Programme
- /usr/sbin ausführbare Programme (i.d.R. nur für den Benutzer root)
- /dev Device-Files (Geräte-Dateien), über die der Zugriff auf die Hardware realisiert wird
- /home Benutzerverzeichnisse
- /root Home-Verzeichnis des Systemadministrators root
- /etc Konfigurationsdateien
- /var veränderliche Dateien, z.B. Log-Dateien
- /proc wird beim Systemstart erzeugt und enthält Informationen über laufende Prozesse, geladene Module oder aktuelle Systemzustände bzw. -einstellungen

Es befinden sich sämtliche Verzeichnisse in dieser Verzeichnisstruktur, auch wenn mehrere Datenträger eingebunden sein sollten (z.B. Festplatten, USB-Sticks, DVD-Laufwerke, Speicherkarten oder auch Netzwerk-Laufwerke). Es gibt daher keine Laufwerksbuchstaben (C:, D:, H: usw.) wie unter Windows.

Das Kommando `df` zeigt alle eingehängten Datenträger und ihre Einhängepunkte (Mount-Points) in der Verzeichnisstruktur an (zudem einige virtuelle Dateisysteme, die aber nur der internen Verwaltung dienen).

Ein Verzeichnis wird mit dem Kommando `mkdir verzeichnisname` angelegt.

In ein Verzeichnis wechselt man mit (change directory) `cd verzeichnisname`, ohne Verzeichnisnamen:

Wechsel in das eigene Home-Verzeichnis, mit `cd ..` Wechsel in das nächst höhere Verzeichnis.

Zusammenfassen mehrerer Schritte ist möglich, z.B.: `cd ../../verz1/verz2/verz3`

Anzeigen des Verzeichnissesinhalts: `ls` (listing)

Einige Optionen zu `ls`:

`ls -a` (all, anzeigen versteckter Dateien (Dateiname beginnt mit einem Punkt))

`ls -l` (long listing, anzeigen von Dateiattributen)

`ls -ltr` (long listing, time reverse: umgekehrte Sortierung der Ausgabe nach dem Modifikations-Datum)

1.1.2 Anlegen einer Datei

1. kurze Textdatei:

`cat > dateiname`, Text eingeben, Zeilen mit Return abschließen, zum Beenden und Speichern `Strg-D`

2. Text-Editor `kate` (grafische Oberfläche):

in einem Terminalfenster `kate &` eingeben (das `&` gibt das Terminal wieder frei), dann „neue Sitzung“

3. für Experten: Text-Editor `vi`

`vi dateiname`

Innerhalb des `vi`-Editors wird zwischen einen Kommando- und einem Eingabe-Modus unterschieden

Wechsel in den Kommandos-Modus immer mit `ESC`

Wechsel in den Eingabe-Modus, u.a. mit den Kommandos:

- i (insert)
- a (append)
- o (open, neue Zeile unterhalb der aktuellen Zeile)
- O (neue Zeile oberhalb der aktuellen Zeile)

Kommandos:

- x (Löschen eines Zeichens hinter der aktuellen Cursorposition)
- bzw. X (vor der aktuellen Position)
- dd (löschen einer ganzen Zeile)
- dw (löschen eines Wortes)
- eine Zahl vor dem Kommando wiederholt den Befehl entsprechend, z.B. 3dd löscht drei Zeilen (die Zeile an der aktuellen Cursorposition und die zwei folgenden)
- u (undo) letzten Befehl rückgängig machen
- Y ganze Zeile kopieren und mit P Kopierregister einfügen
- Beenden und Speichern: in den Kommandomodus wechseln, dann „write/quit“ mit :wq
- oder beenden ohne zu speichern (Änderungen gehen verloren) :q!
- interne Hilfe-Funktion mit F1 in einem eigenen Fensterbereich: dort mit Cursor oder Bildlauf-Tasten bewegen oder suchen mit :help suchwort , Verlassen der Hilfe mit :q , bei geöffnetem Hilfefenster wechseln in die Datei (und wieder zurück) mit Strg-w
- Zeilennummern anzeigen :set number ausschalten mit :set nonumber

schnelles Bewegen des Cursors:

- G (Dateiende)
- gg (Dateianfang), oder z.B. 5g Cursor in Zeile Nr. 5
- \$ (Zeilenende)
- 0 oder ^ (Zeilenanfang)
- falls keine Cursor-Tasten verfügbar sind: h, j, k, l

1.1.3 Dateien

Arbeiten mit Dateien:

Löschen `rm dateiname`

Kopieren `cp quelledatei zieldatei` (auch mit Verzeichnissen kombinierbar z.B. `cp blah /home/tina/xyz` , ist xyz ein Verzeichnis wird die Datei blah in dieses Verzeichnis kopiert, ist xyz ein Dateiname wird blah in das Verzeichnis /home/tina kopiert und dort in xyz umbenannt, Achtung: eine vorhandene Datei mit diesem Namen wird ohne Rückfrage überschrieben!

Verschieben (move) von Dateien: `mv quelledatei zieldatei`

Anzeigen des Dateiinhalts (sofern als ASCII-Zeichen darstellbar):

`cat dateiname`

`more dateiname` (seitenweise Ausgabe, weiterblättern mit Leertaste)

Suchen (Finden) von Dateien in einer Verzeichnisstruktur:

`find . -name "*bla*"` (Suchen unterhalb der aktuellen Verzeichnisposition nach Dateien, in deren Namen die Zeichenkette „bla“ vorkommt)

Dateiattribute

Dateien werden mit bestimmten Eigenschaften abgespeichert, z.B. Benutzer-Kennung (User-ID) des Besitzers, eine Gruppen-ID, Zugriffsrechte, Erstellungs- und Modifikationsdatum sowie „inode-Time“ (ctime), Dateigröße. Diese Eigenschaften lassen sich mit dem Kommando `ls -l` (Zeit: Modifikationsdatum) anzeigen:

```
-rw-r--r--  user-id group-id 4036685 14. Feb 2011  dateiname
drwxr-xr-x  user-id group-id    160 23. Mai 17:26 ein_verzeichnis
```

Rechte: r (read), w (write), x (execute), jeweils für User, Group und World (alle Benutzer eines Systems). Der erste Buchstabe ganz links gibt den Dateityp an: d für Directory (Verzeichnis) und - für eine normale Datei

Zuweisung/Änderung der User-ID: `chown user-id dateiname`

Zuweisung/Änderung der Gruppe: `chgrp group-id dateiname`

Zuweisung der Rechte: `chmod ### dateiname` (### sind drei Zahlen, die sich zusammensetzen aus:

erste Zahl: Rechte des Benutzers, zweite Zahl: Rechte der Gruppe, dritte Zahl: Rechte für alle. die Zahlen bestehen aus einem Code: 4 für Read, 2 für Write, 1 für Execute; sollen mehrere Rechte vergeben werden, müssen die

entsprechenden Zahlen addiert werden). Beispiel: `rw-` entspricht der Zahl $4+2=6$, `r-x` entspricht $4+1=5$
`chmod 644 dateiname` weist dem Besitzer der Datei die Rechte Schreiben (2) und Lesen (4) zu, der Gruppe und World nur das Recht Lesen.

Diese herkömmliche Rechteverwaltung ist für die meisten Anwendungen vollkommen ausreichend. Mit Access Control Lists (ACL) lassen sich Rechte jedoch wesentlich feinteiliger vergeben. Beispielsweise kann einem anderen (einzelnen) Benutzer ein Schreibrecht an einer eigenen Datei gegeben werden, ohne dieses Recht gleich einer ganzen Benutzergruppe geben zu müssen; oder man vergibt Leserechte an eine Benutzergruppe, schließt aber bestimmte Benutzer aus dieser Gruppe davon aus. Ob ergänzend zu den normalen Rechten auch ACL-Regeln existieren, zeigt beim Kommando `ls -l` ein `+` zwischen der Rechtespalte und der Owner-Spalte. Mit dem Kommando `getfacl datei` lassen sich diese Regeln anzeigen. ACLs sind unter Linux wenig verbreitet, zudem können viele Standardkommandos und Programme nicht damit umgehen. Sinnvoll sind ACLs jedoch bei gemeinsam mit Windows genutzten Dateidiensten (z.B. Samba) oder verteilten Speichern.

Dateitypen

Außer den beiden bereits oben genannten Dateitypen `d` und `-` (für Verzeichnisse und normale Dateien) gibt es unter Unix noch weitere spezielle Dateitypen. Mit einem sog. symbolischen Link (Zeichen `l`) wird ein Verweis auf eine Datei angelegt (Kommando `ln`), sodass man z.B. aus seinem Benutzer-Verzeichnis heraus Zugriff auf Dateien hat, die eigentlich in anderen Verzeichnissen gespeichert sind, ohne diese Dateien kopieren zu müssen. Weitere Dateitypen sind:

Named Pipe `p`: hiermit kann eine Pufferdatei (FIFO) für Pipes angelegt werden, wenn die Ausgabe eines Kommandos nur indirekt als Eingabe für ein anderes Kommando verwendet werden soll oder zwei unabhängige Programme untereinander Daten austauschen sollen.

Zeichenorientierte Geräte `c`: „character oriented“ Device-Files, z.B. für serielle Verbindungen `/dev/ttyUSB1`

Blockorientierte Geräte `b`: Festplatte z.B. `/dev/hda` für IDE- oder `/dev/sda` für SCSI/SAS/SATA-Platten

Wildcards

Mit Ersatzmustern (Wildcards) können bestimmte Zeichen oder Zeichenketten von der Shell durch andere Zeichen ersetzt werden. Beispielsweise muss bei der Verwendung des Kommandos `grep` als Argument eine Datei angegeben werden, in der nach einer Zeichenkette gesucht werden soll. Möchte man nicht nur in einer bestimmten Datei suchen, sondern in allen Dateien im aktuellen Verzeichnis, so kann `*` als Ersatzmuster für beliebige Dateien verwendet werden.

Suche nach `abc` in der Datei `datei.txt`: `grep abc datei.txt`

Suche nach `abc` in allen Dateien des aktuellen Verzeichnisses: `grep abc *`

Suche nach `abc` in Dateien, deren Name mit „`dat`“ beginnt: `grep abc dat*`

Mit dem Fragezeichen `?` als Ersatzmuster ist ein einzelnes Zeichen gemeint: `grep abc datei?.txt`

Hier sucht `grep` z.B. in `datei1.txt` oder `dateiX.txt` aber nicht in `datei12.txt`

Mit eckigen Klammern kann ein Zeichenbereich als Ersatzmuster angegeben werden:

Suche nach `abc` in den Dateien `datei1.txt` und `datei2.txt`, aber nicht in `datei3.txt` und auch nicht in `datei12.txt`:

`grep abc datei[12].txt`

Suche in Dateien, die zwischen „`datei`“ und „`.txt`“ zwar irgendein Zeichen enthalten, aber nicht die 1 oder die 2:

`grep abc datei[!12].txt`

Suche nach `abc` in den Dateien `datei1.txt`, `datei2.txt`, `datei3.txt` usw., auch in `datei12.txt`, aber auch in

`datei12A.txt`: `grep abc datei[0-9]*.txt`

Soll nur in Dateien gesucht werden, deren Name mit „`datei`“ beginnt, mit „`.txt`“ endet aber dazwischen nur eine beliebige Anzahl Zahlen aber keine Buchstaben enthält, so wird es schwieriger.

`grep abc datei[0-9]*[!a-zA-Z].txt` bringt nicht die gewünschte Lösung, denn es wird u.a. in `datei125.txt` gesucht, aber auch in `datei37X5.txt`, nicht jedoch in `datei7B.txt` und auch nicht in `datei2.txt`, weil hinter „`datei`“ eine Ziffer vorkommen muss, dann eine beliebige Anzahl von Zeichen (auch keines) und dann ein Zeichen vor „`.txt`“, das aber kein Buchstabe sein darf.

Mithilfe der Klammererweiterung (Brace Extensions) durch geschweifte Klammern, kann die `bash` diverse Kombinationen aus Zeichen zusammensetzen.

`echo {x,y,z}{1,2,3}` ergibt die Ausgabe: `x1 x2 x3 y1 y2 y3 z1 z2 z3`

`touch datei{3..5}` legt folgende Dateien an: `datei3`, `datei4`, `datei5`

`touch dat{0..21..7}{a,b}` legt an: `dat0a`, `dat0b`, `dat7a`, `dat7b`, `dat14a`, `dat14b`, `dat21a`, `dat21b`

Die obige Aufgabe (`dateiZahl.txt`) könnte nun wie folgt erfüllt werden, wobei in allen existierenden Dateien von `datei0.txt` bis `datei299.txt` gesucht wird (nicht jedoch z.B. `datei03.txt`):

`grep abc datei{[0-9],[1-9][0-9],[1-2][0-9][0-9]}.txt`

Es funktioniert auch `grep abc datei{0..299}.txt`

Allerdings wird bei letzterer Variante in allen Dateien mit den hiermit gebildeten Namen gesucht, auch wenn sie gar nicht existieren; im Gegensatz zu den Wildcards, die nur auf existierende Dateien angewendet werden. Die nun

unweigerlich auftretenden Fehlermeldungen (STDERR) müssen geeignet verarbeitet werden (2>).
 Es ist auch möglich, Dateinamen mittels arithmetischer Berechnungen zu bilden; folgender `ls`-Befehl gibt alle Dateien aus, die mit „datei15“ beginnen: `ls datei${3*5}*`

Diese Beispiele zeigen, wie mithilfe von Wildcards Ersetzungen durchgeführt werden können. Wenn jedoch die hierfür eingesetzten Sonderzeichen selbst Teil des Suchmusters sind oder Leerzeichen enthalten sind, so müssen geeignete Klammerungen verwendet werden. Soll z.B. ein `&` innerhalb einer Textdatei gesucht werden, so muss mit doppelten Anführungszeichen geklammert werden: `grep "black & white" *`

Die einzigen Sonderzeichen, die trotz dieser Anführungszeichen noch interpretiert werden, sind `*` und `$`-Zeichen. Mit `$` ist eine Variablensubstitution möglich. Enthält die zuvor definierte Variable `$var` (siehe Kapitel Shell-Programmierung) die Zeichenkette „ABC“, so wird mit folgendem Kommando die Zeichenkette `t&xtABC` gesucht: `grep "t&xt$var" *`

Soll auch das `$`-Zeichen Teil des Suchmusters sein, so muss mit geraden Anführungszeichen geklammert werden; das folgende Kommando sucht die Zeichenkette `t&xt$var`: `grep 't&xt$var' *`

Das `*`-Zeichen muss immer einzeln geschützt werden: `grep "St\*rn" *`

Weiteres zu diesem Thema siehe Kapitel „Reguläre Ausdrücke“.

Die Klammerung in das Gravis-Zeichen (frz.: accent grave) ersetzt das so eingeklammerte Kommando durch sein Ergebnis, sog. „**Kommando-Substitution**“: `ls -l `grep -l Wort *``

Hierbei wird zunächst das `grep`-Kommando gestartet, das alle Dateinamen auflistet, die die Zeichenkette „Wort“ enthalten, und für diese Dateinamen wird dann das „long listing“ mit `ls -l` ausgeführt. Alternativ kann auch `$(kommando)` verwendet werden.

1.1.4 Weiteres

Wichtige Kommandos

Liste der laufenden Prozesse ausgeben: `ps`

alle Prozesse ausgeben: `ps auxw` (hier ausnahmesweise ohne das `-` für Optionen)

Prozesse beenden: `kill PID` mit PID als Prozess-Nummer (als Ausgabe von `ps`)

laufende Anzeige von Prozessen: `top`

Anzeige der Anzahl von Zeilen, Wörtern und Zeichen: `wc datei`

Suchen nach Zeichenketten in Dateien: `grep zeichen datei`

Kommandos verketteten

Ein-/Ausgabe-Umlenkung

Ausgaben, die ein Kommando erzeugt, können an andere Kommandos direkt weitergegeben oder in Dateien gespeichert werden oder es können Dateiinhalte in Kommandos eingelesen werden.

Ausgabeweiterleitung (Pipe), z.B. `ps | wc`

Ausgabeumleitung, z.B. `ps > datei`

auch Kombinationen sind möglich: `ps | wc > datei`

Zwei Kommandos direkt nacheinander ausführen: `ps ; wc datei`

Das zweite Kommando wird nur dann ausgeführt wenn das erste erfolgreich war (exit-Code 0):

`ls datei && echo ja`

Das zweite Kommando wird nur dann ausgeführt wenn das erste fehlerhaft war (exit-Code größer 0):

`ls datei || echo nein`

Netzwerk-Kommandos

Anzeige der Netzwerkkonfiguration: `/sbin/ifconfig`

Anzeige des Netzwerkstatus (offene TCP-Verbindungen): `netstat --tcp`

Verbindungen prüfen: `ping <IP-Adresse oder Rechnername>` z.B. `ping 192.168.76.1`
 oder `ping asterix.opn4`

Verbindungswege anzeigen (Pfadangabe hier erforderlich):

`/usr/sbin/traceroute <IP-Adresse oder Name>`

oder `/usr/sbin/traceroute -n <IP-Adresse>` (die Option `n` unterdrückt die DNS-Abfrage)

Anzeige der Routingtabelle: `/sbin/route` oder `/sbin/route -n`
 Filetransfer: `ftp <IP-Adresse>`
 Remote Login: `telnet <IP-Adresse>`
 Verschlüsseltes Remote Login: `slogin <IP-Adresse>`
 Mail (grafische Oberfläche): `kmail&`
 Mail per Kommandozeile: `mailx -s <Subject> <Mailadresse>`, dann Text eingeben, letzte Zeile mit Return abschließen, absenden mit `Strg-D`

Tastaturkürzel und Maus

Auch wenn die Verwendung von Tastaturkürzeln in den verschiedenen Linux-Distributionen sowie Desktop-Systemen (wie KDE oder Gnome) oder Anwendungsprogrammen nicht ganz einheitlich geregelt ist, gibt es doch viele Gemeinsamkeiten, auch mit der Windows-Welt:

`Strg-C` markierten Text in die Zwischenablage kopieren
`Strg-V` Text aus der Zwischenablage einfügen
`Strg-S` Datei speichern
`Alt-Tab` wechseln zwischen verschiedenen Fenstern (sofern vorhanden)
`Alt-F1` Startmenü (je nach Tastatur auch die Windows-Taste)
`Alt-F4` beendet das aktuelle Fenster

`Strg-Alt-F1` wenn die grafische Oberfläche (KDE oder Gnome) aktiv ist: wechseln in die Konsole 1 (entspr. F2 usw.)

`Strg-Alt-F7` wechseln in die grafische Oberfläche

`Strg-Alt-Backspace` beendet das Grafiksystem gewaltsam, z.B. bei „Aufhängern“ (länger gedrückt halten)

Beim KDE-Desktop genügt im Gegensatz zu Windows oft ein einfacher Maus-Klick für eine Aktion (z.B. Programm vom Desktop starten, Datei öffnen).

Ein Linux-Spezialität: **Kopieren mit der Maus**. Text mit der linken Maustaste markieren und dann mit der **mittleren** Maustaste an der gewünschten Stelle einfügen (dies ist sehr praktisch auch wenn das Handling anfangs etwas gewöhnungsbedürftig sein kann, da die mittlere Taste oft das Maus-Rad ist)

1.1.5 Standard-Quellen für Ein- und Ausgabe:

Beim Standard-Input STDIN (in Zahlform 0) handelt es sich bei interaktiven Programmen wie z.B. einem Shellskript meistens um die Tastatur sonst um die Ausgabe eines anderen Programms (z.B. mittels Pipe | oder

Eingabeumlenkung <)

Die Standard-Ausgabe STDOUT (1) erfolgt in derjenigen Umgebung (z.B. dem Konsolen-Fenster/Terminal), in der das Programm gestartet wurde oder mittels Ausgabeumlenkung (>) in eine Datei oder mittels Pipe (|) in ein anderes Programm.

Fehlermeldungen werden in STDERR (2) geschrieben (2>)

Beispiele mit dem grep-Kommando („Global search for **R**egular **E**xpressions and **P**rint out matched lines“)

`grep suche datei`

Es wird das Textmuster „suche“ in der Datei „datei“ gesucht und alle Zeilen, in denen das Suchmuster vorkommt, ausgegeben. Enthält „datei“ ein Wildcard (*) oder werden mehrere Dateien angegeben, wird zusätzlich der Name der Datei der Zeilenausgabe vorangestellt.

Wichtige Optionen: `-i` (case-**i**nsensitive), `-x` („suche“ muss einer ganzen Zeile entsprechen), `-v` („nicht-vorhandensein“ des Suchmusters), `-w` (das Suchmuster muss als vollständiges Wort vorhanden sein), `-l` (nur Ausgabe des Namens der Datei, in der das Suchmuster gefunden wurde, jedoch nicht die Zeile selbst), `-h` (nur Ausgabe der Zeile, in der das Suchmuster gefunden wurde, jedoch nicht des Dateinamens; Standard bei Angabe nur eines Dateinamens), `-s` (Unterdrückung von Fehlermeldungen bei nicht durchsuchbaren oder nicht existierenden Dateien), `-n` (Ausgabe der Zeilennummer), `-r` (es wird auch in Dateien in Unterverzeichnissen gesucht)

Umlenkung der Standardausgabe in eine Datei: `grep suchwort datei > ausgabedatei`

Achtung: Eingabe- und Ausgabedateiname dürfen nicht gleich sein, da sonst diese Datei leer überschrieben wird!

Standardausgabe an eine Datei anhängen: `grep suchwort datei >> ausgabedatei`

Hierbei dürfen Ein- und Ausgabedatei gleich sein; das Such-Ergebnis wird an die existierende Datei angehängt.

Soll die Ausgabe sowohl in eine Datei geschrieben als auch im Terminalfenster ausgegeben werden, so kann das

Kommando `tee` verwendet werden: `grep suchwort datei | tee ausgabedatei`

Möchte man lediglich wissen, ob die Suche erfolgreich war oder nicht (d.h. ohne die konkrete Ausgabe der entsprechenden Zeile), muss zunächst die Ausgabe unterdrückt werden:

Ausgabeumlenkung „ins Nirvana“: `grep suchwort datei > /dev/null`

oder alternativ beim `grep`-Kommando dasselbe mit der Option `-q`.

Dann kann der sog. Exit-Status abgerufen werden mit: `echo $?`

Üblicherweise ist der Exit-Status 0 „bei Erfolg“ (d.h. beim `grep`-Kommando wurde eine Zeile gefunden) oder Exit-Status 1 „bei Nicht-Erfolg“ (beim `grep`-Kommando wurde keine passende Zeile gefunden) oder Exit-Status 2 oder größer im Fehlerfall (z.B. wenn die angegebene Datei nicht existiert).

Soll außer der Standardausgabe (STDOUT (1)) auch die Standard-Fehlermeldung STDERR (2) umgeleitet werden, so muss zusätzlich eine entsprechende Umlenkung angegeben werden:

`grep suchwort datei > /dev/null 2> fehler`

Statt einer Datei (hier „fehler“) kann auch wieder die Standardausgabe (1) für die Fehlermeldungen verwendet

werden: `grep suchwort datei > /dev/null 2>&1` (Hinweis: ohne das `&`-Zeichen vor der 1 würde eine Datei mit dem Namen „1“ angelegt werden). Beim `grep`-Kommando kann diese Unterdrückung von STDERR auch durch die Option `-s` erreicht werden.

Dasselbe Ergebnis erhält man mit: `grep suchwort datei &> /dev/null`

Nun kann mit dem Kommando `echo $?` der aktuelle Exit-Status abgefragt werden.

Der Zweck solcher Standard-Umlenkungen ist für die Arbeit in der Konsole nicht unbedingt unmittelbar ersichtlich, bei der Verwendung von Kommandos innerhalb von Shell-Skripten ist dies aber oftmals unumgänglich. Sollen beispielsweise innerhalb von Skripten Aktionen mit Dateien nur dann ausgeführt werden, wenn diese Dateien bestimmte Textmuster enthalten, so wäre die Ausgabe dieses Musters wenig hilfreich und Fehlermeldungen könnten sich auf den Ablauf eines Skriptes sogar störend auswirken.

Named-Pipe

Mit der bereits bekannten Pipe „|“ kann die Ausgabe des einen Kommandos direkt in die Eingabe eines anderen Kommandos weitergeleitet werden (vorausgesetzt, beide Kommandos unterstützen diese Form der Standard-Eingabe bzw. -Ausgabe). Dies funktioniert jedoch nur dann, wenn beide Kommandos hintereinander in eine Zeile geschrieben werden können.

Eine andere Möglichkeit, Daten von einem Kommando in ein anderes zu überführen (insbesondere innerhalb von Shell-Skripten) besteht mittels der Kommando-Substitution (s.o.) Hierbei wird zur Laufzeit des einen Kommandos ein anderes ausgeführt, was bspw. bei variabler Zusammenstellung eines Kommandos in Shell-Skripten sinnvoll ist. Beide Vorgehensweisen (Pipe oder Kommando-Substitution) setzen den zeitgleichen Start beider Kommandos voraus bzw. es ist erforderlich, ein Shellskript oder anderweitig ausführbares Programm selbst zu beeinflussen (also zu programmieren).

Szenario: Es gibt zwei Programme A und B, wobei B die Ausgabe von A einlesen soll. Startet nun bspw. A obwohl die Voraussetzungen zum Start von B noch gar nicht erfüllt sind, so ist das Ergebnis evtl. unbrauchbar; dasselbe gilt, wenn B startet obwohl A noch nicht bereit ist. In diesem Fall eignen sich weder die normale Pipe noch die Kommando-Substitution. Hier kommt der Spezialfall der Ausgabe-Umlenkung, die „Named-Pipe“, zum Einsatz. Hierbei wird eine FIFO-Datei (FIFO: first in first out) angelegt, in die Programm A hineinschreibt und B liest.

Anlegen der Named Pipe: `mkfifo FIFFI`

Programm A aktivieren: `ProgA > FIFFI &`

Programm A startet aber erst dann wirklich, wenn B startet und aus der aktivierten Pipe liest: `ProgB < FIFFI &`
Umgekehrt funktioniert dies genauso, indem zunächst B die Named Pipe lesend aktiviert. Eines der Programme greift also auf die Named Pipe zu und wartet solange mit der tatsächlichen Ausführung, bis auch das andere Programm zugreift.

Hinweis: FIFOs sollten nur in eigenen Verzeichnissen angelegt werden, da sonst Programme auch zufällig bzw. ungewollt auf die Pipe zugreifen könnten. Wenn `grep muster *` eine existierende FIFO-Datei öffnet, so wartet `grep` mit der Ausführung, bis ein anderes Programm in diese Datei hineinschreibt!

1.2 Kurze Einführung in Shell-Skripte

Einige kurze Beispiele von Shell-Skripten sollen die prinzipiellen Möglichkeiten insbesondere zur Automatisierung von Administrationsaufgaben zeigen (dies ist nur eine sehr kurze Einführung in Shell-Programmierung!).

Zum Beginn: ein Hello World Skript. Das Skript muss in einem Texteditor eingegeben werden (z.B. kate, kwrite, vi), bei Verwendung von LibreOffice Writer, Word o.ä. muss unbedingt ein „reiner Textmodus“ eingeschaltet werden, da sonst die unsichtbar eingebetteten Steuerzeichen die Ausführung von Shell-Skripten stören können). Der Dateiname sollte nur Buchstaben, Zahlen oder den Unterstrich enthalten, da andere Sonderzeichen (z.B. \$ oder &) von der Shell interpretiert werden, auch sollten die Namen von bereits vorhandenen Kommandos des Betriebssystems vermieden werden (also z.B. das Skript nicht cp, mkdir oder ping nennen).

Im Texteditor wird zunächst die Startzeile eines Shell-Skriptes eingetragen (die sog. „Shebang-Zeile“), mit der der ausführende Shell-Typ bestimmt wird (hier die „Bourne Again Shell“ bash); den hierfür erforderlichen absoluten Pfad kann man mit `which bash` herausfinden. Das Doppelkreuz (hash) markiert normalerweise eine nicht auszuführende Kommentarzeile, im Zusammenhang mit dem Ausrufezeichen (bang) bedeutet es jedoch: Ausführen des unmittelbar dahinter stehenden Kommandos, wobei der Name der Skriptdatei als Argument übergeben wird. Der Typ der aktuell im laufenden Terminalfenster verwendeten Shell ist dabei nicht von Bedeutung, da die in der Shebang-Zeile definierte Shell als Sub-Shell ausgeführt wird. Somit kann dieses Skript auch auf einem anderen Rechner in einer ganz anderen Umgebung problemlos ausgeführt werden, sofern die bash dort installiert ist. Das folgende Beispiel-Skript führt dann den Befehl `echo` aus:

```
#!/bin/bash
echo Hello World
```

Wird diese Datei z.B. unter dem Namen `Test` gespeichert, so kann das Skript von der Kommandozeile gestartet werden mit: `sh Test`

Alternativ können die Dateiattribute auf „ausführbar“ (`chmod 755 Test`) gesetzt werden, dann kann das Skript direkt mit `./Test` gestartet werden. Wenn das Verzeichnis, in dem die Datei gespeichert ist, in der sog. Pfad-Variablen enthalten ist, so könnte das Skript einfach mit `Test` gestartet werden.

Ein Skript wird automatisch nach Abarbeitung der letzten Zeile beendet. Soll jedoch das Skript vorher beendet werden (was natürlich nur bei Verzweigungen sinnvoll ist), so kann das Kommando `exit` verwendet werden oder `exit n`, wobei `n` der Exit-Status ist (`n=0` zeigt das reguläre Beenden an, andere Werte können als Fehler-Codes verwendet werden). Der aktuelle Exit-Status kann nach Beendigung des Skriptes mit dem Kommando `echo $?` abgerufen werden.

Variablen

Variablen werden Werte mit dem Gleichheitszeichen (=) zugewiesen; das Aufrufen der Variablen erfolgt mittels voranstellen des \$-Zeichens vor dem Variablennamen.

```
#!/bin/bash
VARI="Hello World"
echo $VARI
```

Eine häufig vorkommende Aufgabe in Shellskripten ist die dynamische Zuweisung von Variablenwerten mithilfe von Kommandos. Folgendes Beispiel zeigt das Anlegen einer Datei mit aktuellem Datum im Dateinamen mittels Aufruf des Systembefehls `date` (die Manual-Pages (`man date`) zeigen die Bedeutung der Optionen `+%d%m%y`). Ein Kommando wird hierzu in runden Klammern aufgerufen mit vorangestelltem \$-Zeichen.

```
#!/bin/bash
VARI=~/Documents/testdatei-$(date +%d%m%y).txt
touch $VARI
```

Einfache IF-Abfrage:

```
#!/bin/bash
V1="bla"
V2="blah"
if [ "$V1" = "$V2" ]; then
    echo Die Variablen sind gleich
else
    echo Die Variablen sind nicht gleich
fi
```

Im folgenden Skript wird ein zusätzliches Argument beim Kommando-Aufruf angegeben, hier ist es der gewünschte Name einer anzulegenden Datei, wobei das Skript prüft, ob überhaupt ein Dateiname angegeben wurde (wenn nicht: Abbruch des Skriptes). Die Variable \$0 ist der Name der Skript-Datei selbst und \$1 ist das erste Argument, die Option -z prüft, ob dieses Argument die Länge Null hat.

```
#!/bin/bash
if [ -z "$1" ]; then
    echo Bitte Dateinamen angeben: $0 Dateiname
    exit
fi
DATEIPRAEFIX=$1
VERZEICHNIS=~/Documents
DATEI=$DATEIPRAEFIX-$(date +%d%m%Y)
touch $VERZEICHNIS/$DATEI
```

Interaktive Eingabe (ersetzen Sie den if-Block im vorangegangenen Beispiel durch die folgenden zwei Zeilen) und passen Sie die relevanten Variablennamen an

```
echo Bitte einen Dateinamen eingeben:
read EINGABE
```

Mithilfe von Shell-Skripten können sehr umfangreiche oder detaillierte Analysen von Dateien durchgeführt werden, da viele Funktionen für die Arbeit mit Dateien und Texten optimiert sind. Das folgende Skript zählt die Login-Versuche des Users „student“ am heutigen Tag anhand der entsprechenden Einträge im „journal“. Das Kommando `awk` ist ein mächtiges Tool zur Analyse von Zeilen in Textdateien. Versuchen Sie die grundsätzliche Funktion des Skriptes nachzuvollziehen. Loggen Sie sich zuvor mindestens einmal mit dem Kommando `su - student` ein.

```
#!/bin/bash
DAT1=`journalctl -xn | grep "session opened for user student" | awk '{print $1
    $2;}' | awk -F T '{print $1;}'` (dies ist EINE zusammenhängende Zeile!)
AKTUELLES_DATUM=$(date +%b%d)

for i in $DAT1; do
    if [ "$i" = "$AKTUELLES_DATUM" ]; then
        let j=j+1
    fi
done
echo Logins von student am $(date +%d.%m.%Y) : $j
```

2 Reguläre Ausdrücke

2.1 Grundlagen regulärer Ausdrücke

Bei „regulären Ausdrücken“ (engl. Regular Expressions, abgekürzt: regEx) handelt es sich um eine formale Sprache, um insbesondere in Textdateien nach Mustern oder bestimmten Zeichenketten zu suchen oder diese zu ersetzen. Reguläre Ausdrücke finden sich in vielen Programmiersprachen, werden häufig im Zusammenhang mit den Unix-Kommando `grep`, dem Streameditor `sed`, dem Interpreter `awk` oder der Interpretersprache Perl eingesetzt und finden sich in allen Anwendungen, die Muster suchen müssen (z.B. Firewalls, die nach bestimmten schädlichen Inhalten von Datenpaketen im Netzwerk suchen).

Die Syntax ist in den verschiedenen Anwendungen und Programmiersprachen bei den grundlegenden Ausdrücken im Wesentlichen die gleiche, lediglich bei spezielleren Funktionen kann es Abweichungen in den verschiedenen Implementationen geben. In diesem Skript wird die Syntax nach „Perl Compatible Regular Expressions“ (PCRE) verwendet, die häufig in Open-Source-Software eingesetzt wird.

Mithilfe regulärer Ausdrücke können die aus dem `grep`-Kommando bekannten Suchmuster ergänzt bzw. verfeinert werden. Das `grep`-Kommando gibt z.B. immer den Exit-Status „0“ zurück, wenn das Suchmuster an *irgendeiner* Stelle einer Zeile gefunden wurde. Möchte man jedoch nur nach solchen Zeilen suchen, in denen das Suchmuster nur am Beginn oder am Ende einer Zeile vorkommt, so muss das Suchmuster erweitert werden.

Vorkommen des Suchmusters nur am Beginn einer Zeile: `grep '^suche' datei`

Vorkommen des Suchmusters nur am Ende einer Zeile: `grep 'suche$' datei`

Daraus ergibt sich für die Suche nach einer vollständigen Zeile: `grep '^suche$' datei`

Sollen die Zeichen `^` oder `$` selbst in der Datei gesucht werden, so müssen sie geschützt werden:

```
grep '\$' datei
```

Das Einklammern des Suchwortes in einfache, gerade Anführungszeichen ist auch immer dann erforderlich, wenn im Suchmuster Leerzeichen vorkommen, da nur das erste Argument hinter dem Kommando als Muster interpretiert wird und alle weiteren Argumente als Dateinamen: `grep 'Mathematik A' datei`

Das Zeichen „.“ im Suchmuster steht als Platzhalter für *genau ein* beliebiges Zeichen:

```
grep 'Math.matik' datei
```

findet außer „Mathematik“ auch alle „Tippfehlerworte“ wie z.B. Mathimatik, Mathomatik usw. jedoch nicht Mathmatik.

Das Zeichen „*“ bezieht sich dagegen auf das davor stehende Zeichen, das beliebig oft (auch gar nicht) im Suchwort vorkommen darf. `grep 'Math*ematik' datei` hier bezieht sich * auf das davorstehende `h` findet also Mathematik, Matematik, Mathhhhematik, aber nicht Mathomatik.

Die Kombination „.*“ erfasst beliebig viele Zeichen: `grep 'Math.*matik' datei` findet Mathematik oder Mathimatik usw. aber auch z.B. Matheeeematik, Mathabcdefmatik.

Soll ein Zeichen aus einem bestimmten Bereich stammen, so muss dieser Bereich in eckige Klammern gesetzt werden:

```
grep '[a-z]' Bereich zwischen dem kleinen a und dem kleinen z (einschließlich a und z)
```

```
grep '[A-Z]' Bereich zwischen dem großen A und dem großen Z
```

```
grep '[A-z]' Bereich zwischen dem großen A und dem kleinen z (alle Buchstaben, aber Achtung: im ASCII-Code liegen zwischen den Groß- und den Kleinbuchstaben einige Sonderzeichen z.B. eckige Klammern, die damit ebenfalls gefunden werden, daher besser:
```

```
grep '[A-Za-z]'
```

```
grep '[1-F]' Bereich zwischen 1 und dem großen F, Achtung: zwischen 9 und A liegen im ASCII-Code einige Sonderzeichen z.B. „=“ oder „?“, wenn Zahlen und die Großbuchstaben A-F gefunden werden sollen, besser
```

```
grep '[0-9A-F]'
```

```
grep '[0-9A-Za-z]' findet alle Zahlen und Buchstaben
```

```
grep '\w' (\w als Wortzeichen) hat die gleiche Bedeutung wie [0-9a-zA-Z_] in PCRE oder Java, in anderen Implementationen können hier jedoch auch andere Zeichen enthalten sein.
```

```
grep '\W' Die Umkehrung ist \W als Nicht-Wortzeichen, entspricht folgendem Ausdruck
```

```
grep '[^0-9A-Za-z_]' das Zeichen „^“ entspricht (innerhalb einer eckigen Klammer) einer Verneinung
```

Zeichen können auch anhand des 7-Bit-ASCII-Codes gefunden werden (nicht in „Basic RegEx“, daher Option `-P` (für PCRE) erforderlich) :

```
grep -P '\x7A' hinter \x steht im Beispiel der ASCII-Code 7A für ein kleines „z“ (oder 41 für ein großes „A“ oder 40 für das „@“-Zeichen)
```

Beispiele:

`grep 'Math[a-m]matik' datei` findet Mathematik, Mathematik, Mathematik, aber nicht Mathematik, da der Buchstabe zwischen dem h und dem m aus dem Bereich von a bis m stammen muss. Diese Klammerung kann mit „*“ ergänzt werden: `grep 'Math[a-m]*matik' datei` findet außerdem Matheemmatik oder Mathabcdefgmatik. Sinnvollere Beispiele lassen sich hierzu besser mit Zahlenbereichen darstellen:

`grep 'Mathematik [1-6]' datei` findet Mathematik 1, Mathematik 2 usw. aber nicht Mathematik 8.

Kombinationen sind möglich: `grep 'Mathematik [1-2C-Z]' datei` findet u.a. Mathematik 1 oder Mathematik D aber nicht Mathematik B.

Diese Klammersyntax kann auch negiert werden, d.h. es soll im Suchmuster ein bestimmter Zeichenbereich *nicht* gefunden werden: `grep 'Mathematik [^1-2C-Z]' datei` findet z.B. Mathematik 7 oder Mathematik A aber nicht Mathematik 2 oder Mathematik F.

Die mehrfache Bedeutung von „^“ ist hierbei verwirrend: `grep '^[^1-2A-Z]' datei` gibt alle Zeilen aus, die *nicht* mit 1 oder 2 oder den Großbuchstaben von A bis Z *beginnen*.

Reguläre Ausdrücke finden Zeichen und Zeichenkombinationen, jedoch keine Zahlbereiche. Möchte man z.B. einen Zahlbereich zwischen 10 und 47 eingrenzen, so kann man *nicht* `[10-47]` schreiben: dies findet den Zeichenbereich zwischen 0 und 4 sowie die 1 und die 7, also 0, 1, 2, 3, 4 und 7. Stattdessen muss das Ziffernformat genau bezeichnet werden: Die beiden Ausdrücke `grep '[1-3][0-9]'` und `grep '4[0-7]'` finden die Ziffern 1 bis 3 gefolgt von den Ziffern 0 bis 9 sowie eine 4 gefolgt von den Ziffern 0 bis 7. Diese beiden einzelnen Ausdrücke können mit einer Oder-Verknüpfung zusammengefasst werden. Der senkrechte Strich „|“ (das „Pipe-Zeichen“) verbindet zwei Ausdrücke (nur mit PCRE, daher Option -P erforderlich):

`grep -P '[1-3][0-9]|4[0-7]'`

Diese regEx findet jedoch sämtliche Zeilen, in denen die Zahlenfolgen zwischen 10 und 47 vorkommen, also z.B. auch **3479**, **1024** oder **99940**. Das Einschränken auf Wörter (Leerzeichen und Bindestriche sind Worttrennungen) ist mit `\b` möglich:

`grep -P '\b[1-3][0-9]\b|\b4[0-7]\b'` findet nur alleinstehende Zahlen zwischen 10 und 47

`grep '\B4[0-7]\B'` \B dreht die Logik um: es werden die Zahlen zwischen 40 und 47 gefunden, wenn Sie *nicht* an einer Wortgrenze stehen, also **34799** oder **11403**, jedoch nicht **43555** oder „A **42** 77“ oder **1240**, Kombinationen sind möglich:

`grep '\B4[0-7]\b'` findet **3046** oder **141** aber nicht „Z **43** T“ oder **447F**

`grep '\b4[0-7]\B'` findet **4630** oder **411** aber nicht „Z **43** T“ oder **XY46**

Hinweis zur Oder-Verknüpfung: Wenn mehrere Alternativen im Such-String gegeben sind, wird immer nach dem ersten Such-Erfolg abgebrochen und die Suche dann fortgesetzt, Beispiel:

`grep -P 'Maria|Marianne'` findet im Satz „Marianne und Maria gehen nach Hause“ folgendes „**Marianne** und **Maria** gehen nach Hause“, da nach dem ersten erfolgreichen Auffinden des ersten Such-Strings abgebrochen wird. Dreht man die Reihenfolge um

`grep -P 'Marianne|Maria'` wird gefunden „**Marianne** und **Maria** gehen nach Hause“

Eindeutiger ist in solchen Fällen die Beschränkung auf Wortgrenzen mit `\b`

`grep -P '\bMaria\b|\bMarianne\b'`

Das letzte Beispiel wird übersichtlicher, wenn Ausdrücke mithilfe runder Klammern gruppiert werden:

`grep -P '\b(Maria|Marianne)\b'`

Gruppierungen sind besonders dann sinnvoll einsetzbar, wenn Ausdrücke wiederholt werden sollen:

`grep -P '([0-1][0-9]|2[0-3])(:[0-5][0-9]){2}'` findet ein Zeitformat hh:mm:ss wobei hh nur zwischen 00 und 23 liegen darf, mm und ss zwischen 00 und 59. Die Ausdrücke für mm und ss sind gleich und können daher als Gruppe zusammengefasst und zweimal geprüft werden (dafür sorgt die 2 in geschweiften Klammern).

Die runden Klammern definieren eine sog. „einfangende Gruppe“, bei der automatisch ein gefundener Wert gespeichert wird (die erste runde Klammer ist Nr. 1, die zweite Nr. 2, die 23. Klammer die Nr. 23), der später in der regEx wiederverwendet werden kann:

`grep -P '([0-1][0-9]|2[0-3])(:[0-5][0-9])\2'` die zweite runde Klammer wird als Nr. 2 gespeichert und kann später im Ausdruck als `\2` wieder aufgerufen werden, hier werden also Zeitangaben gefunden, bei denen mm und ss den gleichen Wert haben (z.B. **23:59:59** oder **08:00:00**)

Die Suche nach mehrfachem Vorkommen eines Zeichens kann verfeinert werden, indem mithilfe geschweiften Klammern ein bestimmtes x-maliges Auftreten eines Zeichens oder einer Gruppe gefunden werden soll (die geschweiften Klammern müssen (bei Kommandos in der Shell, wie z.B. `grep`) mit dem Backslash \ geschützt werden):

`grep 'Math\{4\}ematik' datei` findet Mathhhhematik aber nicht Mathhematik.

Die geschweifte Klammer enthält die gewünschte Wiederholungszahl. Diese Zahl kann variabel sein, z.B. $A\{3, 8\}$ (dreimalige bis achtmalige Wiederholung von A) oder $A\{9, \}$ (mindestens neunmalige Wiederholung oder mehr), $A\{, 12\}$ (höchstens 12-malige Wiederholung). Der oben bereits beschriebene Ausdruck „ $*$ “ hat die Bedeutung „null oder mehr“, gleichbedeutend mit $A\{0, \}$. Gleichbedeutend mit $A\{1, \}$ ist A^+ und $A^?$ entspricht $A\{0, 1\}$.

Wie oben bereits erläutert, steht der Punkt „ $.$ “ als Platzhalter für ein beliebiges Zeichen; ein darauffolgender Stern „ $*$ “ findet also ein beliebig oft vorkommendes beliebiges Zeichen:

`grep -P '6.'` findet eine 6 gefolgt von einem beliebigen Zeichen

`grep -P '6.*'` findet eine 6 gefolgt von beliebig vielen beliebigen Zeichen

Es ist zu beachten, dass der Stern $*$ die Wiederholung (null oder *beliebig viele*) für das unmittelbar davorstehende Zeichen bedeutet, was zu folgendem Missverständnis führt:

`grep -P '<b>6.*</b>'` findet nicht einen fett gesetzten Bereich in einem HTML-Dokument (beginnend mit Tag `<b>`), der mit einer 6 beginnt, gefolgt von beliebig vielen Zeichen, abgeschlossen mit `</b>`, sondern die Zeichenfolge `<b>6` gefolgt von beliebig vielen beliebigen Zeichen; das abschließende `</b>` wird von der regEx überhaupt nicht mehr erreicht; es wird *nicht* wie gewünscht gefunden „`<b>6A </b>` irgend ein Text `<b>611</b>`“ sondern „`<b>6A </b>` irgend ein Text `<b>611</b>`“

Dieses sog. „gierige Verhalten“ des Stern-Operators (dies gilt ebenso für alle anderen Wiederholungsoperatoren) kann mit dem Fragezeichen $?$ verhindert werden:

`grep -P '<b>6.*?</b>'` findet wie gewünscht nur „`<b>6A </b>` irgend ein Text `<b>611</b>`“, was sehr wichtig ist, wenn (nur) diese fett gesetzten Bereiche in einem HTML-Dokument bspw. ersetzt werden sollen. Falls solche Bereiche (z.B. in HTML-, XML- oder Latex-Dokumenten) über mehrere Zeilen gehen, so muss der Single-Line-Modus aktiviert werden. Im regEx muss $(?s)$ eingesetzt werden, ggf. muss zusätzlich das den regEx verwendende Programm so eingestellt werden, dass Zeilenumbrüche ignoriert werden (bei `grep` die Option `-z`):

`grep -Pz '(?s)<b>6.*?</b>'`

Ein Einsatzgebiet von regulären Ausdrücken ist die Überprüfung von Benutzereingaben in Web-Formularen. Soll in einem Textfeld ein Name eingegeben werden, so kann geprüft werden, ob der erste Buchstabe groß geschrieben ist, gefolgt von einer beliebigen Anzahl von Kleinbuchstaben: `'\b[A-Z][a-z]+'` oder mit einer Begrenzung auf eine maximale Anzahl von Zeichen `'\b[A-Z][a-z]{1,30}'`. Im deutschen Text müssten Umlaute hinzugefügt werden, außerdem muss ein ggf. vorkommender Bindestrich bei Doppelnamen berücksichtigt werden.

Telefonnummern prüfen (internationales Format z.B. +49 123 456789)

`grep -P '^([0-9]\x20?){6,14}[0-9]$'` Zeile beginnt („ $^$ “) mit einem „ $+$ “ (das mit einem Backslash geschützt werden muss, danach folgen Zahlen (gefolgt von einem oder keinem (das Fragezeichen „ $?$ “) Leerzeichen, ASCII-Code: `\x20`). Am Ende der Zeile („ $$$ “) steht eine Zahl (damit Eingaben nicht mit Leerzeichen beendet werden). Man könnte auch ohne die Gruppierung schreiben: `[0-9\x20]`, aber dann wäre auch eine Zeile zulässig, die nur aus Leerzeichen besteht.

Das Prüfen des Formats einer Mail-Adresse (z.B. Vorname.Nachname@haw-hamburg.de) zeigt die Schwierigkeit im Umgang mit regulären Ausdrücken: Es muss sehr genau formuliert werden, was gesucht wird. Im einfachsten Fall könnte man lediglich prüfen, ob zulässige Zeichen vor und nach einem `@`-Zeichen vorhanden sind:

`[A-Za-z0-9._-]+@[A-Za-z0-9._-]+` was jedoch unsinnige Eingaben wie `__ABC.....@-----...123`

zulassen würde. Die Adresse darf nur mit Wortzeichen (`\w`) beginnen und immer nur einzelne Punkte oder Bindestriche enthalten, zudem sollte geprüft werden, ob die Adresse mit mindestens zwei und höchstens sechs Buchstaben endet (Top-Level-Domains). Der folgende Ausdruck stellt *keine* perfekte Syntax-Prüfung einer realen Mail-Adresse mit allen Erfordernissen dar, prüft aber immerhin die Grundanforderungen:

`^\w+([.-]? \w+)+@ \w+([.-]? \w+)+\.[A-Za-z]{2,6}$`

Die zuvor gezeigten Beispiele im Zusammenhang mit dem Kommando `grep` zeigen erste, grundlegende Beispiele für die Verwendung regulärer Ausdrücke. Mit `grep` lassen sich sehr gut Dateien bei Nutzung eines Terminalfensters durchsuchen, allerdings eignet sich `grep` nicht für die Erstellung komplexerer Suchabläufe in Shell-Skripten oder die Manipulation von Dateien. Hierfür stehen die Kommandos `awk` und `sed` zur Verfügung.

2.2 Suchen und Ersetzen mit sed

Das wesentliche Einsatzgebiet des Stream-Editors sed ist das Suchen und Ersetzen von Zeichenketten in einer Datei. Die Suche erfolgt mittels regulärer Ausdrücke. Eine Datei wird immer zeilenweise eingelesen und bearbeitet wobei der Dateinhalt zunächst in einem Puffer (sog. „Patternspace“) geschrieben und nur der Inhalt dieses Puffers von sed bearbeitet wird und nicht die Originaldatei selbst. Die Ausgabe erfolgt entweder mittels STDOUT oder in eine neue Datei.

Syntax: `sed 'Kommando' Datei` z.B. `sed -n 'p' dat.txt`

Hierbei werden alle Zeilen der Datei `dat.txt` auf STDOUT ausgegeben (Kommando `p` für print); die Option `-n` unterdrückt die Ausgabe des gesamten Patternspace (dies ist das Standardverhalten von `sed`).

Die Ausführung des Kommandos kann auf bestimmte Bereiche (sog. Adressen) beschränkt werden. Diese Bereiche können z.B. Zeilennummern sein oder mittels regulärer Ausdrücke festgelegt werden, indem nach Zeichenketten gesucht und das Kommando dann nur auf diejenigen Zeilen angewendet wird, in denen die Zeichenkette gefunden wurde:

`sed 'Adresse Kommando' Datei`

```
sed -n '3 p' dat.txt      Ausgabe der dritten Zeile
sed -n '3,$ p' dat.txt   Ausgabe von der dritten Zeile bis zum Ende der Datei
sed -n '/Mathe/p' dat.txt Ausgabe aller Zeilen, in denen die Zeichenkette „Mathe“ vorkommt.
sed -n '/M[0-9]/p' dat.txt Ausgabe aller Zeilen, in denen auf das Zeichen „M“ eine Zahl folgt
```

Es können auch mehrere Kommandos innerhalb von geschweiften Klammern angegeben werden, z.B.

```
sed -n '3,7{p ; s/Informatik/Malen nach Zahlen/gp}' dat.txt
```

Ausführung von zwei Kommandos auf die Zeilen 3 bis 7: zuerst Ausgabe der ganzen Zeile (Kommando `p`), dann Ersetzen (Kommando „substitute“ `s`) der Zeichenkette „Informatik“ durch „Malen nach Zahlen“; das Substitutions-Flag `g` (global) wendet die Ersetzung auf alle Vorkommen der Zeichenkette in einer Zeile an (sonst würde nur das erste Vorkommen ersetzt werden) und ein erneutes „print“ `p`-Flag gibt die Änderung auf STDOUT aus. Statt der Ausgabe auf STDOUT kann auch in eine neue Datei geschrieben werden (Flag `w`):

```
sed -n '3,7s/Informatik/Malen nach Zahlen/gw ausgabe.txt' dat.txt
```

Das Kommando bzw. Flag `w` kann mit geschweiften Klammern nur dann kombiniert werden, wenn hinter dem Dateinamen ein Newline folgt (hinter dem Dateinamen Enter eintippen; bei dem „>“ handelt es sich um das automatisch erscheinende Zeichen für den Zeilenanfang, es muss *nicht* eingetippt werden!):

```
sed -n '3,7{p; s/Informatik/Malen nach Zahlen/gw ausgabe.txt
> }' dat.txt
```

Mit der Option `-i` (inline) kann eine Änderung auch direkt in der Original-Datei ausgeführt werden (diese Option ist allerdings keine Standard-Option und funktioniert i.d.R. nur mit GNU-sed unter Unix/Linux)

```
sed -i.bak '17d' dat.txt
```

Es wird die Zeile 17 in `dat.txt` gelöscht (Kommando „delete“ `d`), wobei eine Backup-Datei mit dem Namen `dat.txt.bak` angelegt wird (dies ist optional; die Dateierweiterung kann einen beliebigen Namen haben und muss direkt hinter dem `-i` stehen, also ohne Leerzeichen).

Statt des `g`-Flags kann auch eine Zahl `n` eingesetzt werden, sodass nur das `n`-te Vorkommen des Suchtextes ersetzt wird (kann auch mit dem `g`-Flag kombiniert werden, dann wird ab dem `n`-ten Vorkommen ersetzt).

```
sed -n '10,{s/Projekt/Labor/2 ; s/Projekt/Seminar/3 w ausgabe
> }' dat.txt
```

In diesem Beispiel wird ab der zehnten Zeile bis zum Ende der Datei `dat.txt` zunächst das zweite Vorkommen des Wortes „Projekt“ in jeder Zeile gesucht, durch „Labor“ ersetzt und das Ergebnis in den Patternspace geschrieben. Nun wird *dieses Ergebnis* mit dem zweiten Kommando erneut nach „Projekt“ durchsucht, aber diesmal beim dritten Vorkommen durch „Seminar“ ersetzt. Danach wird dieser zweimal geänderte Patternspace in die Datei „ausgabe“ geschrieben. Man beachte: Nach dem ersten Ersetzen ist das ursprünglich zweite Vorkommen des Wortes „Projekt“ nun durch „Labor“ ersetzt, sodass nun im folgenden Kommando das dritte Vorkommen von „Projekt“ in dieser bereits *geänderten* Zeile gesucht wird und *nicht* in der ursprünglichen Zeile. Eine ähnliche Kommando-Syntax wie bei „substitute“ `s` findet sich bei „yank“ `y` (Ersetzen eines Zeichens aus einer Liste durch ein anderes)

```
sed 'y/abc/xyz/' dat.txt
```

Hiermit wird jedes „a“ durch ein „x“ ersetzt, ein „b“ durch ein „y“ und ein „c“ durch ein „z“. Zu beachten ist, das

die Liste der Quellzeichen genauso lang sein muss wie die Liste der Zielzeichen.

Mit dem Kommando „append“ `a` wird hinter einer selektierten Zeile eine neue Zeile eingefügt, mit „insert“ `i` davor und mit „change“ `c` wird die selektierte Zeile selbst ersetzt.:

```
sed '/Gestaltung/ a Inhalt der neuen Zeile' dat.txt
```

Für `a`, `c` bzw. `i` gilt bei Verkettung mehrerer Kommandos in geschweiften Klammern das gleiche wie beim Kommando „write“ `w` (Newline nach dem neuen Zeileninhalt, s.o.).

„Spezialitäten“: Arbeiten mit dem sog. „Holdspace“. Eine Zeile aus einer Datei wird immer in den Patternspace geladen und dort verarbeitet. Möchte man aber eine Zeile im ursprünglichen Zustand erhalten, um sie zu einem späteren Zeitpunkt der Bearbeitung (also in einer weiter unten in der Originaldatei stehenden Zeile) wiederzuverwenden, so kann der Holdspace für die Zwischenspeicherung verwendet werden. Beim Laden und Zurückholen einer Zeile in den bzw. aus dem Holdspace ist zu unterscheiden, ob mit oder ohne „Newline“ gearbeitet werden soll.

```
sed -n '{/AV-Technik/{h;d} ; /Informatik/G ; p}' dat.txt
```

Eine Zeile, die „AV-Technik“ enthält, wird zunächst ohne Newline in den Holdspace geschrieben (`h`) und anschließend gelöscht (`d`); enthält im späteren Verlauf der Bearbeitung eine Zeile das Wort „Informatik“, so wird mit `G` der aktuelle Inhalt des Holdspace zurückgeschrieben, wobei `G` zunächst ein führendes Newline einsetzt, sodass die Zeile mit „AV-Technik“ nun hinter der aktuellen Zeile, die „Informatik“ enthält, eingefügt wird. In diesem Beispiel wird jedoch *ohne* Newline in den Holdspace geschrieben; die Folge ist, dass der Inhalt des Holdspace bei jedem erneuten Auftreten von „AV-Technik“ überschrieben wird, sodass hier zwar jede Zeile mit dem Suchwort „AV-Technik“ gelöscht wird aber nur die jeweils letzte gefundene Zeile in den Holdspace geschrieben wird.

```
sed -n '{/AV-Technik/{H;d} ; /Informatik/g ; p}' dat.txt
```

In diesem „umgedrehten Beispiel“ wird mit `H` eine Zeile mit „AV-Technik“ in den Holdspace geschrieben, diesmal aber mit einem führendem Newline, sodass nun diese Zeile an einen bereits existierenden Inhalt des Holdspace angehängt wird. Durch das fehlende Newline beim Kommando `g` wird hier jedoch die gefundene Zeile mit dem Inhalt „Informatik“ mit dem Inhalt des Holdspace überschrieben (Hinweis: ist der Holdspace leer, so wird die gefundene Zeile nicht gelöscht sondern mit einer leeren Zeile überschrieben).

Als weiteres Kommando gibt es noch „exchange“ `x`, mit dem der Inhalt des Patternspace mit dem Inhalt des Holdspace ausgetauscht werden kann.

Mit dem Kommando „read“ `r` kann der Inhalt einer externen Datei eingelesen werden:

```
sed '10r datei1' dat.txt
```

Nach der Zeile Nr. 10 wird der Inhalt der Datei „datei1“ eingefügt. Natürlich kann auch hier mit Suchmustern und regulären Ausdrücken gearbeitet werden und es ist wieder das „Newline“ nach dem Dateinamen zu verwenden, falls mehrere Kommandos verkettet werden sollen.

2.3 Grundlagen von *awk*

Das spezielle Anwendungsgebiet von *sed* ist das Durchsuchen und die Manipulation von Textdateien mithilfe regulärer Ausdrücke. Im Prinzip leistet *awk* (benannt nach den Anfangsbuchstaben der Entwickler) ähnliches, bietet jedoch insbesondere in Bezug auf reguläre Ausdrücke erheblich mehr. Wie auch *sed* liest *awk* Daten aus Dateien (oder von der Standardeingabe) ein und bearbeitet die Daten zeilenweise.

Die grundlegende Kommandostruktur ist

```
awk 'Suchmuster {Kommando}' eingabe_datei
```

awk bearbeitet eine Zeile spaltenweise, d.h. die einzelnen Elemente einer Zeile, die mit einem beliebigen Zeichen getrennt sein können (Standard ist das Leerzeichen), werden einzeln abgefragt bzw. mit dem Suchmuster verglichen.

```
awk '$2 ~ "Mathematik" {print $4}' eingabe_datei
```

Da die Beispieldatei „Module_MS.txt“ als Trennzeichen den senkrechten Strich verwendet, muss dieses Zeichen mit der Option `-F` angegeben werden (damit die Shell den senkrechten Strich aber nicht als „Pipe“ interpretiert, muss

dieses Zeichen durch doppelte Anführungszeichen geschützt werden)

```
awk -F"|" ' $2 ~ "Mathematik" {print $4}' eingabe_datei
```

Das `$`-Zeichen wird verwendet, um das gewünschte Element (die Spalte) abzurufen, d.h. hier wird das zweite Feld bzw. die zweite Spalte mit dem Suchmuster „Mathematik“ verglichen (Operator `~`). Das Kommando `print` gibt diejenige Zeile aus, in der das Muster gefunden wurde; hier wird die Ausgabe jedoch auf das vierte Element der Zeile (`$4`) beschränkt.

Bei vielen Konfigurationsdateien handelt es sich um spaltenweise organisierte Textdateien, die sich mit `awk` sehr gut durchsuchen lassen. Beispielsweise könnte man die in der Resolver-Konfigurationsdatei (Nameservice) `/etc/resolv.conf` eingetragenen IP-Adressen der erreichbaren Nameserver abfragen:

```
awk ' $1 ~ "nameserver" {print $2}' /etc/resolv.conf
```

Hier wird in der ersten Spalte jeder Zeile das Schlüsselwort „nameserver“ gesucht und bei Auftreten dieses Wortes der Inhalt der zweiten Spalte (die IP-Adresse der Nameserver) ausgegeben. Verwendet man dieses `awk`-Kommando beispielsweise innerhalb eines Shell-Skripts, so könnte mit diesen Ergebnissen eine Konfigurationsdatei eines Netzwerkgerätes oder einer Software automatisch aktualisiert werden.

Im Suchwort können auch die bereits bekannten grundlegenden regulären Ausdrücke verwendet werden:

```
awk -F: '$6 ~ /^\/home/ && $NF ~ /bash$/ {print $1}' /etc/passwd
```

In der Datei `/etc/passwd` (die Doppelpunkte als Trennzeichen enthält) wird der Benutzername (`$1`) immer dann ausgegeben, wenn die sechste Spalte (das Home-Verzeichnis) mit „/home“ beginnt (schützen des Schrägstrichs im Suchwort mit dem Backslash `\` erforderlich sowie Klammerung des Suchmusters in /Schrägstriche/) und (`&&`) die letzte Spalte (`$NF` Anzahl der Spalten) mit „bash“ endet (hiermit erhält man eine Liste mit allen normalen Benutzerkennungen (UserID) im Unix-System), die die Bash als Login-Shell haben. Ein Oder-Vergleich ist mit `||` möglich.

Ein ähnliches Ergebnis erhält man bei der Suche nach allen UserIDs, die eine sog. UID größer als oder gleich 1000 haben:

```
awk -F: '$3 >= 1000 {print $1 " " $(NF-1)}' /etc/passwd
```

Oder Suche nach einem Bereich

```
awk -F: '$3 ~ /[1000 1008]/ {print $1 " " $(NF-1)}' /etc/passwd
```

Die Ausgabe der ersten (`$1`) und vorletzten (`$(NF-1)`) Spalte wird hier mit einem Leerzeichen getrennt. Die Vergleichsoperatoren (z.B. `<`, `>`, `<=`, `=`, `!=`) können auch auf Buchstaben angewendet werden, wobei der ASCII-Wert verglichen wird. Auch wenn die Ausgabe von Daten mit dem `print`-Kommando mit Tabulatoren „`\t`“ oder Newline „`\n`“ noch besser formatiert werden kann, so ist „`print`“ für tabellarische Ausgaben weniger geeignet. Das `printf`-Kommando erlaubt wesentlich detailliertere Ausgabe-Formatierungen:

```
awk -F: '$3 >= 1000 {printf "Benutzer %-15s hat die UID:\t %5d\n", $1,$3}' /etc/passwd
```

Es wird der Inhalt der ersten Spalte linksbündig in ein Feld mit einer Breite von 15 Zeichen (String `s`) (`%-15s`) geschrieben, danach folgt der String „hat die UID:“ und ein Tab sowie ein fünf Zahlen (Digit `d`) breites Feld, rechtsbündig geschrieben (`5d`). Die Prozent-Zeichen beziehen sich in dieser Reihenfolge auf die am Ende des `printf`-Kommandos angegebenen Spalten, d.h. an die Stelle des ersten Prozent-Zeichens wird der Inhalt von `$1` geschrieben und an die Stelle des zweiten Prozent-Zeichens der Inhalt von `$3`.

Die Ausgabe kann auch in eine Datei geschrieben werden:

```
... {print $1 "\t" $3 > "/home/user/datei"}
```

oder an eine existierende Datei angehängt werden:

```
... {print $1 "\t" $3 >> "/home/user/datei"}
```

Die Erzeugung dynamischer Dateinamen z.B. aus den Spaltenwerten ist möglich (hier wird für die UID aus Spalte 3 bei jedem `print`-Kommando ein Dateiname „dateiUID“. (z.B. `/home/user/datei1015`) angelegt):

```
... {print $1 "\t" $3 >> "/home/user/datei"$3}
```

Soll oder darf ein Shellskript nur dann ausgeführt werden, wenn eine bestimmte Betriebssystem- oder Kernel-Version vorliegt (weil z.B. nur dann bestimmte Verzeichnisse oder Programmversionen existieren), so muss zuvor diese Version herausgefunden werden; hierfür gibt es verschiedene Möglichkeiten. Für den Kernel:

```
uname -a | awk '{print $3}'
hostnamectl | awk -F: '$1 ~ "Kernel" {print $2}'
```

Für das Betriebssystem:

```
hostnamectl | awk -F: '$1 ~ "Operating" {print $2}'
cat /etc/os-release | awk -F= '$1 ~ "NAME" || $1 ~ "VERSION" {print $2}'
```

Da die Datei `os-release` mehrere Zeilen mit „NAME“ oder „VERSION“ enthält, muss die Suche auf diejenigen Zeilen begrenzt werden, die genau diese Wörter enthalten:

```
cat /etc/os-release | awk -F= '$1 ~ "^NAME$" || $1 ~ "^VERSION$" {print $2}'
```

Hardware-Adresse(n) der Netzwerkkarte(n) auslesen (sog. MAC-Adressen):

```
/sbin/ifconfig | grep eth | awk '{print $1 "\t" $5}'
```

oder ohne die Vorfilterung mittels `grep`:

```
/sbin/ifconfig | awk '$1 ~ "eth" {print $1 "\t" $5}'
```

Ausgabe des belegten bzw. noch freien Platzes (in kB) auf der Festplatte mit dem Kommando `df`. Folgendes Kommando sucht nur die home-Partition und gibt den freien Plattenplatz in MB an:

```
df | awk '$6 ~ /^\/home$/ { print $6 ": Füllgrad " $5 " , verfügbar " $4/1024 " MB" }'
```

```
ps aux | awk '$3 >= 0.1 {ORS=" "; print $1; for(i=11; i<=NF; i++){print $i} ORS="\n"; print ""}'
```

Der „Output Record Separator“ `ORS` ist standardmäßig Newline `\n`. Dadurch werden Ausgaben z.B. aus einer for-Schleife nicht nebeneinander in einer Zeile ausgegeben sondern untereinander, was meist nicht erwünscht ist. In diesem Fall kann der `ORS` im `awk`-Skript geändert werden, wobei darauf zu achten ist, dass nun aber auch nach Bearbeitung einer Zeile kein Newline in der Ausgabe verwendet wird und zusätzlich mittels Rücksetzen des `ORS` auf „`\n`“ eingefügt werden muss (das `print ""` ist aus irgendeinem Grund wichtig...??).

```
ps -u | awk '{for(i=1; i<=10; i++){printf "%7s", $i; ORS=" ";} print " "; for(i=11; i<=NF; i++){print $i}; ORS="\n"; print ""}'
```

Besteht eine herausgefilterte Zeichenkette aus mehreren Elementen, die z.B. einzeln oder separat voneinander in einer `print`-Ausgabe verwendet werden sollen, so ist die Funktion „`split`“ nützlich

Im folgenden Beispiel besteht `$1` aus der Zeichenkette „xyz:123:ABC“. Die einzelnen, durch Doppelpunkt getrennten Werte sollen jedoch separat ausgegeben werden

```
awk ' $1 ~ "xyz" {split($1,a,":"); print a[2] " und " a[0] }'
```

Die Funktion „`split`“ wertet die Zeichenkette `$1` aus und trennt sie anhand des Doppelpunktes; die Einzelteile werden dann in einem Array `a` gespeichert und können als `a[0]` usw. weiterverwendet werden.

3 Shell-Programmierung

3.1 Grundlagen

Eine „Shell“ ist eine nicht-grafische Benutzerschnittstelle zum Betriebssystem. In einem Terminal-Fenster (auch Konsole genannt) können Kommandos zum Start von Programmen oder Systemaufrufen eingegeben werden; die Ausgabe bzw. die Rückgabewerte dieser Programme werden z.B. in Textform im Terminalfenster dargestellt. Das Unix-Betriebssystem kennt verschiedene Varianten von Shells (u.a. Bourne-Shell `sh`, Bourne-again-Shell `bash`, Korn-Shell `ksh`, Z-Shell `zsh`), aber auch unter Windows/DOS ist eine solche Kommandoeingabe (ohne GUI) mit der `cmd.exe` oder der Windows PowerShell in vergleichbarer Form möglich.

Die folgenden Darstellungen beziehen sich im Wes. auf die Shell `sh`, teilweise werden auch spezielle `bash`-Schreibweisen ergänzend aufgeführt.

Variablen direkt in der Kommando-Shell: Einer Variablen (hier „Name“) kann ein Wert (hier „Paula“) zugewiesen werden. Die Variable kann dann mit vorangestelltem Dollarzeichen wieder aufgerufen werden

```
user@rechner:~/verzeichnis> Name=Paula
user@rechner:~/verzeichnis> echo $Name
Paula
```

Die Anzahl der Zeichen des aktuellen Variablen-Inhalts:

```
user@rechner:~/verzeichnis> echo ${#Name}
```

Umgebungsvariablen: Im Betriebssystem werden diverse Variablen beim Start definiert und können im laufenden Betrieb verändert werden. Den aktuellen Stand dieser sog. Umgebungsvariablen zeigt das Kommando `env`. Diese vorgegebenen Variablenamen werden meistens in Großbuchstaben geschrieben:

```
user@rechner:~/verzeichnis> echo $USER    (Ausgabe des Benutzernamens)
user@rechner:~/verzeichnis> echo $HOME    (Ausgabe des Namens des Home-Verzeichnisses des aktuellen Benutzers)
```

```
user@rechner:~/verzeichnis> echo $PATH    (Pfadvariable: alle Programme in diesen Verzeichnissen können direkt mittels Eingabe des Programmnamens gestartet werden)
```

Manchmal ist es sinnvoll, die Pfad-Variable zu erweitern. Beispiel: Man hat das Shell-Skript „hello“ im Verzeichnis `/home/user/skripte` gespeichert. Zum Ausführen dieses Skriptes muss man entweder in dieses Verzeichnis wechseln oder bei jedem Aufruf den kompletten Pfad eingeben

```
user@rechner:~/verzeichnis> sh /home/user/skripte/hello
```

Oder abgekürzt (sofern `hello` auf „ausführbar“ gesetzt ist):

```
user@rechner:~/verzeichnis> ~/skripte/hello
```

Erweitert man nun jedoch die Pfad-Variable um dieses Verzeichnis, so kann das Skript aus jedem Verzeichnis heraus nur durch die Angabe des Skript-Namens gestartet werden. Vorsicht ist allerdings geboten, wenn eigene Skripte denselben Namen wie Systemkommandos haben, da deren Ausführung dann ggf. nicht mehr möglich ist. Mit der folgenden Zuweisung wird den Pfaden in der existierenden Variablen (`$PATH`) der gewünschte Pfad vorangestellt. Achtung: ohne das „`:`“ `$PATH`“ würde die neue `PATH`-Variable nur noch aus dem eigenen Verzeichnisnamen bestehen und ein Ausführen von Systemkommandos wäre nur noch durch den Aufruf inklusive der geeigneten Verzeichnisse möglich!

```
user@rechner:~> PATH=/home/user/skripte:$PATH
```

Zu beachten ist allerdings, dass Variablenzuweisungen ausschließlich in der aktuellen Shell-Umgebung oder zur Laufzeit eines Skriptes gültig sind und danach wieder freigegeben werden.

Wenn das Skript „hello“ eine Variable ausgeben soll, so ist dies nur mit denjenigen Variablen möglich, die innerhalb bzw. zur Laufzeit dieses Skriptes auch dort definiert wurden.

Beispiel: das Skript `hello` enthält nur die Zeile „`echo $Name`“

```
user@rechner:~> Name=Paula
```

```
user@rechner:~> hello
```

Die Ausgabe ist nun leer, weil das Skript „hello“ innerhalb einer neuen Sub-Shell ausgeführt wird und diese Sub-Shell hat keinen Zugriff auf die Variablen der übergeordneten Shell.

Daher ist es erforderlich die Variable zu exportieren bzw. an eine Subshell zu vererben:

```
user@rechner:~> export Name
```

```
user@rechner:~> hello    (nun Ausgabe des Inhalts „Paula“ der Variablen „Name“)
```

Das `Export`-Kommando ist auch dann wichtig, wenn man innerhalb eines Skriptes ein weiteres Skript aufrufen möchte. Soll dieses zweite Skript ebenfalls den aktuellen Wert der Variablen kennen, muss diese Variable innerhalb

des ersten Skriptes exportiert werden (Alternative `typeset -x`, damit lässt sich ein Export auch wieder rückgängig machen: `typeset +x`)

(Zur Info: Sollen Variablen auch nach einem Rechner-Neustart, nach dem Aus- und wieder Einloggen oder einfach beim Start eines neuen Terminalfensters wieder zur Verfügung stehen, so müssen sie in der Datei `~/.bashrc` eingetragen werden.)

Die umgekehrte Richtung funktioniert so jedoch nicht: Wird innerhalb eines Skriptes eine Variable gesetzt oder verändert, so wird der geänderte Wert nicht an die übergeordnete Eltern-Shell zurückgegeben. Ist dies jedoch erwünscht, so kann mit dem Kommando `source` (vor dem eigentlichen Skriptnamen) veranlasst werden, dass das Skript in der aktuellen Shell und nicht in einer Subshell ausgeführt wird. Dadurch sind die im Skript geänderten Variablen auch in der ausführenden Shell anschließend verfügbar.

Das Skript `hello` enthält die Zeilen: `echo $Name`, `Name=Claudia` und `echo $Name`

```
user@rechner:~> export Name
```

```
user@rechner:~> hello
```

```
user@rechner:~> echo $Name
```

Das Skript `hello` gibt zuerst „Paula“ aus, dann den geänderten Wert „Claudia“, während der folgende `echo`-Befehl wieder „Paula“ ausgibt. Wird stattdessen jedoch der `source`-Befehl verwendet (`source hello`) so erhält man mit dem `echo`-Befehl nun den im Skript geänderten Namen „Claudia“ (statt `source` kann man auch einfach einen Punkt `.` verwenden). Interessant ist der `source`-Befehl dann, wenn Umgebungsvariablen des Systems durch ein Skript angepasst werden sollen (z.B. Spracheinstellungen).

Variablenamen können (immer) auch in geschweiften Klammern geschrieben werden, was insbesondere dann erforderlich ist, wenn Variablenamen zusammen mit Zeichenketten verwendet werden:

```
user@rechner:~> Dateiname=bla
```

```
user@rechner:~> echo $Dateiname_15
```

Die erwünschte Ausgabe „bla_15“ erfolgt hier jedoch nicht, da „Dateiname_15“ als Variable interpretiert wird, die aber gar nicht existiert. In diesem Fall müssen geschweifte Klammern verwendet werden:

```
user@rechner:~> echo ${Dateiname}_15
```

Hinweis: Eine Übernahme von Variablen und ihrer aktuellen Werte in neue Terminalfenster oder der Erhalt von Variablen auch bis zum nächsten Login des Benutzers, ist nicht möglich. Variablen können jedoch fest definiert werden, indem sie in bestimmte Konfigurationsdateien eingetragen werden (je nach Distribution und Shell verschieden; für die `bash`: für jeden Benutzer individuell in der Datei `.bashrc` und systemweit in `/etc/profile.local`)

3.2 Rechenoperationen

Shell-Skripte wurden nicht für die Ausführung von Rechenoperationen entwickelt und verfügen daher auch nur über wenige arithmetische Funktionen. Da es trotzdem aber häufig Situationen geben kann, in denen Variablenwerte miteinander verrechnet werden müssen, seien hier kurz einige arithmetische Funktionen dargestellt.

Rechnen mit Integer-Werten:

```
user@rechner:~> varA=4
```

```
user@rechner:~> varB=5
```

```
user@rechner:~> expr $varA + $varB    (mit Leerzeichen vor und hinter dem „+“: die Ausgabe ist „9“ )
```

```
user@rechner:~> expr $varA+$varB    (ohne Leerzeichen vor und hinter dem „+“: die Ausgabe ist „4+5“ )
```

```
user@rechner:~> expr $varA / $varB    (es wird gerundet: Ausgabe „1“)
```

```
user@rechner:~> expr $varA * $varB    (Ausgabe: „expr: Syntaxfehler“, da hier ein „Wildcard-Stern“ interpretiert wird und kein Mal-Zeichen), stattdessen muss das *-Zeichen mit dem Backslash \ geschützt werden
```

```
user@rechner:~> expr $varA \* $varB    (Ausgabe „20“)
```

Noch umständlicher werden Ausdrücke mit Klammern, da auch die Klammern geschützt werden müssen und unbedingt auf Leerzeichen vor und hinter den Zahlwerten zu achten ist:

```
user@rechner:~> expr \( 5 + 5 \) \* 4    (Operation (5+5)*4 , Ausgabe „40“)
```

Soll das Ergebnis der Rechenoperation in einer weiteren Variablen gespeichert werden, so muss der `expr`-Ausdruck in umgekehrte einfache Anführungszeichen (Gravis-Zeichen) gesetzt werden (sog. Kommando-Substitution).

```
user@rechner:~> ergebnis=`expr \( 5 + 5 \) \* 4`
```

In der `Bash` ist die Arithmetik-Syntax etwas übersichtlicher, allerdings sollte beim Schreiben von Shell-Skripten bedacht werden, dass ein Skript evtl. auch in anderen Umgebungen (also Shells) ausgeführt werden könnte, was mit spezieller `Bash`-Syntax dann nicht mehr funktionieren würde; das Shell-Skript wäre dann nicht mehr allgemein verwendbar.

Nur in der Bash:

Mit einer doppelten runden Klammerung wird eine mathematische Operation definiert:

```
user@rechner:~> ( (y= (5+3) *4) )
```

```
user@rechner:~> echo $y      (Ausgabe „32“)
```

Ohne Zwischenspeicherung des Ergebnisses in einer Variablen:

```
user@rechner:~> echo $ ( ( (19-3) /4) )      (Ausgabe „4“)
```

Allerdings ist auch diese Doppel-Klammerung nicht besonders handlich, sodass es alternativ das `let`-Kommando gibt:

```
user@rechner:~> let y=(19-3) /4
```

```
user@rechner:~> echo $y
```

Insbesondere in komplexen Bash-Skripten wird der Umgang mit Rechenoperationen etwas übersichtlicher, wenn eine Variable direkt als Integer-Zahl deklariert wird. Es genügt, wenn die Ergebnis-Variable mit `typeset -i` als Integer-Variable deklariert wird

```
user@rechner:~> typeset -i y
```

```
user@rechner:~> varA=3; varB=6
```

```
user@rechner:~> y=$varA*$varB
```

```
user@rechner:~> echo $y      (Ausgabe „18“)
```

Weitere Operatoren für das Integer-Rechnen entsprechen im Wesentlichen der üblichen Syntax von anderen Programmiersprachen u.a.: „%“ (Modulorechnen), „+=“, `var>>n` (bitweiser Rechts-Shift um n Bit), „&“, „^“, „|“, „~“ (bitweises UND, XOR, ODER, NEGATION).

Für umfangreichere Rechenoperationen (auch mit Fließkomma-Zahlen) kann der „Unix-Taschenrechner“ `bc` verwendet werden. An dieser Stelle nur ein kurzes Beispiel:

```
user@rechner:~> varA=3.17; varB=6.09
```

```
user@rechner:~> y=`echo "scale=3; $varA*$varB" | bc`
```

```
user@rechner:~> echo $y
```

Für den Informatik-Bereich interessant: Umrechnung zwischen Zahlensystemen (hier wird ein binärer Eingabewert in einen Hexadezimalen Ausgabewert umgerechnet):

```
user@rechner:~> y=`echo "obase=16; ibase=2; 101010" | bc
```

```
user@rechner:~> echo $y      (Ausgabe „2A“)
```

Arrays

Das Anlegen und Verarbeiten von (nur eindimensionalen) Arrays ist nur in der bash und der Korn-Shell möglich und hat zudem in beiden Shells eine unterschiedliche Syntax; die hier gezeigten Beispiele funktionieren daher ausschließlich in der bash.

Die Wertezuweisung erfolgt ähnlich wie bei einer Variablen, außer dass beim Array eine Werteliste in runden Klammern verwendet wird

```
ein_array=(Hund Katze Esel Maus)
```

Auslesen des gesamten Arrays

```
echo ${ein_array[*]}
```

Auslesen einzelner Werte (beginnend bei Null, hier wird also der sechste Wert ausgelesen)

```
echo ${ein_array[5]}
```

Anzahl der Elemente im Array

```
echo ${#ein_array[*]}
```

Länge eines bestimmten Elementes

```
echo ${#ein_array[3]}
```

In aktuellen bash-Versionen sind auch assoziative Arrays möglich (also die Zuordnung von Werten nicht nur zu Indizes sondern auch zu benannten Schlüsseln).

```
declare -A ARR
```

```
ARR[schluesselA]=wertA
```

```
ARR[schluesselB]=wertB
```

Die Ausgabe der Werte erfolgt genauso wie bei normalen Arrays `${ARR[*]}`, die Ausgabe der Schlüssel mit `${!ARR[*]}`

Mit Arrays lassen sich also auch Zeichenketten in einzelne Werte zerlegen, auf die im späteren Verlauf eines Skriptes zugegriffen werden kann. Soll jedoch nur einfach ein Element Zeichenkette „ausgeschnitten“ werden, um es sofort weiter zu bearbeiten, eignet sich auch das `set`-Kommando:

```
set var=`ls -l datei`
echo $3
```

Die Ausgabe des `ls`-Kommandos wird in einzelne Elemente zerlegt, sodass nun mithilfe der bekannten Positionsparameter `$1` bis `$9`, `$*` und `$#` auf diese Elemente (bzw. ihre Anzahl) zugegriffen werden kann.

3.3 Shell-Skripte

3.3.1 Variablen und Argumente

Die bisher gezeigten Kommandos sind lediglich die grundlegenden Funktionen einer Shell. Diese Funktionen lassen sich auch in kleinen Programmen (Shell-Skripten) zusammenfassen, in denen wie auch bei anderen Programmiersprachen mit Schleifen, Bedingungen oder Unterfunktionen gearbeitet werden kann. Diese Programme werden jedoch nicht kompiliert und in Maschinensprache ausgeführt sondern sequentiell interpretiert.

Das Shell-Skript wird von der startenden Shell interpretiert, sofern in der ersten Zeile nichts anderes definiert ist. Soll eine bestimmte Shell als Sub-Shell aufgerufen werden, so muss diese Shell einschließlich des vollständigen Pfades unmittelbar hinter der Zeichenkombination `#!` stehen (sog. She-Bang): `#!/bin/bash`

Die bereits zuvor genannten vordefinierten Umgebungsvariablen einer Shell können auch in Shell-Skripten verwendet werden. Darüber hinaus gibt es diverse weitere solcher Variablen (siehe Kommando `env` und entsprechende Literatur oder Hilfsfunktionen, z.B. `man bash`). Folgende Standardvariablen werden häufig für das Arbeiten mit Skripten benötigt:

- Name des Skriptes: `$0` (Hinweis: wird das Skript nicht als Subshell ausgeführt, also mit `source`, so gibt `$0` den Namen der ausführenden Shell aus)
- Prozessnummer des Skriptes: `$$`
- Exit-Status (s.o.): `$?`
- Prozessnummer des zuletzt gestarteten Hintergrundprozesses: `$!`
- Argumente aus der Kommandozeile: `$1` bis `$n` (bei mehrstelligem Wert von `n`, z.B. `${27}`)
- alle Argumente der Kommandozeile in einer Zeichenkette: `$*`
- alle Argumente der Kommandozeile als Array (einzelne Werte): `@`
- Anzahl der Argumente in der Kommandozeile: `$#`
- letztes Argument in der Kommandozeile: `$_`

Beispiel: Einlesen aller einzelnen Argumente in ein Array und Ausgabe des vorletzten Arguments (Hinweis: Argumente beginnen bei 1, Array-Elemente bei 0)

```
arr=($@)
echo "vorletztes Argument: " ${arr[$#-2]}
```

Mit dem Befehl `getopts` lassen sich Argumente auch in der von üblichen Systemkommandos bekannten Schreibweise von Optionen verwenden (z.B. `ls -la`), dazu später mehr.

Vorgabewerte von Variablen: Wenn ein Skript ein bestimmtes Argument erwartet, das aber beim Aufruf nicht gesetzt wurde, oder bestimmte Variablenwerte als Standard-Vorgaben verwendet werden sollen, die nicht unbedingt übergeben werden müssen, so besteht die Möglichkeit solche Vorgabewerte im Skript zu setzen. In diesem Beispiel wird beim Skript-Aufruf ein Argument (`$1`) erwartet; ist dieses gesetzt, so wird es mit dem `echo`-Befehl ausgegeben, ist es nicht gesetzt so wird stattdessen die zuvor definierte Variable `$wort` ausgegeben.

```
var=$1
wort="ABC"
echo ${var:-$wort}
```

Statt `$wort` kann auch entweder direkt eine Zeichenkette verwendet werden `${var:-"XyZ"}` oder eine Kommandosubstitution `${var:-`ls -l`}` . Wird statt des Minus-Zeichens ein Pluszeichen eingesetzt, so wird der Ausdruck dahinter immer dann verwendet, wenn „var“ nicht leer ist.

3.3.2 Bedingte Anweisung `if`

Die Grundstruktur der Verzweigungen ist der von anderen Programmiersprachen sehr ähnlich

```
if Bedingung
then
    Anweisungen
fi
```

Beispiel: Man möchte feststellen, ob die IP-Adresse 127.0.0.1 (sog. Localhost-Adresse) in der Datei /etc/hosts eingetragen ist. Bei exit-Status 0 (Erfolg bzw. „true“) des grep-Kommandos wird in den then-Bereich gesprungen, sonst wird die Ausführung hinter dem „fi“ fortgesetzt.

```
if grep "127.0.0.1" /etc/hosts &> /dev/null
then
    echo "Localhostadresse ist vorhanden"
    exit 0;    # Skript mit Exit-Code 0 (Erfolg) beenden
fi
echo "Localhostadresse nicht gefunden"
```

(Hinweis: Besteht die Bedingung aus einer Kommando-Pipe, so wird nur der Exit-Status des letzten Kommandos interpretiert; ist dies nicht erwünscht, so kann die bash-Variable PIPESTATUS ausgewertet werden.)

Man kann diese Kommandofolge auch direkt in der Kommandozeile eingeben. Nach Eingabe der if-Zeile öffnet sich ein weiterer Prompt, hinter dem die weiteren Anweisungen eingegeben werden. Die exit-Anweisung darf allerdings nicht verwendet werden, denn es werden in diesem Fall alle Anweisungen in der Shell direkt ausgeführt und nicht in einer Sub-Shell, weshalb das exit-Kommando das Terminalfenster schließen würde.

Alternativ können alle Anweisungen auch in einer einzigen Zeile geschrieben werden, dann allerdings ebenfalls ohne das „exit“ aber mit Semikolons zur Trennung der Befehle

```
if Bedingung ; then Anweisungen ; fi
```

Sollen nicht nur spezielle Anweisungen ausgeführt werden bei Exit-Status 0 (true), sondern auch bei einem Exit-Status größer Null (false), so kann ergänzend ein else-Bereich eingefügt werden:

```
if Bedingung ; then Anweisungen1 ; else Anweisungen2 ; fi
```

Wenn mehrere Bedingungen nacheinander getestet werden sollen, so ist `elif` geeignet.

```
if Kommando1
then
    Anweisungen, wenn Kommando 1 den Exit-Status 0 hat
elif Kommando2
then
    Anweisungen, wenn Kommando 2 den Exit-Status 0 hat
else
    Anweisungen, wenn Kommando 2 einen Exit-Status ungleich Null hat
fi
```

Die if-Verzweigung in shell-Skripten ist standardmäßig dafür vorgesehen, den Exit-Status von Kommandos zu prüfen, jedoch nicht den Wahrheitswert von Ausdrücken oder Vergleichen. Ist dies gewünscht, so muss dies mithilfe des Kommandos `test` erfolgen (es wird also der Exit-Status von `test` geprüft, wobei `test` z.B. Vergleiche durchführen kann). Im einfachsten Fall kann in der Kommandozeile z.B. aufgerufen werden:

```
test 4 -gt 5
echo $?
```

Der Exit-Status ist hierbei 1 (also „nicht erfolgreich“, denn 4 ist nicht größer als 5).

Entsprechend innerhalb eines Shell-Skriptes:

```
if test $varA -eq $var2
then ...
```

Statt das test-Kommando direkt aufzurufen, kann auch die alternative Schreibweise mit eckigen Klammern verwendet werden (wichtig: Leerzeichen vor und hinter den Klammern einfügen):

```
if [ $var1 -eq $var2 ]
then ...
```

Vergleichsoperatoren für ganze Zahlen sind: `-eq` (gleich), `-ne` (ungleich), `-lt` (kleiner als), `-gt` (größer als), `-le` (kleiner oder gleich), `-ge` (größer oder gleich).

In der bash können auch die aus anderen Programmiersprachen bekannten Operatoren (`==`, `!=`, `<`, `>`, `<=`, `>=`) innerhalb von doppelten runden Klammern verwendet werden:

```
if (( $var1 == $var2 ))
```



```
oder let
if let "$var1 <= $var2"
```

Vergleichsoperatoren für Zeichenketten sind:

```
[ "$var1 == $var2" ]      (gleiche Zeichenkette)
[ "$var1 != $var2" ]      (ungleiche Zeichenkette)
[ -z "$var1" ]            („true“, wenn var1 leer ist)
[ -n "$var1" ]            („true“, wenn var1 nicht leer ist)
```

Ein wichtiges Anwendungsgebiet von `if` und `test` innerhalb von Shell-Skripten ist die Abfrage eines Datei-Status. In obigem Beispiel wurde die Datei `/etc/hosts` durchsucht, wobei angenommen wurde, dass diese Datei auch existiert. Existiert sie jedoch nicht, so wäre der Exit-Status 2 und die Anweisungen innerhalb des `then`-Bereiches würden nicht ausgeführt werden. Es wäre in der Ausgabe des Skriptes nun aber nicht zu unterscheiden, ob die eigentliche Abfrage negativ ausgefallen ist oder die Datei überhaupt nicht existiert.

Für den `test`-Befehl gibt es für diese Aufgabe diverse Operatoren für Dateieigenschaften:

Prüfen des Datei-Typs, z.B. prüfen ob `DAT` existiert und eine reguläre Datei ist: `if [ -f DAT ]`

Weitere Datei-Typen: `-d` (Verzeichnis), `-l` (symbolischer Link), `-p` (Named Pipe), `-b` (Block-Device, Gerätedatei)

Das Prüfen der Zugriffsrechte ist ebenfalls wichtig, denn auch hierbei können Fehler auftreten, wenn die Datei gar nicht les- oder schreibbar ist: `-r` die Datei existiert und ist lesbar, `-w` (schreibbar), `-x` (ausführbar), `-o` die Datei existiert und der ausführende User des Skripts ist auch Owner der Datei

Weitere Operatoren: `-e` (die Datei existiert), `-s` (Datei existiert und ist nicht leer), `Dat1 -nt Dat2` bzw. `-ot` (`Dat1` ist neuer (newer time) bzw. älter (older time) als `Dat2`)

Wie in anderen Programmiersprachen sind auch logische Operatoren vorhanden:

```
if [ -f $datei ] && [ -r $datei ]
```

Die Bedingung ist wahr, wenn „datei“ eine existierende, reguläre Datei ist UND außerdem lesbar für den ausführenden Benutzer des Skriptes (entsprechend ODER: `||` sowie Negation `!`)

Insbesondere im Zusammenhang mit Kommando-Ausführungen sind logische Operatoren interessant. Ein Befehl kann bspw. nur dann ausgeführt werden, wenn ein vorhergehender Befehl erfolgreich war (`&&`) bzw. fehlgeschlagen ist (`||`).

```
[ -e datei ] || vi datei
```

Hierbei wird zunächst überprüft, ob „datei“ existiert und nur dann mit `vi` geöffnet, wenn dies nicht der Fall ist (bzw. mit `&&` nur dann geöffnet, wenn die Datei bereits existiert).

3.3.3 case-Anweisung

Wenn mehrere `if-elif`-Anweisungen verwendet werden müssen, so bietet sich alternativ oft der Einsatz von `case` an. Mit `case` wird jedoch nicht der Rückgabewert eines Kommando überprüft, sondern es werden Vergleiche mit Mustern aus einer Liste durchgeführt.

```
case "$var1" in
  muster1)  Kommandos ;;
  muster2)  Kommandos ;;
```

...

```
esac
```

Entspricht „var1“ einem der Muster, so werden die dahinter folgenden Kommandos ausgeführt und anschließend das Skript hinter `esac` fortgesetzt. Der muster-Vergleich wird in der angegebenen Reihenfolge abgearbeitet; sollte kein Muster zutreffen, wird das Skript ebenfalls hinter `esac` fortgesetzt. Die Muster können auch mit logischen Operatoren verknüpft sein: `muster1|muster2|muster3) Kommando ;;`

Möchte man nach dem erfolgreichen Vergleich mit einem der Muster auch die weiteren Muster noch testen (also nicht aus der `case`-Anweisung herauspringen), muss am Ende der Kommandos `;; &` verwendet werden (nur `bash`).

3.3.4 Schleifen

Wie in anderen Programmiersprachen kann in der Shell mit Schleifen gearbeitet werden. Hier soll nur kurz auf die `while`-Schleife eingegangen werden:

```
while [ Bedingung ] ; do Kommando1 ; Kommando2 ; ... ; done
```

Beispiel:

```
j=20
while [ $j -gt 0 ]
do
  echo $j
  sleep 2      # Ausführung des Skripts für 2 Sekunden anhalten
  ((j-=2))    # nur bash; sonst j=`expr $j - 2`
done
```

Beispiel für die Verarbeitung von Kommandozeilenargumenten:

```
[ $# -lt 1 ] && exit 1      # Abbruch mit exit-Code 1 wenn kein Argument vorhanden ist
ARR=($@)
j=0
while [ $j -lt $# ]
do
  echo "Übergabewert Nr. $j: ${ARR[$j]}"
  ((j++))
done
```

Beispiel mit Kommandoausführung

```
datei=(`ls`)      # ls-Kommando in Array einlesen
anzahl=${#datei[*]}
j=0
while [ $j -lt $anzahl ]
do
  echo ${datei[$j]}
  ((j++))
done
```

Die Schleifenausführung kann mit der Anweisung `break` abgebrochen werden (handelt es sich um verschachtelte Schleifen, so wird nur die innerste Schleife verlassen). Soll nur der aktuelle Schleifendurchlauf beendet werden, aber die Schleife mit dem nächsten Durchlauf fortgesetzt werden, so kann die Anweisung `continue` verwendet werden. In folgendem Beispiel wird eine Zahl solange um 1 heruntergezählt, bis sie durch 5 teilbar ist, danach wird die Zahl gleich um 5 reduziert:

```
i=53
while [ $i -gt 0 ]
do
  if [ $((i % 5)) == 0 ]
  then
    echo „$i ist teilbar durch 5“
    ((i-=5))
    continue
  fi
  echo $i
  ((i--))
done
```

Mit `while true` wird das Skript in einer Endlosschleife ausgeführt; es kann in diesem Fall entweder in der Kommandozeile mit Strg-C beendet werden oder es wird eine `break` oder `exit`-Anweisung innerhalb der Schleife verwendet.

3.3.5 Ein- und Ausgabe

In vielen Fällen wird vor der Ausführung eines Skriptes eine Benutzereingabe erwartet. Die bisher verwendeten Kommandozeilenargumente eignen sich oftmals nicht gut dafür, da ein Benutzer wissen muss, welche Parameter in welcher Reihenfolge eingegeben werden müssen. Daher ist oft eine interaktive Abfrage der benötigten Parameter sinnvoll.

Hierfür eignet sich der `read`-Befehl: Gibt man in der Kommandozeile `read var` ein, so wird eine interaktive Eingabe erwartet. Tippt man nun z.B. ABC ein, so findet sich diese Eingabe in der Variablen `var`, wie das Kommando `echo $var` zeigt. Dies funktioniert auch mit mehr als nur einer Variablen: `read varA varB`.

Beim Arbeiten mit Dateien ermöglicht `read` das zeilenweise einlesen einer Textdatei

```
while read zeile
do
    echo $zeile
done < textdatei
```

Sinnvoll ist in diesem Zusammenhang auch das zeilenweise Einlesen einer Textdatei in ein Array. Hierzu muss obiges Beispiel etwas verändert werden:

```
i=0
while read z[$i]
do
    ((i++))
done < textdatei
echo ${z[3]}
```

Beispiel für die interaktive Eingabe (zwei Zahlen in ein Array eingeben und multiplizieren)

```
i=0
zahl=()
while [ $i -lt 2 ]
do
    echo -n "Bitte eine Zahl eingeben: "
    read zahl[$i]
    ((i++))
done
echo ${zahl[0]} \* ${zahl[1]} "=" $((zahl[0] * zahl[1]))
```

Auf ähnliche Weise lassen sich auch assoziative Arrays füllen:

```
declare -A NAME_ALTER
while [ -z "$name" ]
do
    echo -n "Bitte Namen eingeben (beenden: leere Eingabe): "
    read name
    if [ -z "$name" ]
    then
        break
    fi

    echo -n "Bitte Alter eingeben: "
    read alter
    if [ -z "$alter" ]
    then
        echo "kein Alter angegeben, bitte nochmals Name und Alter eingeben"
        continue
    fi
    NAME_ALTER[$name]=$alter
    name=""
done
```

Für `read` gibt es einige nützliche Optionen:

- n nach der Eingabe von n Zeichen wird der Eingabewert automatisch eingelesen (also ohne Enter)
- t wenn keine Eingabe erfolgt, wird das Skript nach t Sekunden fortgesetzt
- s die Eingabe erfolgt „unsichtbar“ (z.B. sinnvoll für Passwörter)

für Ein-/Ausgabe: `declare -u VARI` (VARI wird immer „upper-case“ verwendet, z.B. wenn man nicht unterscheiden möchte ob ein Benutzer nun yes, Yes oder YES bei einer Abfrage eingetippt hat. Umgekehrt „lower-case `declare -l`“)

Argumente als Optionen verwenden: `getopts Optionen Variable`

Hierbei ist „Optionen“ die Liste der möglichen Optionen (hier -a oder -b oder -x) und „Variable“ wird der Optionswert übergeben, der dann schrittweise in der Schleife ausgelesen wird. Ein Doppelpunkt hinter einem der Optionsbuchstaben (hier -x) bedeutet, dass bei Verwendung dieser Option hinter dieser Option ein Argument erwartet wird, welches in der Shell-Variablen `OPTARG` abgelegt wird.

```
while getopts abx: var
do
case $var in
  a) echo "Option a";;
  b) echo "Option b";;
  x) echo "Option x : $OPTARG";;
esac
done
```

4 Systemstart

4.1 Initialisierung der Hardware

4.1.1 BIOS

Jede Hauptplatine (Main-/Mother-Board) verfügt über einen nicht-flüchtigen Speicher (EEPROM – Electrical Erasable Programmable Read-Only Memory oder moderne Flash-Speicher), in dem das sog. BIOS (Basic Input Output System) gespeichert ist. Dieses Programm, das in der Maschinensprache des Prozessors erstellt ist, enthält grundlegende Funktionen, um einen Rechner und sein Betriebssystem zu starten. Hierzu wird zunächst in einem sog. Power-On-Self-Test (POST) die Funktionsfähigkeit unabdingbarer Hardware oder Ein-/Ausgabegeräte geprüft, wie z.B. die CPU, die Tastatur, der Bildschirm oder der Hauptspeicher (RAM). Es werden dann die Bussysteme (PCI, PCIe, IDE, SCSI, SATA, USB, IEEE 1394 usw.) auf angeschlossene Hardware überprüft und den Geräten (i.d.R. Controller-Karten) eindeutige Interrupt-Nummern und Adressen im Ein-/Ausgaberegister zugeordnet, sofern es sich um Plug-and-Play-fähige Geräte handelt.

Da ein BIOS-Programm i.d.R. nur mit der auf dem Mainboard vorhandenen oder mit Standard-Hardware umgehen kann, nicht jedoch mit jeder beliebigen Hardware, wird am Ende des POST das Vorhandensein von BIOS-Erweiterungen geprüft, die auf den entsprechenden Controller-Karten gespeichert sind; z.B. werden die speziellen Funktionen eines RAID-Controllers über ein eigenes BIOS auf der Controller-Karte gesteuert. Die Konfigurationseinstellungen sowohl des Standard-BIOS als auch der BIOS-Erweiterungen sind vor Beginn des Ladens des Betriebssystems mittels spezieller Tastenkombinationen zugänglich.

Die letzte Aktion des BIOS ist die Prüfung des Systems auf prinzipiell boot-fähige Hardware, i.d.R. Speichermedien wie Festplatten, optische Laufwerke oder USB-Speicher. Diese Speichermedien sind dann tatsächlich boot-fähig, wenn im ersten Speichersektor (auf einer Festplatte die ersten 512 Byte) ein sog. Boot-Loader zu finden ist. Sind mehrere boot-fähige Speichermedien vorhanden, so muss in der BIOS-Konfiguration eine Startreihenfolge angegeben sein. Das Programm des Bootloaders wird geladen und ausgeführt; ab diesem Zeitpunkt übernimmt der Boot-Loader alle weiteren Aufgaben. Falls mehrere Betriebssysteme vorhanden sein sollten, so wird zunächst ein Boot-Manager gestartet, über den die Auswahl eines der Betriebssysteme erfolgt, oder es wird sofort das (einzig vorhandene) Betriebssystem gestartet.

Bei älteren Betriebssystemen wie z.B. DOS übernimmt das BIOS auch im laufenden Betrieb die Funktionen zum Zugriff auf die Hardware (z.B. Ansteuerung der Tastatur), moderne Betriebssysteme wie Windows oder Linux verwenden ihre eigenen Hardware-Treiber und benötigen das BIOS nach dem Rechnerstart nicht mehr.

Da das bereits Anfang der 1980er-Jahre eingeführte BIOS trotz einiger Erweiterungen nicht mehr den Anforderungen moderner Hardware entsprach, wurde das Unified Extensible Firmware Interface (UEFI) entwickelt.

4.1.2 UEFI

Ab 1998 wurde von Intel der BIOS-Nachfolger EFI entwickelt, später unter Beteiligung weiterer Firmen als UEFI weiterentwickelt; heute werden beide Begriffe meist synonym verwendet.

Die wesentlichen Vorteile gegenüber dem BIOS sind die Unterstützung von Festplatten größer als 2 TB, die Kompatibilität zur Partitionierung mit der GPT (siehe Kap. 4.1.4) und die parallele Installation mehrerer Betriebssysteme. Apple nutzt UEFI bereits seit längerem, auf PCs verbreitete sich UEFI spätestens mit der Einführung von Windows 8. Für Linux spielte die Einführung von UEFI keine große Rolle, da die nennenswerten Vorteile auch mit dem BIOS und dem Boot-Loader GRUB (siehe Kap. 4.1.5) schon länger verfügbar waren. Im Gegenteil: das sog. UEFI-SecureBoot verursacht bei einigen (meist älteren) Linux-Distributionen Probleme; es soll mittels eines auf dem Mainboard hinterlegten Schlüssels das Einschleusen von Boot-Viren erschweren, allerdings gibt es auf den meisten Boards nur den Schlüssel von Microsoft. Sollte eine Linux-Installation daher nicht möglich sein, so muss SecureBoot deaktiviert werden.

4.1.3 MBR

Der Master Boot Record MBR befindet sich im ersten Sektor eines boot-fähigen Speichermediums (z.B. Festplatte). Die Standardgröße eines Sektors ist i.d.R. 512 Byte, weshalb der MBR keinen besonders umfangreichen Programmcode enthalten kann. Der wesentliche Inhalt des MBR besteht aus dem Boot-Loader (Master Boot Code) und der Partitionstabelle des Speichermediums. Mittels der Partitionstabelle kann der Boot-Loader auf den

tatsächlichen Speicherort des Betriebssystem-Kerns zugreifen und ihn starten.

Der Programmcode des Boot-Loaders umfasst die ersten 440 Byte des MBR, dann folgt eine 4 Byte große Datenträgersignatur, zwei Null-Bytes und die 64 Byte große Partitionstabelle; den Abschluss bilden die zwei Bytewerte 55AA (hex, binär: 01010101 10101010), auch als Bootsektor-Signatur oder Magic Number bezeichnet, die dem BIOS einen gültigen MBR anzeigen; sind diese Bytes nicht vorhanden, so wird der Boot-Prozess abgebrochen, um nicht Zufallsdaten eines gar nicht boot-fähigen Mediums in den RAM zu laden.

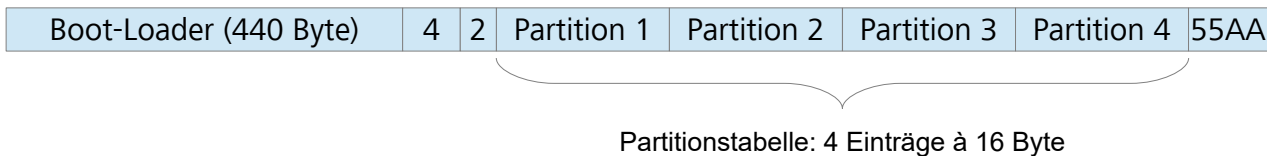


Abbildung 1: MBR Partitionierung

Die Standard-Partitionstabelle umfasst 4 Einträge zu je 16 Byte. Damit ist die Definition von vier sog. „primären Partitionen“ möglich. Eine dieser primären Partitionen kann als „erweiterte Partition“ verwendet werden.

Eintrag (16 Byte) für eine Partition in der Partitionstabelle im MBR (grundsätzlicher Aufbau):

Byteposition (hex)	Größe (Byte)	Inhalt
0x00	1	Wert 80 (hex): bootfähige Partition, Wert 00 : nicht bootfähig
0x01	3	CHS (Cylinder, Head, Sector)-Eintrag des ersten Sektors
0x04	1	Partitionstyp
0x05	3	CHS-Eintrag des letzten Sektors
0x08	4	Startsektor
0x0C	4	Anzahl der Sektoren

In der Partitionstabelle (ab Byteposition 446) des MBR sind max. vier solcher 16 Byte großen sog. „**primären**“ Partitionseinträge möglich; sollen mehr als vier Partitionen verwendet werden, so kann eine (einzige) der Partitionen als Typ „**erweitert**“ angelegt werden. Der Partitionstyp ist diesem Fall 0x05. Weitere Partitionstypen sind z.B. FAT32 (0x0B), Linux (0x83), NTFS (0x07) usw.

Der Startsektor-Eintrag einer erweiterten Partition verweist auf die Position der ersten erweiterten Partitionstabelle, die sich im ersten Sektor dieser Partition befindet. In dieser erweiterten Partitionstabelle gibt es nun zwei 16-Byte-Einträge: der erste definiert die sog. „**logische**“ Partition selbst, der zweite (Typ 0x05) verweist ggf. auf die Position der nächsten erweiterten Partitionstabelle. Durch diese Verkettung von erweiterten Partitionstabellen können prinzipiell beliebig viele logische Partitionen angelegt werden.

Die CHS-Werte (Cylinder, Head, Sector) beziehen sich auf die sehr einfache Geometrie der ersten Festplatten aus den 1960er Jahren, die aus mehreren Magnetplatten (ansprechbar über Schreib-/Lese-Köpfe) und den Spuren einer Platte bestanden (jeweils übereinanderliegende Spuren werden zusammengefasst als Zylinder bezeichnet), die in eine (feste Anzahl) von Sektoren unterteilt waren. Diese triviale Geometrie existiert heute jedoch nicht mehr; stattdessen werden die CHS-Werte mithilfe der Festplatten-Controller gebildet und an das Betriebssystem übermittelt. Der Wertebereich der CHS-Adressierung reicht aber trotz verschiedener Konvertierungsverfahren nicht mehr für moderne Festplatten aus und hat für SSDs ohnehin keine technische Bedeutung mehr. Zwar werden aus Kompatibilitätsgründen auch heute noch CHS-Werte von den Festplattencontrollern (auch SSDs) an das Betriebssystem geliefert, aber im MBR werden diese Werte heute nicht mehr verwendet. Ebenfalls aus Kompatibilitätsgründen wird weiterhin mit Sektorgrößen von 512 Byte gerechnet, obwohl die meisten Festplatten eine physikalische Sektorgröße von 4 kB haben.

Heute wird anstelle der CHS-Werte das LBA-Verfahren (Logical Block Address) verwendet, mit dem die vorhandenen Blöcke einfach (mit Null beginnend) „durchgezählt“ werden. Aus den 4-Byte-Werten (LBA) „Startsektor“ und „Anzahl der Sektoren“ werden die jeweiligen Positionen der Partitionen berechnet. Die tatsächliche physikalische Positionierung des Schreib-/Lese-Kopfes wird über interne Mapping-Tabellen vom Festplatten-Controller durchgeführt.

Die Art der Partitionstabelle und die Tabelle selbst können unter Linux mit dem Kommando (nur als Administrator) `parted -l` ausgelesen werden. Das Label „msdos“ bezeichnet die MBR-Partitionierung, „gpt“ die GUID Partiton Table (s.u.)

4.1.4 GUID Partition Table

Der heute oft verwendete Nachfolger des MBR ist die Globally Unique Identifier Partition Table (GUID-Partitionstabelle oder GPT), der einige Beschränkungen des MBR-Konzepts aufhebt.

Ein GUID ist eine 16-Byte-Zahl (128 Bit), die auf verschiedene Weise erzeugt werden kann, z.B. als Pseudo-Zufallszahl oder basierend auf der MAC-Adresse und einem Zeitstempel oder als Hashwert (MD5, SHA-1) von gegebenen Zeichenketten. Das Format dieser Zahl ist jedoch immer gleich: hexadezimale Zahlen in der Schreibweise 8-4-4-4-12 (je zwei hexadezimale Zahlen repräsentieren ein Byte), z.B. a0ff09de-b109-0c66-5daa-98ee2968ba0f. Damit kann also ein eindeutiger Bezeichner z.B. für eine Festplatte erzeugt werden, sodass Zugriffe auf falsche Datenträger aufgrund zufällig gleicher Bezeichner praktisch ausgeschlossen sind oder sich z.B. nach dem Einbau einer zusätzlichen Festplatte die Nummerierung von Geräte-Dateien verschoben hat und damit der Zugriff nicht mehr möglich ist.

Unter Linux wird die (dort nicht GUID sondern Universally UID (UUID) genannte) Kennung beim Formatieren eines Datenträgers automatisch erzeugt. Der Befehl `/sbin/blkid` gibt die vorhandenen Kennungen aus, alternativ kann man sich den Inhalt des Verzeichnisses `/dev/disk/by-uuid` ansehen: `ls -l /dev/disk/by-uuid`

Statt nur eines einzigen Sektors beim MBR, umfasst die GPT (standardmäßig) die ersten 34 Sektoren eines Datenträgers und (als Backup-Kopie) zusätzlich die letzten 34 Sektoren; dieser Bereich kann bei Bedarf auch vergrößert werden. Der erste Sektor (512 Byte) ist als Schutz-MBR aufgebaut, damit ältere Hardware (z.B. ein BIOS oder Partitionierungs-Software), die die GPT nicht kennt, die GPT-Sektorstruktur nicht versehentlich löscht oder beschädigt. Dieser Schutz-MBR enthält eine übliche Partitionstabelle sowie eine Kennung, die den Datenträger als vollständig belegt bezeichnet. Im zweiten Sektor befindet sich der primäre Header der GPT, der u.a. die Anzahl (prinzipiell beliebig viele) und die Größe (standardmäßig 128 Byte) der einzelnen Einträge in der Partitionstabelle enthält, eine CRC32-Prüfsumme für den Header selbst sowie die folgende Partitionstabelle, die Position des sekundären Backup-Headers (Standard: letzter Sektor des Datenträgers), die Nummer des ersten für eine Partition nutzbaren LBA (Standard 34) und die Datenträger-GUID/UUID (diese UUID wird bereits beim Initialisieren des GPT-Headers gebildet (parted mklabel)).

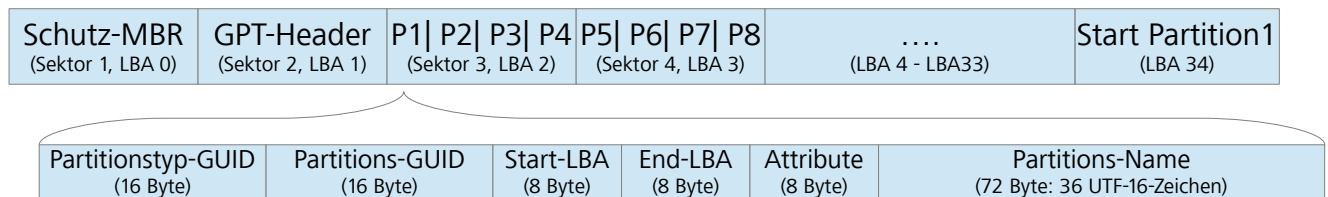


Abbildung 2: GPT Partitionierung

Die einzelnen Partitionseinträge (hier als P1, P2 usw. bezeichnet) umfassen je 128 Byte und haben folgenden Aufbau: standardisierte GUID für den Partitionstyp (z.B. für FAT32, Linux-Native, NTFS, Mac OS X usw.), GUID/UUID für die Partition, Start- und End-Position (LBA) der Partition, Attribute (z.B. Systempartition, BIOS-bootfähig, Read-Only). Diese zweite UUID wird beim Anlegen der Partition gebildet (parted mkpart). Die o.g. dritte UUID wird beim Formatieren der Partition gebildet (mkfs) und (abhängig vom Typ des Filesystems) in der jeweiligen Headerbereich des Filesystems geschrieben.

4.1.5 Boot-Loader GRUB

Für die beiden Startsysteme BIOS und EFI gibt es zwei verschiedene Varianten des Grant Unified Bootloader GRUB, die beim Installieren eines Linux-Systems automatisch ausgewählt werden. GRUB ist nicht nur in der Lage ein Linux-System zu starten sondern kann auch verschiedene Betriebssysteme starten, die parallel auf einem Rechner installiert sind (Hinweis: alle Informationen in diesem Kapitel beziehen sich auf die aktuelle Version GRUB2). <http://www.gnu.org/software/grub/manual/grub.pdf>

Bei EFI-Systemen mit GPT-Partitionierung kann die Auswahl der Betriebssysteme über ein von EFI selbst bereitgestelltes Menü erfolgen (F12), da für jedes Betriebssystem der eigene Bootloader innerhalb einer eigenen, speziell dafür vorgesehenen Partition abgelegt wird. Diese Partition muss zwingend als FAT32 oder VFAT formatiert sein; als Größe sind wenige 100MB ausreichend. Unter Linux wird diese Partition später im Verzeichnis `/boot/efi` eingehängt. Jedes auf dem Rechner installierte Linux-System wird also mit seinem eigenen GRUB-Boot-Loader (dem core-Image) gestartet, ein Windows-System direkt mit seinem eigenen „bootmgr“.

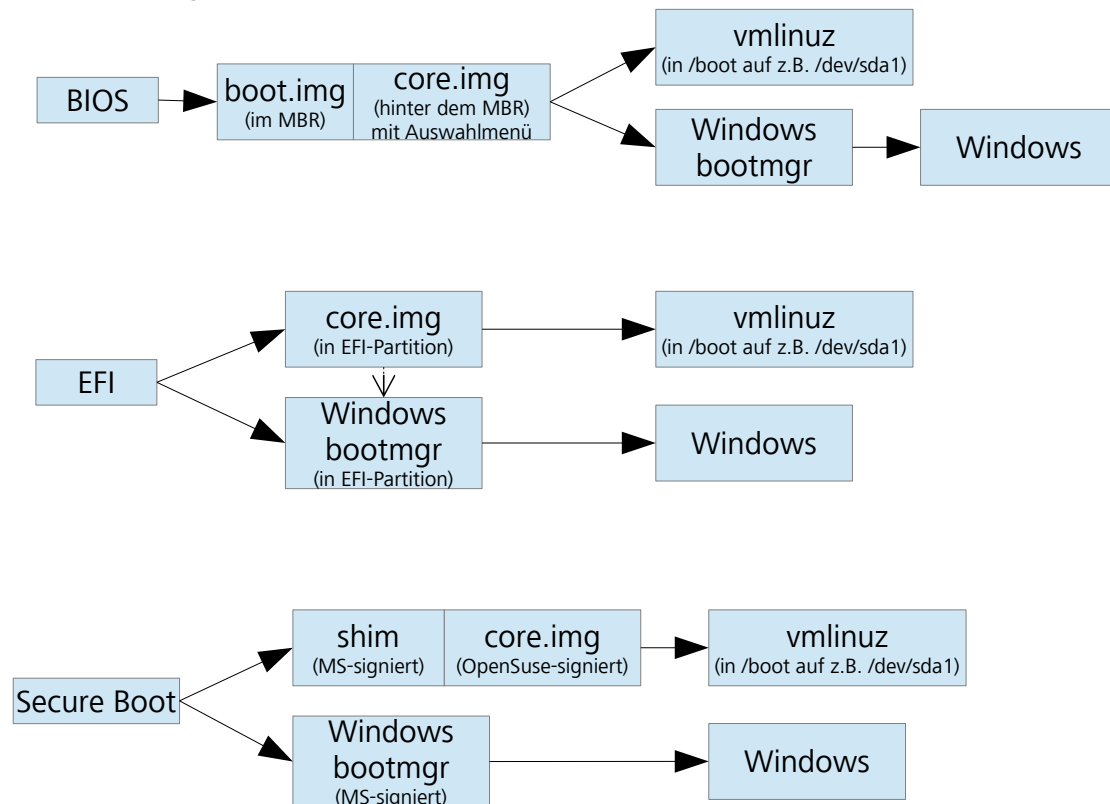
Bei BIOS-basierten Systemen oder bei einer klassischen MBR-Partitionierung übernimmt GRUB zusätzlich die

Aufgabe, ein Auswahl-Menü für die verschiedenen Betriebssysteme anzubieten (funktioniert auch mit EFI). GRUB muss in diesem Fall außerdem in der Lage sein, den Speicherort des jeweiligen Bootloaders zu finden sowie den Boot-Code zu lesen und zu laden. Im MBR (bzw. im Schutz-MBR bei GPT) wird ein Boot-Image (boot.img, max. 446 Byte) gespeichert, dessen einzige Aufgabe es ist, das sog. Core-Image (core.img) zu finden und zu laden. Bei MBR-partitionierten Datenträgern befindet sich das Core-Image unmittelbar hinter dem MBR (ab dem zweiten Sektor) in einem üblicherweise frei gelassenen Bereich zwischen MBR und erster Partition (ca. 32kB), bei EFI und GPT wird die o.g. EFI-Partition dafür reserviert, bei der heute eher unüblichen Kombination GPT und BIOS muss eine eigene BIOS-GRUB-Partition angelegt werden. Das Core-Image wird beim Installieren von GRUB dynamisch erzeugt und enthält alle grundlegenden Funktionen bzw. Module, um auf die Dateisystemstruktur des eigentlichen Boot-Verzeichnisses zuzugreifen, dort die GRUB-Konfigurationsdatei (grub.cfg) zu lesen und letztlich den Betriebssystem-Kernel starten zu können. Dieses Boot-Verzeichnis (/boot) kann entweder auf einer eigenen Partition liegen oder auf der Systempartition im Root-Verzeichnis.

In diesem Normalfall des Betriebssystemstarts von einer lokalen Festplatte besteht der erste Sektor des Core-Image aus dem „diskboot.img“; es besteht jedoch auch die Möglichkeit, GRUB über das Netzwerk mittels PXE (Preboot eXecution Environment) zu verwenden, dann besteht der erste Sektor aus dem pxeboot.img).

Ist das UEFI Secure Boot vorhanden und aktiv, so werden von EFI nur solche Bootloader gestartet, die mit einem auf dem Mainboard fest gespeicherten kryptografischen Schlüssel signiert sind. Damit soll verhindert werden, dass Schadsoftware bereits im Boot-Vorgang eingeschleust wird und so von Viren-Scannern, die ja erst mit dem eigentlichen Betriebssystemstart aktiviert werden können, nicht mehr erkannt werden kann. Auf praktisch allen PC-Mainboards ist jedoch nur das Zertifikat von Microsoft gespeichert, sodass nur von Microsoft signierte Bootloader gestartet werden können. Damit jedoch auch andere Betriebssysteme mit Secure Boot gestartet werden können, bietet Microsoft den Linux-Distributoren einen Signier-Dienst an. I.d.R. wird aber nicht der gesamte Bootloader mit dem Microsoft-Schlüssel signiert sondern nur ein kleines Programm namens „shim“, das lediglich die Zertifikatskette sicherstellen soll. EFI startet in diesem Fall erst das korrekt signierte shim und dieses startet dann das mit einem distributions-eigenen Schlüssel signierte GRUB. Probleme kann es hierbei jedoch mit nicht-signierten Kernel-Modulen (z.B. proprietäre, vorkompilierte Grafik-Module) geben; in diesen Fällen hilft nur das Deaktivieren von Secure Boot. Selbstgebaute Kernel sind von dem „Secure Boot-Problem“ ebenfalls betroffen: es gibt jedoch die Möglichkeit, mittel eines Machine Owner Key (MOK) auch selbst signierte Kernel mit shim zu starten.

<https://doc.opensuse.org/documentation/leap/reference/html/book.opensuse.reference>



4.2 Kernel

Nachdem mittels des Boot-Loaders die grundlegenden Module aktiviert wurden, kann der eigentliche Betriebssystem-Kern (Kernel) gelesen und gestartet werden. Der kompilierte Kernel liegt im Verzeichnis `/boot` unter dem Namen „`vmlinuz`“ (das „z“ steht für die gzip-Komprimierung), der einen Link auf den aktuellen Kernel darstellt; dieser hat einen Namen wie z.B. `vmlinux-4.4.27-default.gz`. Der von den Distributoren vorkompilierte Kernel enthält nur allgemeine, wichtige Module für den Systemstart, wodurch sein Umfang klein bleibt (nur wenige Megabyte).

Andere Module, wie z.B. Treiber für die spezielle Hardware des Rechners, werden erst mit dem Systemstart ergänzend geladen. Dadurch kann die „unglückliche Situation“ entstehen, dass der Standard-Kernel auf der vorliegenden Hardware überhaupt nicht starten kann, weil elementare Module fehlen. Dies kann bspw. der Fall sein, wenn eine spezielle Festplatten-Hardware oder nicht-Standard-Dateisysteme für die Systempartition verwendet werden oder wenn die Systempartition über einen Netzwerkspeicher z.B. per iSCSI erreichbar ist oder auf einem RAID-System installiert worden ist. Daher wird während der Installation des Systems zusätzlich zum gegebenen Kernel eine sog. Initiale RAM-Disk (`initramfs`) angelegt. Es handelt sich hierbei um eine `cpio`-Archiv-Datei (`initrd`), die vom Bootloader zusätzlich zum Kernel geladen wird. Diese `initrd`-Datei wird in eine RAM-Disk entpackt und dient dem Kernel zunächst als Root-Filesystem mit elementaren Grundfunktionen des Betriebssystems und den für die Hardware des Rechners wichtigen Modulen.

Dies bedeutet grundsätzlich folgendes: Eine `initrd`-Datei ist nicht zwingend erforderlich, z.B. wenn man selbst einen auf die eigene Rechner-Hardware angepassten Kernel „gebaut“ hat, der bereits alle nötigen Module und Funktionen enthält. Die `initrd`-Datei darf ausschließlich die zum jeweiligen Kernel passenden Module enthalten. Und wenn die für den Boot-Prozess relevante Hardware oder der Kernel nachträglich geändert wird, muss eine neue `initrd`-Datei erstellt werden.

Der Kernel startet also zunächst mit dem root-Filesystem der initialen RAM-Disk, lädt mittels des `init`-Prozesses die dort enthaltenen Module, sucht mithilfe jedes Moduls die dazu passende angeschlossene Hardware, erzeugt die für Block-Devices (z.B. Festplatte) erforderlichen Device-Files im Verzeichnis `/dev` und kann dann die bei der Installation angegebene Systempartition als Root-Verzeichnis einhängen (die nun allerdings in der RAM-Disk existierenden Device-Files müssen im weiteren Start-Prozess noch in das tatsächliche Filesystem verschoben werden). Nun kann mit dem realen Root-Verzeichnis auf der Festplatte weitergearbeitet werden.

Dieser prinzipielle Startprozess erlaubt es, auch Netzwerk-Dateisysteme wie iSCSI oder NFS als Root-Verzeichnis zu nutzen; hierzu müssen lediglich entsprechende Netzwerk-Module gestartet werden und die Informationen zu den Netzwerk-Geräten (Adressen, Authentifizierung) vorhanden sein.

Nach dem Laden des Kernels werden zwei Prozesse gestartet: `systemd` (Process ID PID 1) und `kthreadd` (PID 2). Grundsätzlich gibt es zwei Bereiche für die Prozess-Verarbeitung: User-Space und Kernel-Space.

Programme im **Kernel-Space** führen die Basisfunktionen des Betriebssystems aus (Schnittstelle zur Hardware, Speicher- und Prozessverwaltung, Lastverteilung) und haben den vollen Zugriff auf alle Ressourcen bzw. Hardware (Speicher, CPU usw.) des Systems; dadurch können sie die CPU jederzeit nutzen (hohe Priorität) oder alle Speicherbereiche lesen und schreiben. Für ein stabiles System müssen alle Prozesse im Kernel-Space daher „sauber arbeiten“, um nicht andere Prozesse zu stören oder durch Fehlverhalten sogar das Betriebssystem „abzuschießen“. Der im Kernel liegende Code sollte daher immer gut getestet sein. Der Linux-Kernel ist ein eher kleiner, monolithischer Betriebssystemkern, der alle weiteren benötigten Funktionen (wie z.B. Treiber für Hardware) über Module flexibel zusätzlich laden kann. Welche Module und Funktionen im eigentlichen Kernel direkt enthalten sind und somit beim Systemstart sofort zur Verfügung stehen, muss beim Erstellen/Kompilieren des Kernels festgelegt werden.

Im **User-Space** werden alle anderen Programme in einer Art „Sandkasten“ ausgeführt. Diese Programme haben einen beschränkten Zugriff auf den Speicher (sie dürfen z.B. niemals in den Speicherbereich des Kernel schreiben). Diese Programme werden alle vom `systemd` gestartet (s.u.).

Mit dem Start des Kernels stehen alle wichtigen, grundlegenden Funktionen zur Verfügung. Oftmals ist es jedoch notwendig, dass bestimmte Funktionen mehrfach, parallel ausgeführt werden, z.B. sind bei Mehrkernprozessoren mehrere Instanzen des „`kworker`“ nötig, der für das Interrupt-Handling der Prozessorkerne zuständig ist. Die „Oberaufsicht“ über diese Funktionen führt der Kernel Thread Daemon (`kthreadd`), der solche Funktionen als Teilprozesse des Kernels (Kernel-Threads) startet.

4.3 Unix systemd

Bevor ein Betriebssystem genutzt werden kann, müssen die notwendigen Prozesse gestartet werden. Um den geordneten Startvorgang und die spätere Kontrolle dieser Prozesse kümmert sich der sog. „systemd“ (der System-Daemon). Der systemd ist Nachfolger des System-V-Init-Systems (V als römische Fünf). Dieser ältere Dienst startete das Betriebssystem mit den erforderlichen Prozessen sequentiell in einer starren Reihenfolge, die manuell festgelegt werden musste. Alle zum Start eines Dienstes nötigen Parameter wurden in ausführbaren Shell-Skripten eingetragen. Beispielsweise kann so definiert werden, wo der Apache-Webserver seine zum Start erforderlichen Konfigurationsdateien findet und dass er erst gestartet wird, nachdem die Netzwerkfunktionalität hergestellt und der MySQL-Datenbank-Server aktiv ist. Der Nachteil einer solchen Vorgehensweise ist, dass nicht nur diejenigen Prozesse auf den Start des Netzwerks warten müssen, die darauf zwingend angewiesen sind, sondern auch alle anderen Dienste, die in der Reihenfolge nach dem Netzwerkstart einsortiert sind. Man könnte zwar die Reihenfolge umdrehen, aber dann müsste ohne Grund der Netzwerkstart warten bis diese Dienste aktiviert sind. Da zudem die Ausführung von Shell-Skripten vergleichsweise langsam ist, dauert der Start eines Betriebssystems auf diese Weise sehr lange. Beim Vorhandensein von Mehrkernprozessoren wäre jedoch der parallele Start solcher Prozesse problemlos möglich, zwischen denen gar keine Abhängigkeit besteht; der Systemstart würde sich erheblich verkürzen.

Beim systemd handelt es sich nicht um ein (langames) Shell-Skript sondern um ein kompiliertes Programm, das nicht nur in der Lage solche parallelen Startvorgänge zu ermöglichen, sondern das sich auch um die Anbindung von Geräten (Devices) oder Netzwerkspeichern (Mountpoints) kümmert. Der Ablauf des Startvorgangs ist für jeden Prozess über ein Unit-File geregelt. In einer solchen Textdatei sind beispielsweise die Abhängigkeiten definiert (Start vor oder nach anderen Prozessen), erforderliche Parameter, spezielle Rechte oder zusätzliche Aktionen, die vor dem Start oder nach dem Beenden des Dienstes ausgeführt werden sollen.

Das folgende Beispiel zeigt das Unit-File (Service-Datei) für den Mailserver „postfix“. Generell enthalten Service-Dateien (Endung „.service“) konkrete Handlungsanweisungen für den Start eines bestimmten Dienstes. Der Mailserver benötigt das zwingende Vorhandensein (Requires) u.a. des Netzwerk-Dienstes oder des sog. „Name Service Switch“-Dienstes (der den Zugriff auf verschiedene Systemdatenbanken regelt, z.B. wenn mehrere Informationsquellen für Benutzerkennungen oder Hostnamen existieren). Außerdem darf der Mailserver erst dann gestartet werden, wenn diese Dienste bereits aktiv sind (After). Zudem darf der Mailservice auch erst dann gestartet werden, wenn bspw. andere Dienste wie LDAP (u.a. für Benutzerkennungen), Cyrus (Mailspeicher-Dienste IMAP oder POP) oder der Mail-Virus-Scanner amavis bereits laufen, sofern sie vorhanden sind. Alternative Mailserver wie sendmail dürfen nicht parallel laufen und werden daher ggf. wieder gestoppt (Conflicts). Mittels der Service-Section werden die eigentlichen Programme (Binaries) gesetzt, die tatsächlich gestartet werden müssen (ExecStart), es werden Konfigurationsdateien für Umgebungsvariablen benannt (EnvironmentFile), Dateien für Prozesskennungen (PID, process ID) festgelegt oder der Typ des Dienstes definiert. Der hier gesetzte Typ „forking“ sagt dem systemd, dass es sich hierbei um einen Dienst handelt, der ausgehend vom eigentlichen Start- oder Elternprozess mehrere Kind-Prozesse abspaltert (engl. forking) und dann den Elternprozess wieder beendet.

Unit-File für den Mailserver „postfix“

/usr/lib/systemd/system/postfix.service

```
[Unit]
Description=Postfix Mail Transport Agent
Requires=var-run.mount nss-lookup.target network.target local-fs.target time-sync.target
After=var-run.mount nss-lookup.target network.target local-fs.target time-sync.target
After=amavis.service mysql.service cyrus.service ldap.service openslp.service ypbind.service
Conflicts=sendmail.service exim.service

[Service]
Type=forking
PIDFile=/var/spool/postfix/pid/master.pid
ExecStartPre=/bin/echo 'Starting mail service (Postfix)'
EnvironmentFile=/etc/sysconfig/postfix
ExecStartPre=/etc/postfix/system/config_postfix
ExecStartPre=/etc/postfix/system/update_chroot
ExecStartPre=/etc/postfix/system/update_postmaps
ExecStart=/usr/sbin/postfix start
ExecStartPost=/etc/postfix/system/wait_qmgr 60
ExecStartPost=/etc/postfix/system/cond_slp register
ExecReload=/usr/sbin/postfix reload
ExecReload=/usr/sbin/postfix flush
ExecStop=/usr/sbin/postfix stop
ExecStopPost=/etc/postfix/system/cond_slp deregister
```

Unix-Betriebssysteme kennen im Gegensatz zu Windows verschiedene Zustände, in die ein System gefahren werden kann (im Init-V-System als „Runlevel“ bezeichnet). Der übliche Systemzustand ist i.d.R. das Multi-User-System mit Netzwerkfunktionalität und grafischer Benutzeroberfläche (Runlevel 5). Es gibt aber auch den Systemzustand ohne grafische Oberfläche (Runlevel 3) oder das Herunterfahren des Systems (Runlevel 0). Dieses Konzept wird mit dem `systemd` wesentlich flexibler gestaltet. Mithilfe von Target-Dateien (Endung `.target`) können bestimmte Units zusammengefasst werden, aus denen ein bestimmter Systemzustand bestehen soll. Das Target `multi-user.target` startet das System ohne grafische Oberfläche (entspricht Runlevel 3), `rescue.target` fährt das System in einen Single-User-Modus ohne Netzwerk, `halt.target` fährt das System herunter. Der Standard-Zustand ist das Multi-User-System mit grafischer Oberfläche (`graphical.target`). Es können aber auch spezielle Zustände mithilfe von Targets definiert werden, z.B. ein „Mail-Server-Target“, das ausschließlich diejenigen Dienste aktiviert, die für die gewünschte Funktionalität eines Mail-Servers erforderlich sind.

Für alle installierten Dienste, die mithilfe des `systemd` gestartet werden können, gibt es entsprechende Unit-Files im Verzeichnis `/usr/lib/systemd/system` (distributions-abhängig, hier OpenSuse). Aber nicht alle diese Dienste müssen auch zwangsläufig aktiv sein. Erst durch das Aktiv-Schalten der Dienste werden diese beim Systemstart auch tatsächlich gestartet. Dies geschieht einfach dadurch, dass im Verzeichnis `/etc/systemd/system` symbolische Links auf die Target- oder Service-Dateien gesetzt werden, und nur die dort existierenden Dienste werden gestartet. Hierfür existieren auch grafische Tools sowie unter OpenSuse das Konfigurations-Tool „yast“.

Das Kommando `systemctl` dient zur Administration des `systemd`; hier nur einige Beispiele:

```
systemctl status postfix.service  Abruf des Zustands des Postfix-Mail-Dienstes
systemctl start postfix.service   nur als User root möglich: Start des Dienstes, entsprechend stop
oder restart (Dienst beenden und neu starten; falls der Dienst vorher nicht lief: starten), reload
(Konfigurationsdateien neu einlesen ohne den Dienst zu stoppen - wird nur von ausgewählten Diensten
unterstützt), reload-or-try-restart (falls reload nicht unterstützt wird, dann restart, dies aber nur wenn
der Dienst vorher bereits lief) usw.
```

Eine alternative Syntax ist: `service postfix status`

```
systemctl isolate multi-user.target (das System in den Multi-User-Target fahren (Runlevel 3),
hierbei werden alle in diesem Target nicht aktivierten Dienste gestoppt)
```

```
systemctl isolate graphical.target (das System in den grafischen Modus fahren (Runlevel 5))
```

```
systemctl list-units -t service  alle aktivierten (aber nicht unbedingt gestarteten) Dienste anzeigen
```

```
systemctl list-unit-files -t service  alle (auch die nicht aktivierten) Dienste anzeigen
```

```
systemctl enable postfix.service  Dienst aktivieren, entsprechend disable
```

4.4 Prozesse

Bei grafischen Oberflächen kann ein Programm – wie gewohnt - per Mausklick auf das Start-Menü (Symbol in der Task-Leiste unten links) ausgewählt und gestartet werden (alternativ mit Alt-F1 (bei Notebook-Tastaturen oft in Verbindung mit der Funktionstaste Fn) oder teilweise auch mit der Windows-Taste, dann weiter mit den Cursor-Tasten), bspw. Start-Menü – Dienstprogramme – Kcalc (Taschenrechner). Dieses Programm kann jedoch auch durch eine entsprechende Eingabe in einem Terminalfenster gestartet werden: `kcalc` (wobei das Expandieren des Dateinamens mithilfe der Tab-Taste möglich ist). Allerdings ist nun das Fenster, in dem das Programm gestartet wurde blockiert. Das Programm kann jedoch mit der Tastenkombination Strg-z unterbrochen und anschließend mit `bg` (Background) im Hintergrund fortgesetzt werden. Setzt man das Zeichen `&` hinter den Programmaufruf (`kcalc &`) so wird das Programm sofort im Hintergrund gestartet und das Terminalfenster steht für weitere Eingaben zur Verfügung. Mittels der Eingabe des Programmnamens können jedoch nur solche Programme gestartet werden, die zum einem als „ausführbar“ gekennzeichnet sind und zum anderen in einem Verzeichnis gespeichert sind, das in der systemweiten Pfad-Variablen (PATH) enthalten ist. Ist letzteres nicht der Fall, so muss entweder der vollständige Pfad beim Programmaufruf angegeben werden (`/usr/bin/kcalc`) oder man wechselt in dieses Verzeichnis und startet das Programm mit `./kcalc` (wobei der Punkt das aktuelle Verzeichnis bezeichnet).

Die im aktuellen Terminalfenster laufenden Prozesse lassen sich mit dem Kommando `ps` auflisten

```
PID TTY          TIME CMD
2406 pts/1      00:00:01 bash
9010 pts/1      00:00:00 ps
```

In der Spalte PID wird die Prozess-Identifikationsnummer angegeben, unter TTY (Terminal-Type) der Name des Terminals, in dem der Prozess läuft, TIME ist die verbrauchte Rechenzeit und CMD der Name des Kommandos. Hier läuft im Terminal namens pts/1 die Shell „bash“ und das Kommando „ps“.

Das Kommando `ps a` (im BSD-Style ohne „-“) gibt alle Prozesse des Benutzers aus, die aus einem Terminal heraus gestartet wurden:

```
PID TTY          STAT TIME COMMAND
1020 tty1      Ss+   0:00 /sbin/agetty --noclear tty1 linux
1243 tty7      Ssl+  1:18 /usr/bin/X -nolisten tcp -auth /run/sddm/{a404dd...
1801 pts/0      Ss    0:00 /bin/bash
2406 pts/1      Ss    0:01 /bin/bash
3596 pts/0      S     0:00 su -
3599 pts/0      S     0:00 -bash
9072 pts/0      S+    0:00 man ps
9085 pts/0      S+    0:00 less
9343 pts/1      R+    0:00 ps a
```

In der obersten Zeile (PID 1020) steht der ursprüngliche Login-Prozess (agetty), der standardmäßig im Terminal tty1 läuft, gefolgt vom Startprozess der grafischen Oberfläche (sog. X-Server, `/usr/bin/X`) in tty7. Es laufen zwei Terminalfenster pts/0 und pts/1 jeweils mit der „bash“, in einem dieser Fenster wurde das Kommando `su -` aufgerufen (Substitute User), um mit Superuser-Rechten (root) arbeiten zu können. Es wurde dort die Manualpage zu „ps“ aufgerufen, die seitenweise „less“ ausgegeben wird. Die Status-Zeile zeigt ruhende Prozesse (S: sleep) bzw. aktiv laufende (R: running), „s“ bedeutet „session leader“ also solche Prozesse, aus denen heraus weitere Prozesse gestartet wurden, sowie Vordergrundprozesse „+“ (weiteres siehe „man ps“)

Alle laufenden Prozesse lassen sich mit `ps -fe` ausgeben (siehe folgender Kasten). Zusätzlich ist in der folgenden Auflistung die Spalte PPID (Parent PID) enthalten, die den Eltern-Prozess angibt. Die beiden (bereits oben angesprochenen) wichtigsten Prozesse `systemd` (PID 1) und `kthreadd` (PID 2) haben keinen Elternprozess, da sie aus dem bootenden Kernel heraus gestartet wurden. Der `kthreadd` erzeugt die erforderlichen Kernel-Threads und der `systemd` startet alle Prozesse im User-Space, die in der Konfiguration des Betriebssystems als aktive Prozesse oder Dienste definiert sind, wie z.B. den FTP-Server `vsftpd` (PID 1068), den Druck-Dienst `cupsd` (1069), den Secure Shell-Dienst `sshd` (1145) oder den Display Manager `sddm` (Simple Desktop DM, 1185).

Letzterer startet beim Login-Versuch eines Benutzer (hier der User „admin“) einen Hilfsprozess (`sddm-helper` PID 1562, mit der PPID des `sddm` 1185), der sich wiederum um den Start der grafischen Oberfläche KDE kümmert (1570, PPID 1562), nachdem sich der Benutzer beim Dienst `systemd-pam` (1566) erfolgreich mit einem Passwort authentifiziert hat. Darauf aufbauend werden dann die für die grafische Oberfläche erforderlichen Prozesse gestartet, u.a. auch die Konsole (PID 1746) von der sich die bereits bekannten Terminalprozesse (bash usw.) ableiten.

```
# ps -fe
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	1	0	0	08:20	?	00:00:10	/usr/lib/systemd/systemd ...
root	2	0	0	08:20	?	00:00:00	[kthreadd]
root	3	2	0	08:20	?	00:00:00	[ksoftirqd/0]
root	5	2	0	08:20	?	00:00:00	[kworker/0:0H]
root	9	2	0	08:20	?	00:00:00	[migration/0]
root	10	2	0	08:20	?	00:00:00	[watchdog/0]
root	11	2	0	08:20	?	00:00:00	[watchdog/1]
root	12	2	0	08:20	?	00:00:00	[migration/1]
root	13	2	0	08:20	?	00:00:00	[ksoftirqd/1]
root	15	2	0	08:20	?	00:00:00	[kworker/1:0H]
root	1068	1	0	08:20	?	00:00:00	/usr/sbin/vsftpd /etc/vsftpd.conf
root	1069	1	0	08:20	?	00:00:00	/usr/sbin/cupsd -f
root	1145	1	0	08:20	?	00:00:00	/usr/sbin/sshd -D
root	1185	1	0	08:20	?	00:00:00	/usr/bin/sddm
root	1562	1185	0	08:20	?	00:00:00	/usr/lib/sddm/sddm-helper --socket /tmp/sddm-auth..
admin	1563	1	0	08:20	?	00:00:01	/usr/lib/systemd/systemd --user
admin	1566	1563	0	08:20	?	00:00:00	(sd-pam)
admin	1570	1562	0	08:20	?	00:00:00	/bin/sh /usr/bin/startkde
admin	1658	1	0	08:20	?	00:00:00	kdeinit5: Running...
admin	1659	1658	0	08:20	?	00:00:00	/usr/lib64/libexec/kf5/klauncher --fd=9
admin	1662	1658	0	08:20	?	00:00:04	kded5 [kdeinit5]
admin	1674	1	0	08:20	?	00:00:00	/usr/bin/kaccess
admin	1682	1570	0	08:20	?	00:00:00	kwrapper5 /usr/bin/kmsserver
admin	1686	1	0	08:20	?	00:00:00	/usr/bin/kglobalaccel5
admin	1687	1658	0	08:20	?	00:00:01	/usr/bin/kmsserver
admin	1694	1	0	08:20	?	00:00:00	/usr/lib/dconf-service
admin	1700	1687	0	08:20	?	00:00:59	kwin_x11 -session 101bd16a16415d00...
admin	1701	1687	0	08:20	?	00:00:04	baloo_file
admin	1702	1687	0	08:20	?	00:00:01	/usr/lib64/libexec/kdeconnectd
admin	1703	1687	0	08:20	?	00:00:07	krunner
admin	1704	1687	0	08:20	?	00:00:57	plasmashell --shut-up
admin	1746	1658	0	08:20	?	00:00:31	/usr/bin/konsole -session
admin	1801	1746	0	08:20	pts/0	00:00:00	/bin/bash
root	3596	1801	0	11:30	pts/0	00:00:00	su -
root	3599	3596	0	11:30	pts/0	00:00:00	-bash
man	9072	3599	0	15:35	pts/0	00:00:00	man ps
man	9085	9072	0	15:35	pts/0	00:00:00	less

Besser sichtbar wird das Verhältnis von Eltern- und Kind-Prozessen mit dem Kommando pstree (kurze Erläuterungen zu einigen Prozessen und ihren Aufgaben im Kasten)

```

systemd-+-ModemManager-+-{gdbus}      (in geschweiften Klammern: Child-Prozesse bzw. Bibliotheksfunktionen)
|                                     `-{gmain}
|-NetworkManager-+-{gdbus}
|-agetty           (alternative get Terminal Type: User-Login und Start eines Virtual Terminals)
|-auditd---{auditd}
|-avahi-daemon     (Zeroconf-Netzwerkdienst, IP 169.254.0.0/16, ähnlich „Bonjour“)
|-bluetoothd
|-cron             (Dienst zur Ausführung zeitgesteuerter Kommandos)
|-cupsd---dbus     (Drucker-Dienst)
|-4*[dbus-daemon]  (Desktop-Bus: Interprozess-Kommunikation, z.B. USB-Stick öffnet Dateimanager)
|-3*[dbus-launch]
|-dconf-service-+-{gdbus}  (Gnome-Konfigurationsdatenbank)
|-fail2ban-server  (schützt z.B. den SSH-Server vor zu vielen Login-Versuchen)
|-haveged          (Zufallszahlengenerator, HAVEGE-Algorithmus, nutzt die (im normalen Betrieb massenweise auftretenden) Status-
                    änderungen der CPU zur Berechnung von Zufallszahlen. Alternativen: /dev/random (gut aber langsam, nutzt
                    Hardware-Interrupts) und /dev/urandom)
|-irqbalance      (Interrupt-Verteilung bei Mehrkernprozessoren)
|-kaccess-+-{QDBusConnection}
|-kactivitymanage-+-{QDBusConnection}
|                                     |-4*[{QThread}]
|                                     `-{QXcbEventReader}
|-kde5-nm-connect-+-{QDBusConnection}  (Network-Manager)
|                                     `-{QXcbEventReader}
|-kdeinit5-+-kded5-+-{KCupsConnection}  (Init-Prozess für die grafische Oberfläche KDE)
|                                     |-{QDBusConnection}
|                                     |-{QThread}
|                                     |-{QXcbEventReader}
|                                     |-{Qt bearer threa}
|                                     |-{gdbus}
|                                     `-{gmain}
|                                     |-klauncher-+-{QDBusConnection}
|                                     `-{QXcbEventReader}
|                                     |-konsole-+-bash      (Prozess für Terminalfenster, darin ist als Sub-Prozess die bash gestartet)
|                                     |-bash---pstree      (... und in dieser bash das Kommando pstree)
|                                     |-ksmserver-+-      (KDE Session Manager, für Auto-Start-Anwendungen und Speicherung von Session-Zuständen)
|                                     | baloo_file---{QDBusConnection}  (Datei-Indizierungs-Dienst)
|                                     |-kdeconnectd-+-{QDBusConnection}  (KDE Kontakte-Dienst)
|                                     |-krunner-+-{QDBusConnection}
|                                     |-kwin_x11-+-{QDBusConnection}      (Fenster-Management)
|                                     |-plasmashell-+-      (zuständig u.a. für das Menü-Layout und das Starten von
|                                                             Programmen in der grafischen Oberfläche)
|                                     |-kate-+-{QDBusConnection}  (Editor „kate“)
|                                     |-ksysguardd          (eine Art „Task Manager“)
|                                     |-xdg-su---kdesu-+-su---kdesu_stub---yast2--
|                                                             (Start des Konfigurations-Tools Yast, das Root-Rechte erfordert, daher ist zuvor
|                                                             der Rechte-Wechsel notwendig (su: Substitute User))
|-kdesud
|-mcelog
|-nscd---10*[{nscd}]  (Nameservice-Cache DNS)
|-obexd             (Client-Server-Dienst insbes. Zur Kommunikation von Bluetooth-Geräten)
|-polkitd-+-{JS GC Helper}  (Interprozess-Kommunikation)
|-pulseaudio-+-{alsa-sink-HDMI }  (Audio-Dienst)
|                                     `-{alsa-sink-USB A}
|-sddm-+-X---{X}      (Display-Manager)
|                                     |-sddm-helper---startkde-+-gpg-agent  (Verwaltung kryptografischer Schlüssel)
|                                     |                                     |-kwrapper5
|                                     |                                     `-ssh-agent
|-smartd            (Festplatte-Überwachungsdienst)
|-sshd              (Secure-Shell-Service)
|-start_kdeinit     (Startprozess der grafischen Oberfläche)
|-2*[systemd---(sd-pam)]  (PAM: Pluggable Authentication Module, Verwaltung von User Sessions)
|-systemd-journal    (Sammeln von Logging-Daten)
|-systemd-logind
|-systemd-timesyn---{sd-resolve}
|-systemd-udev       (Userspace Device Daemon, überwacht Hotplug-Ereignisse und fügt automatisch Gerätedateien ins /dev-
                    Verzeichnis ein)
|-udisksd-+-{cleanup}  (D-Bus-Funktionalität für Storage Devices)
|                                     `-{probing-thread}
|-upowerd-+-{gdbus}    (Energie-Management)
|-vsftpd            (FTP-Service)
`-wpa supplicant     (Wireless Protected Access, Verschlüsselung im WLAN)

```

5 Dateisysteme

5.1 Grundlagen

Ein Dateisystem ermöglicht dem Betriebssystem den Zugriff auf Dateien und Verzeichnisse. Hierzu muss ein Ordnungssystem geschaffen werden, das mithilfe von Adressen und Metadaten die Dateien auf dem Speichermedium lokalisieren kann. Auf dem Speichermedium muss in Form von Partitionen bzw. der Partitionstabelle eine technische Grundstruktur vorhanden sein (siehe Kapitel 4.1), die Start- und Endpunkte der nutzbaren Bereiche definiert. Über diese technische Struktur des Speichermediums (Festplatte) wird eine logische Struktur gelegt, über die das Betriebssystem die Festplatte verwaltet. Beim **Formatieren** der Partition werden die wesentlichen Parameter eines Dateisystems festgelegt. Es existieren verschiedene Typen von Dateisystemen, mit unterschiedlichen Eigenschaften, wobei nicht jedes Dateisystem von jedem Betriebssystem unterstützt wird.

Ein wichtiges (später nicht mehr veränderbares) Element beim Anlegen eines Dateisystems ist die Blockgröße: Unabhängig von der physikalischen Blockgröße des Speichermediums, verwenden Dateisysteme oft größere Blöcke (z.B. 1024 Byte = 1 KiB, 2 KiB, 4 KiB), um einen effizienten Lese- und Schreibzugriff in Abhängigkeit von der durchschnittlichen Dateigröße zu ermöglichen. Ein Block im Dateisystem ist die kleinste Einheit bei der Speicherung einer Datei. Sollte eine Datei größer sein als die Blockgröße, so wird sie auf mehrere Blöcke verteilt, wobei die einzelnen Blöcke an verschiedenen Stellen des Speichermediums abgelegt sein können. Es ist die Aufgabe des Dateisystems, mithilfe einer geeigneten Adressierung diese Blöcke beim Lesen der Datei wieder aufzufinden und sie einem Dateinamen zuzuordnen. Sollte eine Datei kleiner als die Blockgröße sein, so wird trotzdem mindestens der Speicherplatz eines ganzen Blockes belegt. Daher sollte vor dem Formatieren bekannt sein, welche Art von Daten gespeichert werden, um die Blockgröße geeignet festzulegen: Sind die Blöcke sehr groß, so können zwar große Dateien effizient gespeichert werden, aber bei sehr kleinen Dateien wird eventuell viel Speicherplatz verschwendet, weil einzelne Dateien die jeweiligen Blöcke nicht ausfüllen. Sind die Blöcke sehr klein, so müssen große Dateien auf sehr viele Blöcke verteilt werden, was ggf. die Lese- und Schreibgeschwindigkeit verringert.

Heute stehen viele unterschiedliche Typen von Dateisystemen zur Verfügung: Von einfachen, klassischen Systemen (Vorteil: einfache Verwaltungsstrukturen), über Journaling-Systeme (Vorteil: Aufzeichnen von Änderungen zur effizienten Fehlerkontrolle) bis hin zu modernen Systemen, die Baum-Datenstrukturen zur Ablage der Metadaten verwenden (sehr aufwändige, aber sehr schnelle Arbeit mit den Metadaten), existieren diverse Alternativen für die Wahl des geeigneten Dateisystems.

5.2 Einbinden von Dateisystemen

Das Betriebssystem Linux ordnet jeder Hardware ein so genanntes Device-File (Geräte-datei) zu, das als Schnittstelle zwischen Betriebssystem und Hardware dient. Ein Device-File wird automatisch beim Starten des Betriebssystems oder beim Einhängen der Hardware angelegt. Eine heute übliche SATA-Festplatte erhält im Verzeichnis `/dev` ein Device-File beispielsweise mit der Bezeichnung `sde`, wobei „sd“ SCSI-Disk bedeutet und „e“ ein laufender, dynamisch vergebener Buchstabe ist. Werden Partitionen auf dieser Festplatte angelegt (siehe Kapitel 4.1), so erhält jede Partition eine laufende Nummer: `/dev/sde3` ist die Bezeichnung für die dritte Partition auf der Festplatte `sde`. Der Nachteil dieser Kurzbezeichnung ist, dass sie sich nach jedem Neustart des Systems oder nach dem Einhängen zusätzlicher Hardware ändern kann. Daher werden (sofern es technisch möglich ist) zusätzlich z.B. in den Unterverzeichnissen `/dev/disk/by-id` oder `/dev/disk/by-partuuid` Links (Verknüpfungen) zu den jeweils aktuellen Device-Files angelegt, deren Benennungen fest mit hardware-abhängigen Parametern wie Seriennummern oder sog. World Wide Names verknüpft sind und sich daher auch beim Anschluss an andere Computer nicht ändern.

Die Formatierung wird mit (in Abhängigkeit vom Dateisystem) unterschiedlichen Kommandos durchgeführt. Das Kommando `mke2fs` legt ext2, ext3 oder ext4-Systeme an („Extended Filesystem“) oder `mkfs.btrfs` ein B-Tree-Filesystem, z.B. `mke2fs -t ext2 -b 2048 /dev/sdb1` (Typ ext2, Blockgröße 2048 Byte).

Das Betriebssystem benötigt einen Zugriffspunkt, um auf dieses Dateisystem zugreifen zu können. Es muss daher ein sog. Mount-Point existieren, in den die (formatierte) Partition „eingehängt“ (engl.: mount) wird (dieser Mount-Point muss als Verzeichnis vorhanden sein): `mount /dev/sdb1 /mnt/mein_verzeichnis`

Wichtiger Hinweis: Dieser Mount-Point ist ein normales Verzeichnis innerhalb der Root-Partition, über das beim Mounten das externe Dateisystem lediglich „drübergelegt“ wird. Das bedeutet, dass zum einen der vorhandene Inhalt dieses Verzeichnisses beim Mounten nicht gelöscht oder verändert wird, zum anderen werden beim Fehlschlagen des Mountens alle Daten nicht in das externe Dateisystem sondern in das Root-Filesystem geschrieben.

5.3 Dateisystem ext2

Das ext2-Dateisystem ist ein einfaches, klassisches Linux Standard-Dateisystem ohne Journaling (Vorteil des „Journaling“ siehe Kapitel 5.4), das in der Praxis heute nur noch selten eingesetzt wird. Anhand von ext2 lassen sich jedoch die grundsätzlichen Funktionen und Parameter von Dateisystemen gut darstellen.

Die wichtigste Aufgabe eines Dateisystems ist die Verknüpfung einer Datei mit den Datenblöcken auf dem Speichermedium. Hierzu wird der Dateiname (sowie wichtige Parameter wie Datumsangaben oder Benutzerrechte) zusammen mit den Nummern aller Blöcke, in denen die Daten auf dem Speichermedium abgelegt sind, in einem sog. „Index-Knoten“ (**Inode**) gespeichert. Alle Datei- und Verzeichnisnamen werden in einer **Inode-Tabelle** zusammengefasst. Wird auf eine Datei zugegriffen, wird zunächst die Dateinummer in der Inode-Tabelle gesucht, dann geprüft, ob der betreffende Benutzer die für diese Aktion erforderlichen Rechte besitzt und anschließend alle zugehörigen Blöcke auf dem Speichermedium gelesen. Die Inode-Tabelle selbst ist auch direkt auf dem Speichermedium in einer speziellen Struktur am Anfang einer Partition abgelegt.

Abbildung 3 zeigt die Dateisystemstruktur, die bei ext2 (hinter dem MBR- bzw. GPT-Bootblock) aus mehreren Block-Groups besteht. Die Block-Groups enthalten u.a. jeweils die Inode-Tabelle und dahinter die Daten-Blöcke.

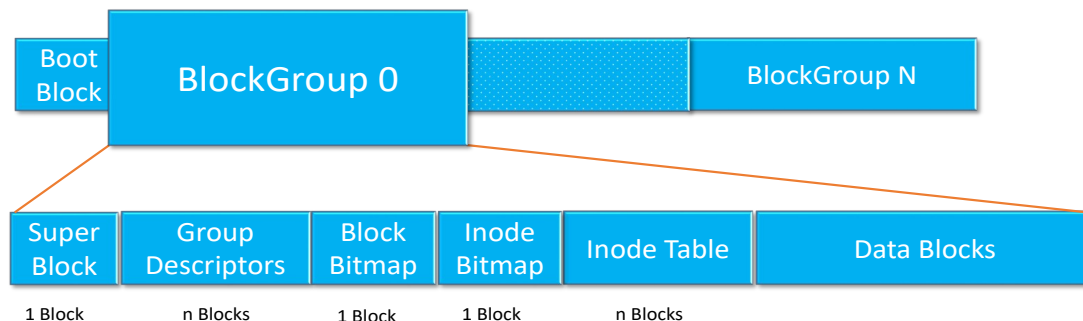


Abbildung 3: Dateisystemstruktur ext2

Die erste Block-Group beginnt mit dem Superblock. Er enthält wichtige Informationen zum Dateisystem: Maximale Anzahl der Blöcke und Inodes, die Blockgröße, die Anzahl der Blöcke und Inodes innerhalb einer Block-Group, wie viele Blöcke die Inode-Tabelle umfasst sowie Daten zu „Last Mount“, „Last Checked“, „Check Intervall“ und Filesystem-ID. Hinter dem Superblock folgt immer der Group Descriptor, in dem die Block-Nummern der folgenden Elemente (Block-Bitmap, Inode-Bitmap, Inode-Tabelle) gespeichert sind; außerdem enthält er die Anzahl der aktuell noch freien Blöcke und freien Inodes.

Da die gesamte anschließende Datenstruktur ohne diese Informationen aus dem Superblock und dem Group Descriptor nicht mehr auflösen wäre, werden Backup-Kopien in weiteren Block-Groups angelegt, nach festem Schema in den Block-Groups Nr. 3, 5, 7 und Potenzen davon, z.B. $3^3=27$ oder $7^2=49$ usw.

Die Block-Bitmap zeigt an, welche Blöcke konkret belegt oder noch frei sind (Null-Bit: frei, Eins-Bit: belegt). Die Größe der Block-Bitmap beträgt einen Block, weshalb die tatsächliche Größe der Bitmap von der beim Formatieren festgelegten Blockgröße abhängt: Bei 1024 Byte Blockgröße kann die Block-Bitmap maximal $1024 * 8$ Bit enthalten und somit 8192 Blöcke als belegt anzeigen, was einer maximalen Datenmenge von $8192 * 1024$ Byte (entsprechend 8 MiB) pro Block-Group entspricht (bei Blockgröße 4096 Byte entsprechend 128 MiB). Die Inode-Bitmap zeigt an, welche Inodes in der Inode-Tabelle frei oder belegt sind. Danach folgen die Datenblöcke.

In der Inode-Tabelle, deren Größe im Superblock angegeben ist (z.B. 128 Blöcke), gibt es für jede Datei (referenziert über ihre Inode-Nummer) einen Eintrag (dessen Größe ist ebenfalls im Superblock definiert, z.B. 256 Byte). Der Inode-Nummer einer Datei sind die Nummern derjenigen Blöcke zugeordnet, die die Nutzdaten der Datei enthalten, sowie alle Datei-Attribute.

Abbildung 4 zeigt alle Elemente eines Inode-Eintrags innerhalb der Inode-Tabelle.

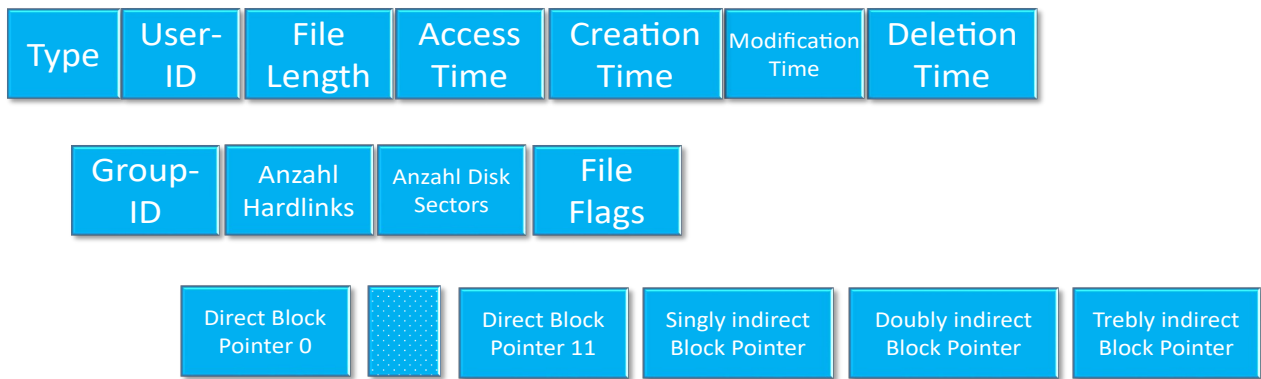


Abbildung 4: Inode-Struktur

Der Inode-Eintrag beginnt mit dem Datei-Typ (16 Bit z.B. reguläre Datei, Verzeichnis, Link, Named Pipe), es folgen Parameter für eine Datei, wie z.B. Besitzer der Datei, Größe und Zeitstempel (je 32 Bit). Die letzten Elemente im Inode-Eintrag sind die Block Pointer (je 32 Bit): Die 12 Direct Block Pointer enthalten die Blocknummern (Logical Block Address) derjenigen Blöcke auf der Festplatte, in denen die Nutzdaten gespeichert sind. Die 12 Direct Block Pointer können bei einer Blockgröße von 4096 Byte maximal eine Dateigröße von $12 \cdot 4 \text{ kiB} = 48 \text{ kiB}$ adressieren. Bei größeren Dateien wird der „Singly Indirect Block Pointer“ (SIBP) verwendet, der auf einen Block zeigt, der seinerseits Block Pointer enthält: Bei einer Blockgröße von 4096 Byte können 1024 Block Pointer in einem Block gespeichert werden ($4096 \text{ Byte Blockgröße} \cdot 8 \text{ Bit} / 32 \text{ Bit (Größe eines Pointers)} = 1024$), sodass eine Dateigröße von maximal $1024 \cdot 4096 \text{ Byte} = 4 \text{ MiB}$ adressiert werden kann. Reicht dies nicht aus, so wird der „Doubly Indirect Block Pointer“ (Abbildung 5) verwendet: Er verweist auf einen Block mit 1024 SIBP (zu je 32 Bit), die ihrerseits auf 1024 Blöcke mit jeweils 1024 Block Pointern verweisen (maximale Dateigröße: $1024 \cdot 1024 \cdot 4096 = 4 \text{ GiB}$). Reicht auch das nicht aus, so wird der „Trebly Indirect Block Pointer“ eingesetzt, der theoretisch Dateigrößen bis zu 4 TiB verwalten könnte, allerdings speichert ein Inode (Feld: Anzahl Disk Sectors) die Anzahl von Festplatten-Sektoren (zu je 512 Byte), sodass dadurch maximal $512 \cdot 2^{32} = 2 \text{ TiB}$ adressiert werden können.

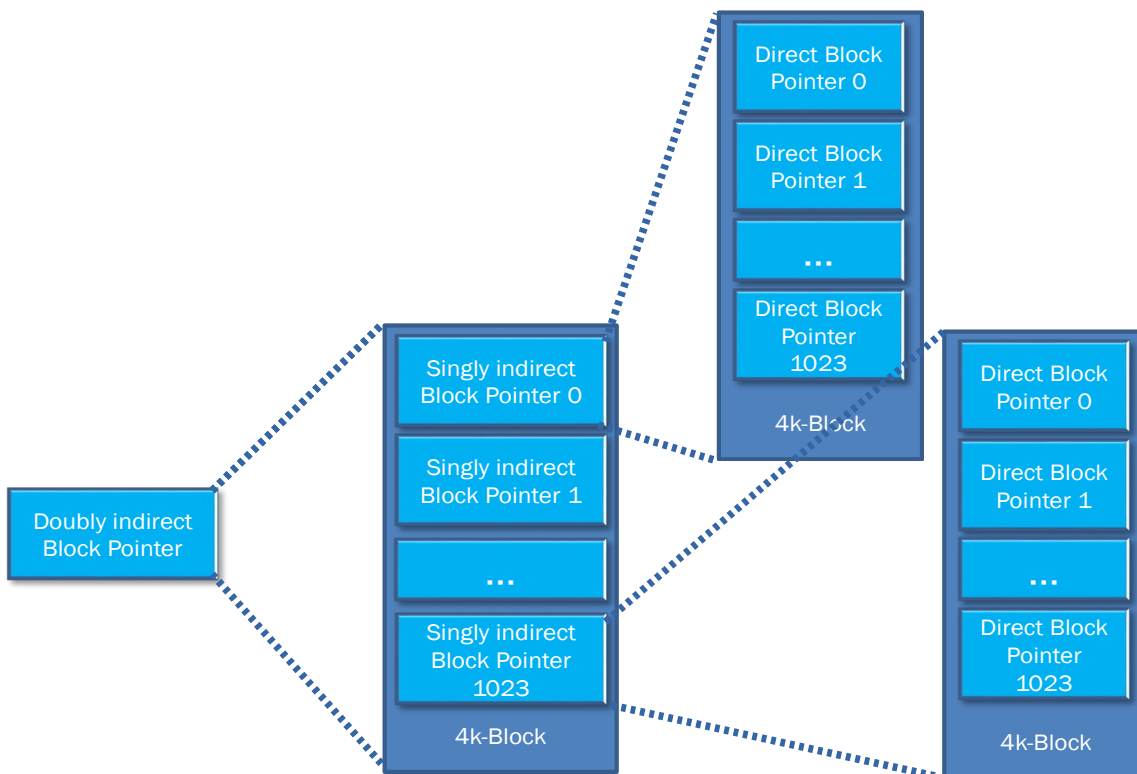


Abbildung 5: Doubly Indirect Block Pointer

Die Position eines Inode-Eintrags in der Inode-Tabelle entspricht der Inode-Nummer. Diese Inode-Nummer muss mit dem Dateinamen verknüpft werden. Dies erfolgt in der Verzeichnis-Datei. Aus Sicht eines Benutzers scheint ein Dateisystem eine Verzeichnis-Hierarchie zu besitzen (z.B. `/etc/apache2/config`): in Wirklichkeit gibt es diese Hierarchie aber gar nicht; alle Dateien sind lediglich als Blöcke gespeichert, die mithilfe der Inode-Tabelle gefunden werden können. Dasselbe gilt auch für Verzeichnisse (Ordner): Ein Verzeichnis wird lediglich mittels einer Datei definiert, in der die Namen der im Verzeichnis abgelegten Dateien und Unterverzeichnisse eingetragen sind. Diese Datei wird genauso auf der Festplatte gespeichert, wie reguläre Dateien. In den Verzeichnis-Dateien sind die Namen der Dateien bzw. Unterverzeichnis-Dateien zusammen mit ihren Inode-Nummern gespeichert. Wenn ein Benutzer eine Datei lesen möchte, so wird die entsprechende Verzeichnis-Datei gesucht (genauer: die zugehörigen Blöcke auf der Festplatte), dann geöffnet und die Inode-Nummer der gewünschten Datei ausgelesen. Mit dieser Inode-Nummer kann anschließend der Inode-Eintrag der Datei gesucht und damit die Nummern der Nutzdaten-Blöcke gefunden werden.

Abbildung 6 zeigt vereinfacht den Ablauf beim Öffnen einer Datei bzw. beim Lesen der zugehörigen Datenblöcke.

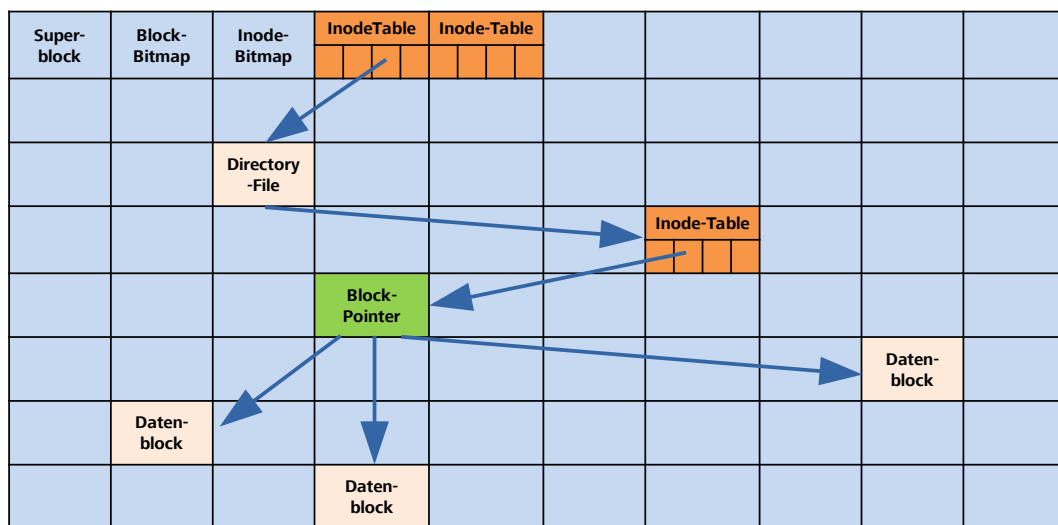


Abbildung 6: Datenblöcke finden (Prinzip)

Ein detailliertes Beispiel soll die Vorgehensweise beim Öffnen einer bestimmten Datei aufzeigen: Es soll die Konfigurationsdatei `/etc/postfix/main.cf` des Mailservers „postfix“ geöffnet werden.

Die Suche beginnt im Root-Verzeichnis „/“. Die Inode-Nummer der Verzeichnis-Datei (Directory-File) des Root-Verzeichnisses ist fest als Inode Nr. 2 definiert. Daher muss zuerst Inode Nr. 2 gesucht werden:

1. In welcher Block-Group befindet sich die Inode-Tabelle mit Inode-Nr. 2?

$$\text{BLOCKGROUP} = (\text{INODE} - 1) / \text{INODES_PER_GROUP}$$
Hinweis: Es gibt keinen Inode Null (daher „- 1“), „Inodes-per-Group“ steht im Superblock z.B. 512.
Für Inode-Nr. 2 ergibt sich damit $\text{BLOCKGROUP} = 0$. Für einen Inode 1200: $\text{BLOCKGROUP} = 1199/512 = 2$ (der Nachkomma-Anteil wird abgeschnitten).
2. Welchen Index hat der Eintrag der gesuchten Inode-Nr. innerhalb der Inode-Tabelle, bzw. an welcher Stelle der Inode-Tabelle steht der Eintrag?

$$\text{INDEX} = (\text{INODE} - 1) \bmod \text{INODES_PER_GROUP}$$
Für den gesuchten Eintrag Inode-Nr. 2 ergibt sich $\text{INDEX} = 1$. Für Inode 1200: $\text{INDEX} = 1199 \bmod 512 = 175$, d.h. Inode 1200 befindet sich im 175. Inode-Eintrag von Block-Group 2
3. Da eine Inode-Tabelle über mehrere Blöcke geht, muss derjenige Block innerhalb der Inode-Tabelle gefunden werden, in dem sich der gesuchte Index 1 befindet:

$$\text{CONTAINING_BLOCK} = (\text{INDEX} * \text{INODE_SIZE}) / \text{BLOCK_SIZE}$$
Die Größe eines Inodes sei 256 Byte, die Blockgröße 1024 Byte: $\text{CONTAINING_BLOCK} = (1 * 256) / 1024 = 0$
Bei Inode 1200: $\text{CONTAINING_BLOCK} = (175 * 256) / 1024 = 43$
4. Mit diesen Werten kann nun der tatsächliche Block im Dateisystem gefunden werden, wobei hier davon ausgegangen wird, dass es sich um eine „normale“ Block-Group ohne Superblock und Group Descriptor handelt:

$$\text{BLOCK_NR} = \text{BLOCKGROUP} * \text{BLOCKS_PER_GROUP} + 2 + \text{CONTAINING_BLOCK}$$

Da die Inode-Tabelle im dritten Block der Block-Group beginnt: „+2“.

$BLOCK_NR = 0 * 8192 + 2 + 0 = 2$ (Hinweis: dies ist kein realistischer Wert, da Block-Group 0 immer den Superblock enthält). Bei Inode 1200: $BLOCK_NR = 2 * 8192 + 2 + 43 = 16429$

5. Nachdem dieser Block eingelesen wurde, muss noch die Byte-Position des gewünschten Inode innerhalb dieses Blockes gefunden werden (denn bei Blockgröße 1024 und Inode-Größe 256 Byte gibt es vier Inode-Einträge pro Block):

$INODE_BYTE = INODE_SIZE * [INDEX \bmod (BLOCK_SIZE / INODE_SIZE)]$

$INODE_BYTE = 256 * [1 \bmod 4] = 256$. Bei Inode 1200: $256 * [175 \bmod 4] = 768$

6. Mithilfe dieses Inode-Eintrags können die Datenblöcke der Root-Verzeichnis-Datei gelesen werden. In dieser Datei gibt es den Eintrag für das Unterverzeichnis mit dem Namen `etc` verknüpft mit der dazugehörigen Inode-Nummer. Die Suche beginnt nun von vorne mit der Inode-Nummer der Verzeichnis-Datei von `etc` (z.B. 1200). Dort findet sich die Inode-Nummer der Verzeichnis-Datei von `postfix` und letztlich dort die Inode-Nummer der gesuchten Datei `main.cf`

5.4 Dateisysteme mit Journaling

Beim Einhängen eines Dateisystems kann zwischen dem synchronen und den asynchronen Schreib-Modus unterschieden werden. Beim **synchronen** Schreiben gibt die Festplatte erst dann eine positive Rückmeldung über den erfolgreich abgeschlossenen Schreibvorgang, wenn tatsächlich alle Bytes auf der physikalischen Hardware gespeichert sind. Beim **asynchronen** Schreiben werden die Daten vom Betriebssystem zunächst in einen schnellen Zwischenspeicher (Cache) geschrieben; hierbei kann es sich um einen Speicher-Chip innerhalb der Festplatten-Hardware handeln oder um den Hauptspeicher (RAM) des Computers. Aus diesem Pufferspeicher heraus werden die Daten auf die (langsamere) Festplatten-Hardware physikalisch geschrieben. Bereits nach dem Schreiben in den Pufferspeicher wird eine positive Rückmeldung über den erfolgreichen Schreibvorgang gegeben, sodass die schreibende Anwendung sich schneller wieder mit anderen Aufgaben beschäftigen kann. Daher ist das synchrone Schreiben aus Sicht der Anwendung bzw. des Benutzers erheblich langsamer als das asynchrone Schreiben, weshalb letzteres der Standard beim Einhängen eines Dateisystems ist (synchrones Schreiben wird in der Praxis nur dann verwendet, wenn bei hohen Sicherheitsanforderungen an Schreibprozesse geringe Schreibraten zu erwarten sind). Wird hierbei der Kopiervorgang vom Pufferspeicher auf die Festplatte wegen eines technischen Problems (Stromausfall, Prozess- oder Betriebssystem-Absturz) nicht vollständig abgeschlossen, entspricht der Zustand der Inode-Tabelle, die sofort nach der positiven Rückmeldung angepasst wurde, evtl. nicht mehr dem Zustand der tatsächlichen Blöcke auf der Festplatte (z.B. verweist ein Inode-Eintrag auf einen Block, der keine oder nicht verwertbare Daten enthält (u.a. fehlendes EOF)). Ein Dateisystem führt daher nach einem Systemabsturz oder in regelmäßigen Abständen einen Filesystem-Check durch, um solche Fehler aufzuspüren und – sofern möglich – zu reparieren (ggf. unter Verlust einzelner Dateien). Bei großen Partitionen dauert dieser Vorgang jedoch sehr lange (ggf. mehrere Stunden, in denen das Filesystem nicht genutzt werden kann).

Abhilfe schaffen hier so genannte „Journaling-Filesysteme“ (`ext3` und `ext4`), die – vereinfacht ausgedrückt – wie folgt funktionieren:

Beim Anlegen oder Ändern einer Datei werden die Daten zunächst in das so genannte „Journal“ (ein Transaktions-Log) physikalisch auf die Festplatte geschrieben. Erst wenn diese Aktion abgeschlossen ist, wird die eigentliche Schreiboperation gestartet, bei der die Daten in die Blöcke auf der Festplatte aus dem Journal heraus geschrieben werden. Kommt es zu einem Crash während dieser Aktionen, so gibt es folgende Szenarien:

- Das Journal wird während des Crash geschrieben: Die Daten im Journal sind unvollständig und können nach dem Neustart des Dateisystems nicht wiederhergestellt werden. Es kommt hierbei zu einem Verlust der zu aktualisierenden Daten, aber das Dateisystem ist in einem konsistenten Zustand, da die eigentlichen Datenblöcke noch nicht beschrieben wurden, und die Daten sind im bisherigen Zustand erhalten.
- Die Datenblöcke werden während des Crash geschrieben: Das Dateisystem ist nun in einem inkonsistenten Zustand, aber die bereits durchgeführten bzw. noch auszuführenden Änderungen sind vollständig im Journal gespeichert, sodass ein Filesystem-Check alle Änderungen aus dem Journal heraus neu einspielen kann. Eine Datei ist danach im aktuellsten Zustand vorhanden.

Ein wesentlicher Vorteil ist hierbei, dass ein nach dem Crash erforderlicher Filesystem-Check nicht alle Blöcke „anfassen“ muss, um die Konsistenz des Dateisystems zu prüfen, sondern nur diejenigen, die als inkonsistent mithilfe des Journals markiert sind. Der hier beschriebene „Data Mode: Journal“ bietet zwar die höchste Sicherheit, ist aber wenig performant, da alle Daten zweimal geschrieben werden müssen. Eine höhere Schreibgeschwindigkeit, aber auch eine geringere Sicherheit bietet der „Data Mode: Writeback“, bei dem nur die Metadaten einer Datei (z.B. Besitzer, Rechte Zeitstempel) ins Journal geschrieben werden, die eigentlichen

Dateninhalte einer Datei aber direkt in die endgültigen Blöcke. Kommt es während des Schreibvorgangs zu einem Crash, so enthalten die einer Datei per Inode zugeordneten physikalischen Blöcke teilweise bereits neue und teilweise noch alte Daten. Zwar kann per Journal die alte Zuordnung „Inode zu Blöcken“ mit den alten Metadaten wiederhergestellt werden, sodass das Dateisystem wieder konsistent ist, aber die Blöcke enthalten evtl. keine verwertbaren Daten. Alternativ sind die Blöcke noch gar nicht beschrieben (d.h. die bisherigen Daten sind noch vorhanden), aber im Journal stehen bereits die neuen Metadaten: Dann ist nach einem Filesystem-Check der (scheinbar) neue Dateizustand mit den alten Daten verknüpft. Aus diesem Grund dreht der „Data Mode: Ordered“ die Reihenfolge um: Erst werden die Daten geschrieben, danach die Metadaten ins Journal. Unvermeidlich können auch hierbei Daten verloren gehen, aber in dem Fall, bei dem die neuen Daten bereits geschrieben sind (das Journal aber noch nicht), sind die neuen Daten lediglich mit den alten Metadaten verknüpft. Beide letztgenannten Modi sorgen zwar für eine schnelle Dateisystem-Konsistenz nach einem Crash, können aber Verluste von Dateiinhalten nicht vermeiden.

5.5 Dateisystem ext4

Die Einführung des Journaling-Dateisystems ext3 brachte zwar Verbesserungen bzgl. der Dateisystem-Konsistenz, allerdings ist ext3 aufgrund der 32-Bit-Blocknummern auf eine maximale Größe von 16 TiB beschränkt (bei Blockgröße 4096 Byte: $4096 * 2^{32} = 16 \text{ TiB}$) und die maximale Größe einer einzelnen Datei auf 2 TiB. Das Dateisystem ext4 verwendet 48-Bit-Blocknummern, womit sich die maximale Größe eines Dateisystems auf 1 Exabyte erhöht (Exabyte: 10^{18} Byte, Exbibyte: 2^{60} Byte) und die Größe einer einzelnen Datei auf 16 TiB. Wichtigste Änderung gegenüber ext3 ist die Verwendung so genannter „Extents“ zur Blockadressierung. Das von ext2 und ext3 verwendete Schema der direkten und indirekten Adressierung einzelner Blöcke ist dann gut geeignet, wenn viele kleine Dateien vorhanden sind oder bei starker Fragmentierung der Festplatte. Bei großen Dateien erhöht sich jedoch der Verwaltungsaufwand, da auch bei unmittelbar aufeinander folgenden Blöcken jeder Block einzeln adressiert werden muss. Das Dateisystem ext4 arbeitet stattdessen mit Extents, mit denen möglichst große zusammenhängende Bereiche auf der Festplatte angesprochen werden können. Im Idealfall wird eine Datei nur mit einer Start-Blocknummer und der Bereichsgröße adressiert. Im ungünstigen Fall einer starken Fragmentierung der Festplatte ist dieses Verfahren allerdings weniger effizient als einzelne Blocknummern. Da die Blöcke *innerhalb* einer Datei mit 32 Bit adressiert werden, ist die maximale Dateigröße auf $4096 * 2^{32} = 16 \text{ TiB}$ beschränkt.

Die grundsätzliche Datenstruktur von ext2 und ext3 ist nicht verändert: auch hier gibt es Block-Groups mit maximal 128 MiB (32768 Blöcke mit 4096 Byte), Block- und Inode-Bitmaps sowie die Inode-Tabellen. Es wird jedoch der 32-Bit-Bereich der der Block-Pointer (12 direkte, zzgl. SIBP, DIBP, TIBP: $15 * 32 \text{ Bit} = 60 \text{ Byte}$) in der Inode-Tabelle für vier Extents und einen Extent-Header zu 12 Byte genutzt.

Diese 12-Byte Extent-Struktur besteht aus einem 48-Bit-Wert für den physikalischen Startblock, einem 32-Bit-Wert für den Start-Block *innerhalb* einer Datei (falls eine Datei in mehrere Fragmente geteilt ist, muss der Startpunkt jedes Fragments innerhalb der Datei gefunden werden) und einem 16-Bit-Wert für die Bereichslänge (siehe Abbildung 7). Die maximale Länge des Bereichs beträgt aufgrund der Länge einer Block-Group (s.o.) maximal 128 MiB. Bei vier Extents kann eine so adressierte Datei maximal $4 * 128 \text{ MiB} = 512 \text{ MiB}$ groß sein. Bei größeren Dateien wird ein Baum aus Extents (H-Baum) aufgebaut (siehe Kapitel 9.4): die eigentlichen Extent-Werte werden durch Extent-Indizes ersetzt, die auf Blöcke mit weiteren Extent-Indizes verweisen, die letztlich auf Blöcke mit den tatsächlichen Extents verweisen.

Logischer Startblock 32 Bit	Bereichslänge 16 Bit	Physikalischer Startblock 48 Bit
-----------------------------	----------------------	----------------------------------

Abbildung 7: 12-Byte Extent-Struktur bei ext4

Bei ext4 gibt es weitere Optimierungen, wie z.B. die Pre-Allocation, die es einer Anwendung ermöglicht, dem Dateisystem bereits vor dem Schreiben mitzuteilen, wie groß die Datei sein wird, sodass zusammenhängende Bereiche reserviert werden können. Außerdem gibt es ausgefeilte Caching-Strategien, bei der mehrere Blöcke gleichzeitig zum Schreiben alloziert werden. Dies erhöht die Schreib-Performace, jedoch steigert sich das Risiko von Datenverlusten bei Abstürzen, da das Schreiben von Daten und Metadaten entkoppelt wird (im Dateisystem ist die Datei vorhanden, aber die zugehörigen Dateninhalte sind noch nicht auf der Festplatte).

Abbildung 8 zeigt vereinfacht das grundlegende Prinzip der Extent-Datenstruktur.

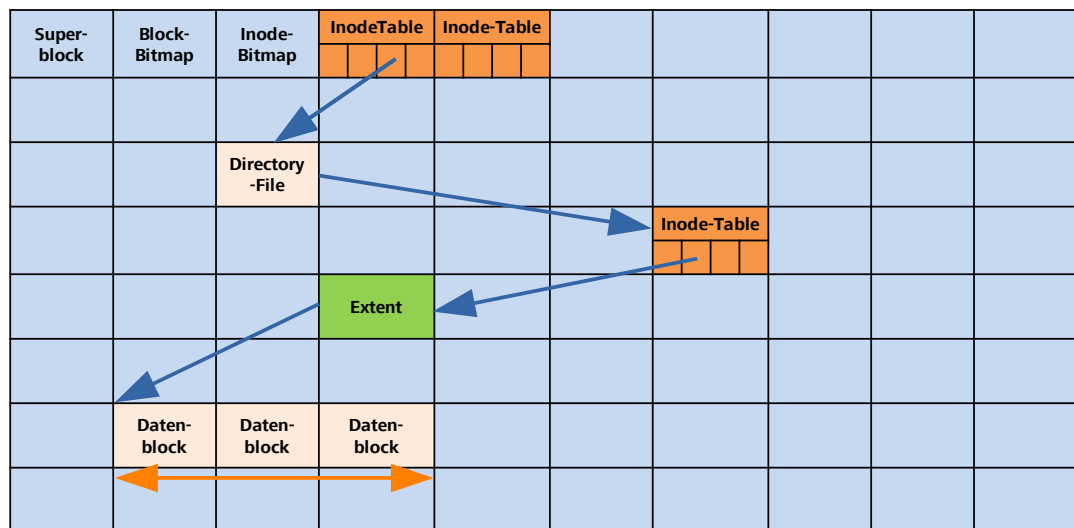


Abbildung 8: Extent-Datenstruktur (Prinzip)

5.6 B-Tree-Filesysteme

Moderne Dateisysteme wie XFS oder Btrfs (B-Tree-Filesystem) verwenden zum Speichern der Dateisystem-Informationen keine starren Blockstrukturen sondern Baum-Strukturen, wie z.B. B-Bäume (siehe Kapitel 9.4.5). Vorteil ist die größere Flexibilität, allerdings sind die Hardware-Anforderungen erheblich höher, da die komplette Baum-Struktur in den Hauptspeicher des Computers geladen werden muss, um effizient damit arbeiten zu können.

Diese Dateisysteme besitzen viele zusätzliche Funktionen, wie die Möglichkeit der Vergrößerung und Verkleinerung des Dateisystems oder der Anzahl der Inodes im laufenden Betrieb oder Tools für Backup, Restore, Images und Snapshots. Aufgrund der flachen Baum-Struktur (B- oder B+-Baum) mit hoher Verzweigungsanzahl pro Knoten haben solche Dateisysteme eine deutlich bessere Performance als ext-Dateisysteme.

Einen anderen Weg als die Journaling-Filesysteme ext3 und ext4 geht das Filesystem Btrfs: Um ein stabiles Filesystem herzustellen, wird kein Journal angelegt sondern das Prinzip „Copy-on-Write“ (CoW) angewendet. Hierbei werden Datenblöcke nach einer Änderung nicht an dieselbe Blockposition zurückgeschrieben, sondern es wird ein neuer Block dafür angelegt. Auch bei diesem Prinzip kann also in einer Fehlersituation auf die alten Daten zurückgegriffen werden, da diese bei einem Schreibvorgang (zunächst) gar nicht überschrieben werden. Damit lassen sich prinzipbedingt auch sehr einfach Snapshots (virtuelle Kopien) anlegen, da ältere Dateien auch nach Änderungen weiterhin auf der Platte vorhanden sind. Diese Dateien im Snapshot werden also nicht physikalisch kopiert sondern lediglich in einem neuen Filesystem-Baum referenziert, dessen Metadaten auf die realen Datenblöcke zeigen.

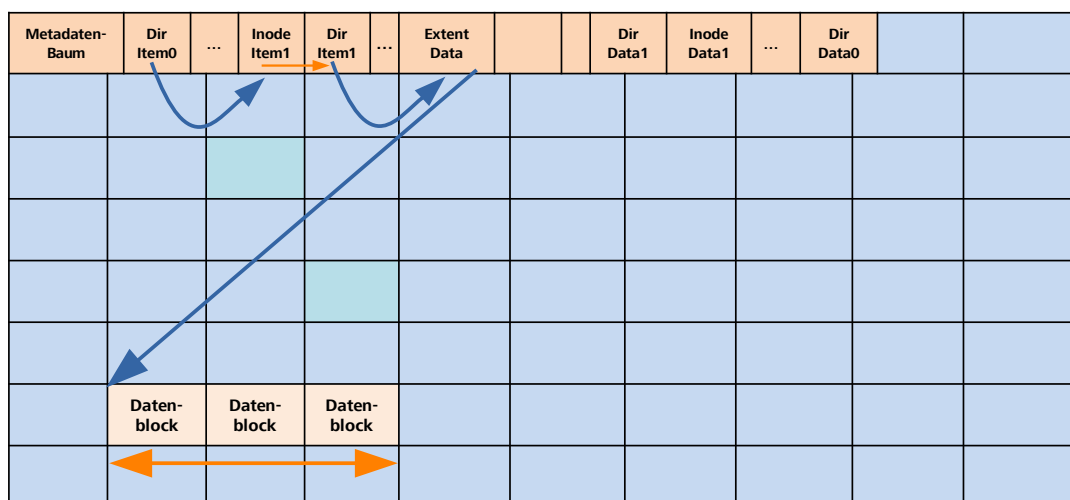


Abbildung 9: B-Tree-Filesystem (Prinzip)

6 OSI-Modell

Das OSI-Modell (Open System Interconnection) ist ein Strukturmodell zur Unterteilung der verschiedenen Aufgaben bei der Übertragung einer Dateneinheit. Das Modell beschreibt sieben Schichten (Abbildung 10), in denen eine Dateneinheit (z.B. eine Mail) beim Sender schrittweise zerlegt wird, sodass letztlich die Übertragung einzelner Bits als physikalische Signale ermöglicht wird. Der Empfänger fügt die Einzelteile in gleicher Weise wie beim Sender wieder zusammen (vom einzelnen Bit bis zur vollständigen Mail). Das Modell (bzw. seine Protokolle) beschreibt lediglich die Abläufe auf den einzelnen Schichten; die konkrete Implementierung ist unabhängig von Geräten, Herstellern, Betriebssystemen oder Programmiersprachen, sodass dadurch technisch verschiedene Rechner oder Geräte miteinander kommunizieren können. Für das OSI-Modell wurden konkrete Protokolle entwickelt, die heute in der Praxis jedoch keine Bedeutung mehr haben. Im Internet wird eine vereinfachte Variante dieses Strukturmodell verwendet.

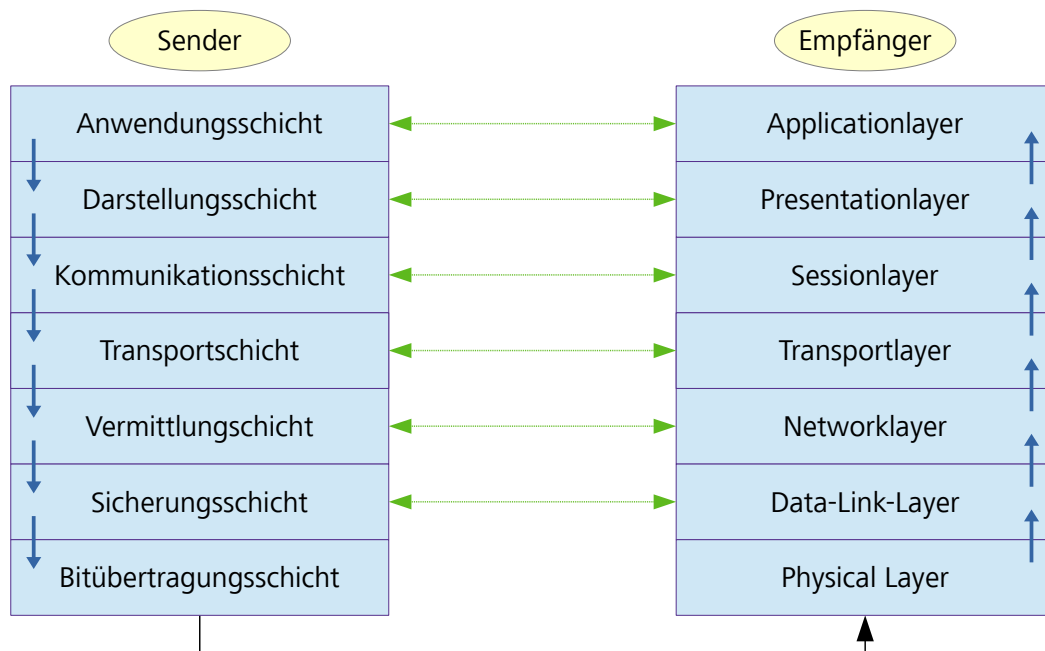


Abbildung 10: OSI-Modell

Beim Sender setzt jede Schicht fest definierte Aufgaben um (definiert in den „Protokollen“) und übergibt dann die Daten an die darunterliegende Schicht zur Weiterverarbeitung. Beim Empfänger werden die gleichen Aufgaben in umgekehrter Reihenfolge (von unten nach oben) ausgeführt. Dabei ist es wichtig, dass auf jeder Schicht die in dem jeweiligen Protokoll formulierten Regeln zur Verarbeitung der Daten genau eingehalten werden, d.h. die Transportschicht (engl.: Transportlayer) beim Empfänger verarbeitet die Daten genauso wie die Transportschicht beim Sender. Die Schichten sind im Wesentlichen gegeneinander transparent, beispielsweise muss der Transportschicht nicht bekannt sein, welche Aufgaben die darunterliegende Vermittlungsschicht hat oder welche Art von Bitübertragungstechnik verwendet wird; aus Sicht der Transportschicht des Senders findet daher eine unmittelbare Kommunikation mit der entsprechenden Schicht des Empfängers statt (waagerechte gestrichelte Linie).

In diesem Skript werden in Kapitel 7 nur die Aufgaben der Vermittlungsschicht detaillierter betrachtet; die weiteren Schichten werden im Studien-Modul „Netze“ (Netzwerksicherheit und -anwendungen, Media Systems 3. Semester) behandelt. Daher seien an dieser Stelle nur die wesentlichen Aufgaben der Schichten kurz zusammengefasst.

Aufgaben der Schichten:

1. Bitübertragungsschicht

- Umwandlung der vorliegenden Bits in physikalische Signale (elektrisch, elektromagnetisch, optisch)
- Definition mechanischer Schnittstellen und Kabeltypen

2. Sicherungsschicht
 - Zerlegen von Datenblöcken (Rahmen, engl. Frames) in einzelne Bits
 - Fehlerkontrolle
 - Flusssteuerung
 - Kanaluordnung bei Mehrfachzugriffsnetzen (z.B. Ethernet)
 - sofern erforderlich: Verwaltung von Verbindungen
 - technische Adressierung (z.B. MAC-Adressen)
3. Vermittlungsschicht
 - Adressierung der Endgeräte (z.B. IP-Adressen)
 - Übertragung von Datenpaketen zum Endgerät, auch über mehrere Netzwerkknoten (Router) hinweg
 - Wegfindung im Netz (Routing)
4. Transportschicht
 - logische Ende-zu-Ende-Verbindung
 - Verbesserung der Dienstqualität
 - Fehlerkontrolle
 - Verbindungskontrolle
 - Zuweisung der Datenpakete zu den jeweiligen Anwendungen
 - zusammengefasst: Bereitstellung eines zuverlässigen Datentransports in einem unzuverlässigen Netz
5. Kommunikationsschicht
 - Steuerung der Kommunikation zwischen Anwendungen bzw. Prozessen (wird im Internet-Modell nicht verwendet)
6. Darstellungsschicht
 - Umwandlung der Datendarstellung (Format, Kodierung) in ein betriebssystemunabhängiges Netzwerkformat (wird im Internet-Modell nicht verwendet)
7. Anwendungsschicht
 - Definition der der Netzerkanwendungen (z.B. Formate und Art der Übertragung von Mail, Dateien, Webseiten)

7 Vermittlungsschicht

In der Veranstaltung „Netzwerk-Grundlagen“ (Modul Informatik B, Media Systems 2. Semester) werden zunächst nur die Schicht 3 im OSI-Modell (die Vermittlungsschicht) mit dem IP-Protokoll (IPv4 und IPv6) sowie Routing-Protokolle und Routing-Algorithmen betrachtet. Die weiteren OSI-Schichten und der Bereich Netzwerksicherheit sind Teil des Moduls „Netze“ im 3. Semester.

7.1 Grundlagen

Die wesentliche Aufgabe der Vermittlungsschicht (Network Layer, OSI-Schicht 3) ist der Transport von Datenpaketen (Datagrammen) vom Quell- zum Zielrechner. Im Unterschied zur Sicherungsschicht (Schicht 2), die für einen zuverlässigen Rahmentransport innerhalb eines einzigen physikalischen Netzes sorgt, muss die Vermittlungsschicht Daten über Netzwerkgrenzen hinweg übertragen; sie benötigt dafür keine Kenntnisse über die Zugriffsmechanismen der unterschiedlichen Netzwerktechniken, denn dies fällt in den Zuständigkeitsbereich der jeweiligen Sicherungsschicht. Weitere Aufgaben: Kenntnisse über das Kommunikationsnetz, Wegefindung (Routing), Ausgleich bei Überlastsituationen (fällt heute aber eher in die Zuständigkeit der Transportschicht), Vermittlung von Paketen zwischen unterschiedlichen Netztopologien, Bereitstellung von Diensten für die Transportschicht.

Erst mithilfe der Vermittlungsschicht können einzelne Subnetze (lokale Netze und Weitverkehrsstrecken) zu einem logischen Gesamtnetz gekoppelt werden. Heute ist das (einzige in der Praxis verbliebene) Protokoll das Internet Protocol IP, das im Wesentlichen für eine eindeutige Adressierung der Datenpakete sorgt. Die Vermittlungsschicht stellt den höheren Schichten eine transparente Verbindung zur Verfügung, sodass sich diese Schichten nicht um die Übertragung der Datenpakete zu kümmern brauchen. Aus Sicht einer darüberliegenden Transportschicht ist daher direkt nicht feststellbar, ob sich der Zielrechner innerhalb desselben Subnetzes (in demselben Raum) befindet oder viele tausend Kilometer entfernt auf einem anderen Kontinent.

Auf dem Weg zu einem entfernten Zielrechner stehen oftmals verschiedene Alternativ-Strecken zur Auswahl, sodass eine Wegewahlentscheidung (Routing) getroffen werden muss. Auch diese Aufgabe wird nur von der Vermittlungsschicht übernommen. Die für die Wegewahl zur Verfügung stehenden Verfahren werden in den Kapiteln 7.5 und 8 genauer betrachtet. Abbildung 11 zeigt anschaulich die verschiedenen Router, die zwischen Quellrechner A und Zielrechner B liegen: Jeder Router packt ein Paket bis zur Vermittlungsschicht aus, liest die Ziel-IP-Adresse und entscheidet anhand seiner Routing-Tabelle, an welche Ausgangsleitung bzw. an welchen Nachbar-Router das Paket weitergeleitet werden muss.

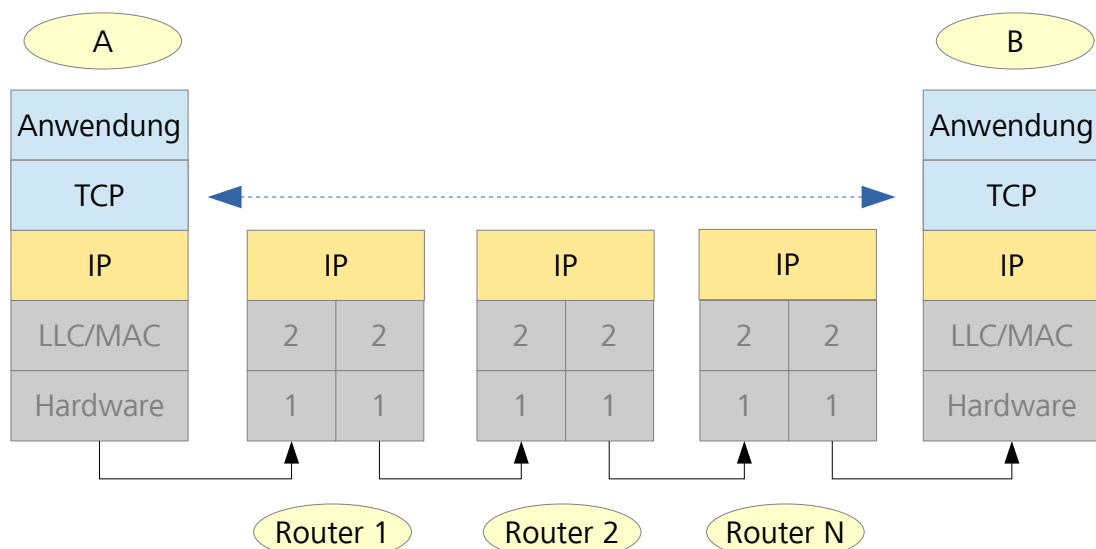


Abbildung 11: Internet-Modell

Prinzipiell kann eine Vermittlungsschicht entweder einen verbindungslosen oder einen verbindungsorientierten Dienst anbieten. Die Grundüberlegung beim (im Internet verwendeten) verbindungslosen Dienst ist, dass die Vermittlungsschicht nur für die reine Übertragung von Paketen zuständig ist, wogegen weitere Aufgaben wie Reihenfolge der Pakete, Flusssteuerung, Fehlerüberwachung von höheren Schichten übernommen werden; die hierfür verantwortlichen Schichten (in Abbildung 11 das Transmission Control Protocol TCP) sind erst auf dem lokalen Zielrechner realisiert, sodass sich Netzwerkkomponenten auf Schicht 3 im Wesentlichen nur um Adressierung und Routing zu kümmern brauchen. Beim verbindungslosen Dienst (Datagramm-Dienst) ist jedes Paket unabhängig vom vorhergehenden und nachfolgenden Paket, sodass es die gesamte Zieladresse enthalten muss und eine Routing-Entscheidung für jedes Paket einzeln getroffen werden muss. Jedes Paket *kann* je nach aktueller Wegewahl daher einen anderen Weg zum Ziel nehmen, mit der Folge, dass die Pakete nicht in der ursprünglichen Reihenfolge beim Empfänger ankommen müssen (dieses Verfahren wird im Internet angewendet). Der wesentliche Grund für die Verwendung eines verbindungslosen Verfahrens ist seine Robustheit: Da jedes Paket einzeln für sich durch das Netzwerk geleitet wird, stören Leitungs- oder Routerausfälle oder Überlastsituationen den Datenfluss nur wenig (sofern Alternativstrecken zur Verfügung stehen).

Im Gegensatz dazu soll bei einem verbindungsorientierten Dienst auch eine Vermittlungsschicht eine zuverlässige Datenübertragung gewährleisten. Hierzu baut die Vermittlungsschicht des Senders (Rechner oder Router) vor der Datenübertragung eine feste Verbindung mit dem nächsten Ziel auf, sodass stückweise eine vollständige Verbindung vom Quellrechner zum Zielrechner entsteht. Vorteile dieses (im Internet *nicht* verwendeten) Verfahrens sind: Aushandlung spezieller Parameter, Einhaltung der Reihenfolge, effektive Flusssteuerung und Ressourcen-Planung (Vermeidung von Überlastsituationen) sowie Fehlerkontrolle bereits auf einer unteren Schicht, einfachere Adressierung (und damit schnelleres Routing) mittels einer Verbindungsnummer. Nachteil: bereits auf unterer Ebene muss ein hoher Aufwand betrieben werden (viel Speicherplatz für die Verbindungstabellen, Zeitaufwand für die Aufbauphase, Abbruch von Verbindungen auch bei nur kurzzeitigem Ausfall eines Knotens).

7.2 IPv4 im Internet

7.2.1 IPv4-Header

Damit ein IP-Paket von der Vermittlungsschicht verarbeitet bzw. weitergeleitet werden kann, müssen bestimmte Informationen in einem Paket enthalten sein, wie z.B. die Zieladresse. Diese Informationen werden in einer strukturierten (und standardisierten) Form dem eigentlichen Paketinhalt (Payload) in Form eines sog. Headers vorangestellt. Der Header eines IP-Paketes enthält diese Informationen in Feldern fest definierter Länge. Der gesamte Header umfasst 20 Byte (fünf 32-Bit-Wörter) und kann um Optionen ergänzt werden.

IPv4-Header

0					31							
Version		IHL		ToS/DiffServ			Total Length					
Identification						Flags		Fragmentation Offset				
TTL			Protocol			Header Checksum						
Source Address												
Destination Address												
Optionen ...												

Header-Felder:

- Version: IPv4 (4 Bit)
- IHL (4): IP Header Length, Länge des (variablen) Headers als Anzahl der 32-Bit-Wörter, Standard (ohne Optionen) 20 Byte (fünf 32-Bit-Wörter),
- ToS (8): Type of Service (alte Bezeichnung) oder Differentiated Services-Feld zur Definition von Dienstklassen (Expedited und Assured Forwarding) sowie einige Bits für ECN (Explicit Congestion Notification, Stauvermeidung)
- Total Length (16): Gesamtlänge einschließlich Header und Payload, max. 65535 Byte
- Identification (16): Zugehörigkeit von Fragmenten zu einem IP-Paket
- Flags (3 Bit): mittleres Bit: Don't Fragment (keine Fragmentierung erlaubt), rechtes Bit: More Fragment (ist bei allen Fragmenten eines Paketes außer dem letzten gesetzt, um festzustellen, wann alle Fragmente

- eingetroffen sind)
- Fragmentation Offset (13): Position eines Fragments im Paket
- TTL (8): Time to Live
- Protocol (8): Protokoll im Payload (Transportprotokolle TCP (6), UDP (17) oder Art des Routingprotokolls oder ICMP (1))
- Header Checksum (16): Prüfsumme *nur* für den Header
- Adressen: 32-Bit IPv4-Quell- und Zieladresse

Der Header beginnt mit einem 4-Bit Versionsfeld, mit dem eine Unterscheidung zwischen Paketen im IPv4-Format und im IPv6-Format möglich ist. Da die Länge des Headers variabel ist (nur die ersten fünf 32-Bit-Wörter sind fest definierte Pflichtfelder), folgt bei IPv4 das Feld „IP Header Length“ (IHL), das die Länge des gesamten Headers einschließlich Optionen als Anzahl von 32-Bit-Wörtern angibt (Optionen müssen daher immer ein Vielfaches von 4 Byte umfassen und können wegen der maximal 4 Bit in diesem Feld nur 40 Byte groß sein (maximaler Wert: 15 32-Bit-Wörter, entspricht 60 Byte, davon 20 Byte Standard-Header). Mit dem Feld „Type of Service ToS“ sollten ursprünglich Dienstklassen unterschieden werden, um bestimmte Dienste schneller als andere weiterzuvermitteln; dieses Feld fand in der Praxis jedoch nie weite Verbreitung und wurde durch „differenzierte Dienste“ (DiffServ) abgelöst. Die ersten 6 Bit werden für die Dienstklassen „Expedited Forwarding“ und „Assured Forwarding“ verwendet, die verbleibenden 2 Bit für Stauvermeidungsstrategien (Explicit Congestion Notification ECN); dazu mehr im Modul Netze (3. Semester Media Systems). Das Feld „Total Length“ (16 Bit) gibt die Gesamtlänge des Datagrams einschließlich Header und Nutzdaten an; ein IP-Paket kann daher maximal eine Größe von 2^{16} Bit (entspricht 64 kB) haben.

Das zweite 32-Bit-Wort enthält alle Informationen, die für die sog. Fragmentierung eines Paketes erforderlich sind. In dem Fall, dass ein IP-Paket nicht als Ganzes an die darunterliegende Sicherungsschicht weitergegeben werden kann, da diese nur kleinere Dateneinheiten verarbeiten kann (im Ethernet beträgt die Maximum Transmission Unit MTU nur 1500 Byte), oder eine Netzwerktechnik auf dem Weg zum Ziel nur kleinere Datenpakete übertragen kann, muss das IP-Paket an diese Beschränkung angepasst werden. Hierzu wird es in kleinere Teilstücke (Fragmente) zerlegt. Jedes Fragment wird als eigenständiges Paket über das Netzwerk übertragen, benötigt also auch einen vollständigen IP-Header, um zum Ziel zu gelangen. Damit die Vermittlungsschicht beim Zielrechner alle Fragmente wieder zu dem Gesamt-Paket zusammensetzen kann, wird eine Identifikationsnummer für das Gesamt-Paket benötigt (jedes Fragment, das Teil eines größeren Gesamt-Paketes ist, trägt dieselbe ID-Nummer in seinem Header). Es gelten aber auch alle genannten Einschränkungen bei der Übertragung von IP-Paketen, wie Paketverluste oder Paketreihenfolge. Daher muss zusätzlich die Position eines Fragments im Gesamt-Paket (Fragmentation-Offset) angegeben werden, da die Reihenfolge beim Abschicken der Fragmente nicht notwendig der Reihenfolge beim Empfänger entsprechen muss. Das „Don't Fragment Bit“ zeigt an, dass ein Paket nicht fragmentiert werden darf, wenn z.B. der Empfänger die Fragmentierung gar nicht unterstützt oder mithilfe des MTU Path Discovery herausgefunden werden soll, welche maximalen Datenmengen je Paket in *entfernten* Teilnetzen übertragen werden können (dazu mehr im Modul Netze, Media Systems 3.Semester).

Das dritte 32-Bit-Wort beginnt mit einem Zähler (Time to Live TTL) für die Lebensdauer eines Paketes. Dieser Zähler wird vom Quellrechner auf einen bestimmten Wert gesetzt (maximal 255) und dann bei jedem Router um 1 verringert; derjenige Router, der die TTL auf Null setzt, verwirft das Paket und schickt eine Fehlermeldung. Damit können Pakete aus dem Netz entfernt werden, die z.B. aufgrund fehlerhafter Routingtabellen das Ziel gar nicht erreichen können und somit „ewig im Netz umherirren“ würden. Mit dem Protocol-Feld wird es einer Vermittlungsschicht ermöglicht zu entscheiden, an welchen Transportprozess das Paket weitergeleitet werden soll (TCP oder UDP), ob es von einem Routing-Protokoll zu verarbeiten ist oder ob es sich um ein Steuerprotokoll (siehe Kapitel 7.4.2) handelt. Die Daten des Headers sind für die Weiterverarbeitung sehr wichtig, weshalb eine fehlerfreie Übertragung des Headers essentiell ist. Die Header-Checksum wird daher verwendet, um Fehler bei der Übertragung des IP-Headers festzustellen. Diese Prüfsumme wird beim Sender aus der Addition aller 16-Bit-Halbwörter des Header gebildet (wobei der Wert der Prüfsumme selbst auf Null gesetzt wird) und anschließend wird das binäre Komplement gebildet. Der Empfänger (auch ein Zwischenziel wie ein Router auf dem Weg) berechnet ebenfalls diesen Wert und vergleicht sein Ergebnis mit dem Wert aus dem übermittelten Checksum-Feld; bei Ungleichheit wird das Paket verworfen. Wichtig: Die Inhaltsdaten (Payload) werden mit dieser Checksum *nicht* überprüft.

Die letzten beiden 32-Bit-Wörter enthalten die Quell- und die Zieladresse.

7.2.2 IPv4-Adressen

Jeder Rechner oder jedes Gerät (genauer: jede Netzwerkschnittstelle eines Gerätes) benötigt eine IP-Adresse, um über ein Netzwerk erreichbar zu sein. IPv4-Adressen bestehen aus 32 Bit; sie werden meistens in dezimaler Form geschrieben, wobei die einzelnen Bytes in Dezimalform durch einen Punkt getrennt werden (z.B. 172.16.45.209). IP-Adressen bestehen aus einem variablen Netzwerkanteil und einem Rechneranteil. Der Netzwerkanteil wird durch die linken Bit der Adresse dargestellt, der Rechneranteil durch die rechten Bit. Der Netzwerkanteil kann als

„Vorwahl“ verstanden werden, unter der ein Unternehmensnetzwerk im Internet erreichbar ist. Der Rechneranteil der Adresse stellt den „Anschluss“ oder „Teilnehmer“ innerhalb dieses Unternehmensnetzwerkes dar.

Die Größe des Adressbereiches wird über die Anzahl der Bits des Netzwerkanteils definiert: Soll eine Adresse z.B. aus 24 Bit Netzwerkanteil und 8 Bit Rechneranteil bestehen, so wird zusätzlich zur Adresse die Anzahl der Netzbits (Präfixlänge) angegeben: 172.16.45.0 /24. Das Netzwerk hat damit das Präfix 172.16.45.0. Es stehen 8 Bit zur Adressierung der Rechner innerhalb dieses Netzwerks zur Verfügung ($2^8 = 256$), d.h. das rechte Byte kann (dezimal) die Werte zwischen 0 und 255 annehmen.

Da sich die Präfixlänge nicht allein anhand der IP-Adresse erkennen lässt, muss ein Router im Internet zusätzlich die Länge des Netzwerkanteils kennen, um mittels seiner Routing-Protokolle und Routing-Algorithmen den Weg zum Zielnetzwerk zu finden (siehe Kapitel 7.5 und 8). In den Anfangszeiten des Internets war dies anders: Es gab nur fünf fest definierte Klassen von Adressen („classful addressing“). Die Klasse A legte die Länge des Präfix auf 8 Bit fest, sodass innerhalb eines Klasse-A-Netzes 24 Bit für den Rechneranteil zur Verfügung standen. Definiert wurde eine Klasse-A-Adresse mittels des linken Bit, das Null sein musste. Die Klasse B hatte 16 Bit als Präfix-Anteil (d.h. auch 16 Bit für den Rechneranteil innerhalb eines Klasse-B-Netzes) und wurde über die linke Bitfolge Eins-Null definiert. Ein Klasse-C-Netz bestand aus 24 Netzbits (entsprechend nur 8 Bit für den Rechneranteil) und wurde mittels der linken Bitfolge Eins-Eins-Null definiert. Diese drei Klassen wurden zur Adressierung von Einzeladressen verwendet (Unicast), während die Klasse D für Multicast-Anwendungen vorgesehen war, bei denen mehrere Rechner unter einer Gruppenadresse zusammengefasst werden können. Die Klasse E sollte für zukünftige Anwendungen reserviert bleiben. Aufgrund dieser starren Festlegung lässt sich das Präfix direkt aus der IP-Adresse ablesen; die folgende Tabelle stellt die fünf Klassen und ihre Adressbereiche dar.

	Byte 1		Byte 2	Byte 3	Byte 4	
Klasse A	0	Netz	Rechner			0.0.0.0 bis 127.255.255.255
Klasse B	10	Netz		Rechner		128.0.0.0 bis 191.255.255.255
Klasse C	110	Netz			Rechner	192.0.0.0 bis 223.255.255.255
Klasse D	1110	Multicast-Adressen				224.0.0.0 bis 239.255.255.255
Klasse E	1111	reserviert				240.0.0.0 bis 255.255.255.255

Es gibt im gesamten Internet damit nur 126 (2^7) Klasse-A-Netze (eigentlich 128, aber Adressen beginnend mit 0 und 127 sind für spezielle Zwecke reserviert), 16384 Klasse-B-Netze (2^{14}) und ca. 2 Millionen Klasse-C-Netze (2^{21}). Das größte Problem dieser starren Klasseneinteilung war die Verschwendung von IP-Adressen: Ein Klasse-A-Netz enthält ca. 16 Millionen Rechneradressen, Klasse B 65536 und Klasse C nur 256. Während ein Klasse-C-Netz bereits für mittelständische Unternehmen zu klein ist, ist ein Klasse-B-Netz viel zu groß, d.h. es wird nur ein kleiner Teil des Adressraumes aktiv genutzt. Da ein Präfix immer ortsgebunden ist, besteht jedoch keine Möglichkeit, Klasse-B-Netze zu teilen.

Es war daher bereits Anfang der 1990er Jahre dringend erforderlich, eine flexiblere Einteilung der Adressbereiche vorzunehmen. Mit dem Verfahren „Classless InterDomain Routing“ CIDR wird (wie oben bereits beschrieben) die Anzahl der Netzbits festgelegt, sodass damit die Größe eines Netzwerks in Zweierpotenzen variabel zugeschnitten werden kann. Diese Anzahl der Netzbits muss jedoch als zusätzliche Information zur Netzadresse angegeben werden (dies war bei der starren Klasseneinteilung nicht erforderlich) und die Prozesse zur Berechnung der Wegefindung (Routing) in den Routern werden deutlich aufwändiger.

Das Internet stellt im Prinzip eine freie Ansammlung von Netzen, Netzwerk-Technologien, Anwendungen und Protokollen dar, mit dem Internet-Protokoll IP als gemeinsames Dach. Es gibt daher kaum starre Vorgaben oder zwingend vorgeschriebene Standardisierungen oder zentrale Regulierungen. Lediglich bei den Adressen muss sichergestellt sein, dass jede IP-Adresse eindeutig ist. Für die Vergabe der IP-Adressen ist daher eine zentrale Stelle, die IANA (Internet Assigned Numbers Authority, heute als Aufgabenbereich der Internet Corporation for Assigned Names and Numbers ICANN) zuständig. Die IANA delegiert die Adressvergabe an sog. Regional Internet Registries RIR (in Europa: Reseaux IP Europeen RIPE), die wiederum große Adressblöcke an die Internet Service Provider ISPs weitergeben. Die ISPs teilen ihre Adressblöcke (sog. Provider Aggregatable PA) in geeigneter Größe den Kunden zu.

Vorteil dieser Vorgehensweise ist, dass die Provider über große zusammenhängende Adressbereiche verfügen. Da die heutige Netzwerk-Grundstruktur des Internets im Wesentlichen aus Provider-Netzen gebildet wird, wird das globale Routing dadurch vereinfacht; in den Routingtabellen der Providernetze brauchen nicht sämtliche einzelnen Netzadressen aufgelistet zu werden, sondern nur die zusammengefassten Adressbereiche der jeweils anderen Provider. Lediglich innerhalb des jeweiligen ISP-Netzes müssen die Einzeladressen in den Routingtabellen stehen.

Diese Strategie der Adressvergabe ist jedoch für solche Unternehmen oder Organisationen problematisch, die an

mehrere Provider-Netze angeschlossen sind (sog. „Multi-homed Networks“); in diesen Fällen gibt es drei Optionen:

- Es wird beim RIR eine Provider Independent (PI) Adresse beantragt (Vorteil: eine Organisation verfügt über ihre eigenen, unveränderlichen IP-Adressen, Nachteile: Adressbereich muss global geroutet werden, sehr kleine PI-Blöcke (z.B. kleiner als /20) werden u.U. von globalen Routern ausgefiltert)
- ein eigener PA-Block, der heute jedoch ausschließlich für ISPs verfügbar ist
- Verwendung eines Blocks aus *einem* ISP-PA-Bereich und Bekanntmachung als PI-Adresse

In Fällen, in denen ein Netzwerk gar keinen festen Zugang zum weltweiten Internet haben soll oder durch einen NAT-Router (Network Address Translation) verdeckt wird, werden keine offiziellen IP-Adressen benötigt; man könnte prinzipiell jede beliebige Adresse in solchen Netzwerken verwenden. Um jedoch Probleme mit dem Routing zu vermeiden falls es doch zu einem Kontakt mit dem Internet kommen sollte, ist es empfehlenswert, spezielle für diesen Zweck vorgesehene Adressbereiche zu verwenden. Die folgenden Adressbereiche werden im Internet nicht geroutet und können daher beliebig verwendet werden:

10.0.0.0 /8	Ein Klasse-A-Netz: 10.0.0.0 bis 10.255.255.255
172.16.0.0 /12	16 Klasse-B-Netze: 172.16.0.0 bis 172.31.255.255
192.168.0.0 /16	256 Klasse-C-Netze: 192.168.0.0 bis 192.168.255.255

7.2.3 Subnetzmasken

Einem Unternehmen oder einer Hochschule wird (anders als im privaten Bereich) vom Provider ein Bereich von IPv4-Adressen in der benötigten Größe zugewiesen. Dieser Adressbereich kann innerhalb dieses Netzwerkes beliebig unterteilt werden. Eine Aufteilung in solche Subnetze kann aus verschiedenen Gründen erforderlich sein: Beispielsweise sollen aus Sicherheitsgründen Abteilungsnetze gegeneinander abgegrenzt werden, aus technischen Gründen müssen verschiedene Netzwerktechnologien eingesetzt werden, die Verwaltung der Netze soll dezentral organisiert werden oder es müssen größere Entfernungen überbrückt werden.

Der insgesamt zur Verfügung stehende IP-Adressbereich muss daher in mehrere kleinere Teile zerlegt werden. Die Größe dieser Teile ist variabel und kann mithilfe der so genannten **Subnetz-Maske** festgelegt werden. Per Definition symbolisiert die Anzahl der linken Eins-Bits in dieser Maske den Netzwerkanteil und die Anzahl der rechten Null-Bits den Rechneranteil. Die Darstellung der Maske kann entweder binär erfolgen (als Anzahl der Eins-Bits) oder dezimal, wobei wie bei der IP-Adresse jeweils ein Byte als Dezimalzahl dargestellt wird.

Beispiel: Die linken 21 Bit der Maske sind binäre Einsen, die rechten 11 Bit Nullen, d.h. in diesem Netzwerk ist ein Block mit $2^{11} = 2048$ Adressen zusammengefasst (die jeweils fett gesetzten Bits entsprechen dem Rechneranteil). Die Schreibweise der Maske ist entweder 255.255.248.0 (dezimal) oder /21 (binär).

IP-Adresse dezimal	192	168	34	49	
IP-Adresse binär	1100 0000	1010 1000	0010 0010	0011 0001	/21
Maske dezimal	255	255	248	0	
Maske binär	1111 1111	1111 1111	1111 1000	0000 0000	/21

Wird die IP-Adresse mit einer logischen UND-Operation mit der Maske verknüpft, so erhält man die Netzadresse:

IP-Adresse binär	1100 0000	1010 1000	0010 0010	0011 0001
& Maske binär	1111 1111	1111 1111	1111 1000	0000 0000
= Netzadresse	1100 0000	1010 1000	0010 0000	0000 0000
Netzadresse dezimal	192	168	32	0

Die Netzadresse ist immer die erste (kleinste) Adresse aus einem Bereich, während die Broadcast-Adresse, unter der alle Rechner angesprochen werden können, die letzte (höchste) Adresse ist. In diesem Beispiel definieren wegen der Maske /21 die rechten 11 Bit die Rechneradresse ($32-11=21$), also ergibt sich für die Broadcast-Adresse:

Netzadresse binär	1100 0000	1010 1000	0010 0000	0000 0000
Broadcast-Adresse binär	1100 0000	1010 1000	0010 0111	1111 1111
Broadcast-Adresse dezimal	192	168	39	255

Bevor ein Rechner ein IP-Paket abschicken kann, muss er feststellen, ob die IP-Adresse des Zielrechners innerhalb oder außerhalb des eigenen Subnetzbereiches liegt. Hierzu berechnet er den Netzwerkanteil seiner eigenen IP-Adresse und den Netzwerkanteil der entfernten Adresse jeweils mittels seiner *eigenen* Subnetzmaske. Mithilfe einer logischen UND-Verknüpfung der beiden Adressen mit der Maske wird diese Berechnung ausgeführt. Ist das Ergebnis in beiden Fällen gleich, so liegt der Zielrechner innerhalb des eigenen Subnetzes, ist es verschieden, so liegt der Zielrechner außerhalb.

Beispiel 1: Rechner A habe die IP-Adresse 192.168.34.49 /27 (Maske dezimal: 255.255.255.224), Rechner B die Adresse 192.168.34.35. Beide Rechner liegen offensichtlich im Netz 192.168.34.32 (dies ergibt sich aus der UND-Verknüpfung in der Tabelle), sodass die UND-Verknüpfung in beiden Fällen dasselbe Ergebnis liefert; Rechner A und B liegen in demselben Subnetz.

	192	168	34	49
IP-Adresse A	1100 0000	1010 1000	0010 0010	0011 0001
& Maske A	1111 1111	1111 1111	1111 1111	1110 0000
Netzanteil	1100 0000	1010 1000	0010 0010	0010 0000

	192	168	34	35
IP-Adresse B	1100 0000	1010 1000	0010 0010	0010 0011
& Maske A	1111 1111	1111 1111	1111 1111	1110 0000
Netzanteil	1100 0000	1010 1000	0010 0010	0010 0000

Beispiel 2: Rechner A habe die IP-Adresse 192.168.34.49 /27 (Maske dezimal: 255.255.255.224), Rechner B die Adresse 192.168.34.210. Rechner A befindet sich im Netz 192.168.34.32 /27, Rechner B aber in einem anderen (welches das ist, lässt sich aus dieser Berechnung jedoch nicht erkennen, ist aber hier auch nicht von Bedeutung), sodass die UND-Verknüpfung in diesen Fällen ein unterschiedliches Ergebnis liefert; Rechner A und B liegen *nicht* in demselben Subnetz.

	192	168	34	49
IP-Adresse A	1100 0000	1010 1000	0010 0010	0011 0001
& Maske A	1111 1111	1111 1111	1111 1111	1110 0000
Netzanteil	1100 0000	1010 1000	0010 0010	0010 0000

	192	168	34	210
IP-Adresse B	1100 0000	1010 1000	0010 0010	1101 0010
& Maske A	1111 1111	1111 1111	1111 1111	1110 0000
Netzanteil	1100 0000	1010 1000	0010 0010	1100 0000

Abbildung 12 zeigt anschaulich die Notwendigkeit der Feststellung, ob sich der Zielrechner eines Paketes innerhalb oder außerhalb des eigenen Subnetzes befindet.

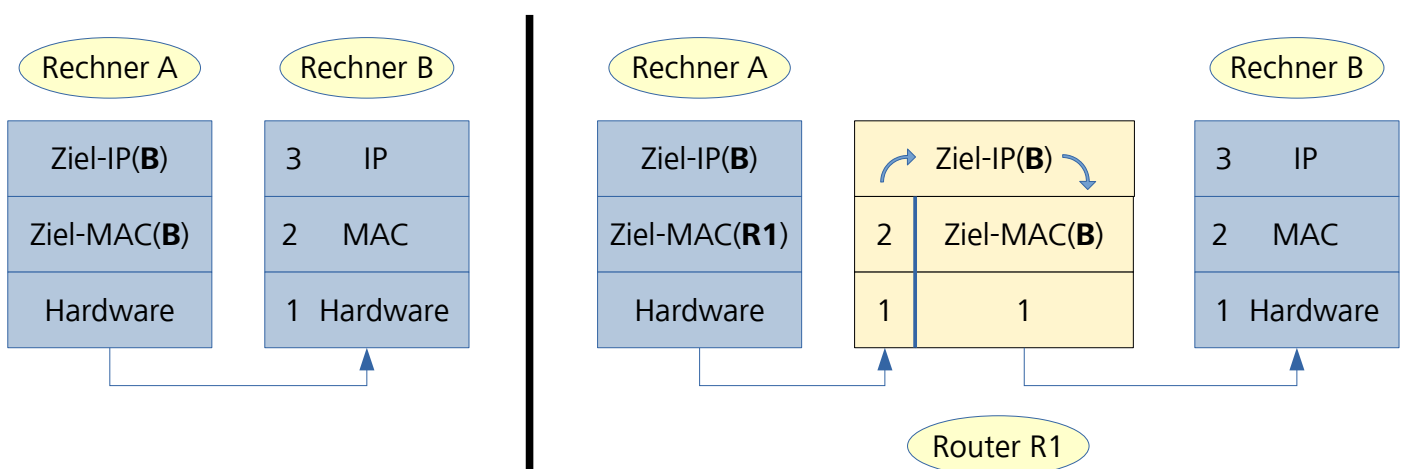


Abbildung 12:

Im linken Teil der Abbildung sind Quell-Rechner A und Ziel-Rechner B in demselben Subnetz miteinander verbunden: Rechner A schickt ein IP-Paket mit der **IP**-Zieladresse von Rechner B ab; auf der Sicherungsschicht (MAC-Teilschicht im Ethernet) muss die **MAC**-Zieladresse (Hardware-Adresse) von Rechner B eingesetzt werden. Das Paket (genauer: der Ethernet-Rahmen) gelangt i.d.R. zu einem Switch (hier nicht abgebildet), der anhand der MAC-Zieladresse den Rahmen an die Anschlussleitung von Rechner B weiterleitet.

Im rechten Teil der Abbildung befindet sich Zielrechner B nicht in demselben Subnetz, sodass Rechner A das Paket an seinen lokalen Router R1 weitergeben muss, da nur dieser den Weg zum Zielrechner anhand seiner Routing-tabelle kennt. Die IP-Zieladresse ist in diesem Fall natürlich wie zuvor die IP-Adresse von Rechner B, aber die Ziel-**MAC**-Adresse ist diejenige des Routers R1. Rechner A muss daher die IP-Adresse seines Routers (mithilfe der IP-Konfiguration „Gateway-Adresse“) kennen. Hinweis: Die Zuordnung zwischen IP- und MAC-Adressen erfolgt automatisch mittels des ARP-Protokolls (siehe Kapitel 7.4.1).

7.2.4 Beispiel für IPv4-Adressen und Subnetzmasken

Das LAN in Abbildung 13 ist über den Router R aus dem Internet mit der Adresse 137.18.3.64 erreichbar. Die Größe des Netzes wird in diesem Beispiel mittels der Maske 255.255.255.224 (oder binär /27) festgelegt, es werden damit 27 Bit als Länge der Netzadresse definiert. Dadurch ist auch die maximale Anzahl der in diesem Netz verfügbaren IP-Adressen (Länge 32 Bit) definiert: Es stehen 5 Bit ($32-27=5$) zur Verfügung, also $2^5=32$ Adressen. Der Adressbereich beginnt mit 137.18.3.64 und endet bei 137.18.3.95. Dieser gesamte Bereich kann mithilfe von Subnetzmasken innerhalb des LANs weiter unterteilt werden.

Bei der Festlegung der Größe der einzelnen Subnetze ist zu beachten, dass jedes Subnetz eine eigene Netzadresse (die erste Adresse) benötigt sowie eine eigene Broadcast-Adresse (die letzte Adresse aus dem Bereich). Für das Subnetz 1 sind daher fünf Adressen erforderlich: Adressen für die Rechner A und B, das Router-Interface R1, die Netzadresse und die Broadcast-Adresse. Ein Subnetz mit 4 Adressen reicht daher nicht aus, sodass ein Subnetz mit 8 Adressen gewählt werden muss: $8=2^3$ also 3 Bit für den Host-Anteil und $32-3=29$ Bit für den Netzanteil (Maske 255.255.255.248 oder binär /29). Subnetz 1 erhält die Netzadresse 137.18.3.64/29, die Broadcast-Adresse ist die letzte aus diesem Bereich 137.18.3.71, die verbleibenden sechs Adressen von 137.18.3.65 bis 137.18.3.70 können beliebig auf A, B und R1 verteilt werden. Ein Hinweis: Das Subnetz 1 hat dieselbe Netzwerkadresse wie das gesamte Netzwerk, anhand der Maske kann jedoch unterschieden werden, welche Adresse jeweils gemeint ist. Das Subnetz 2 kann die nächstfolgende Netzadresse 137.18.3.72 erhalten, es wird auch hier ein Netz mit 8 Adressen benötigt, somit Maske /29 und Broadcast-Adresse 137.18.3.79. Das Subnetz 3 benötigt nur zwei freie Adressen, sodass ein Netz mit insgesamt vier Adressen (2 Bit Host-Anteil) ausreicht: Netzadresse 137.18.3.80, Maske 255.255.255.252 (oder binär /30), Broadcast-Adresse 137.18.3.83, die beiden verbleibenden Adressen stehen für R3 und E zur Verfügung.

Das Subnetz 4 benötigt insgesamt sieben Adressen, also Maske /29. Aber Achtung: Die nach Subnetz 3 folgende Netzadresse 137.18.3.84 kann dafür nicht verwendet werden!

Man betrachte die Zahl 84 binär: 1010100. Nur die beiden rechten Bit sind Null und stehen daher als „freie Bit“ zur Verfügung, der maximale Adressbereich darf daher nur 2 Bit umfassen, also mindestens $32-2=30$ Bit für die Netzmaske. Die für Subnetz 4 erforderliche Maske /29 hat aber 3 Host-Bits, womit die letzten drei Bit der Adresse als Rechner-Adressen definiert wären. Damit wäre aber der Bereich (binär) von 1010**000** (dezimal 80) bis 1010**111** (dezimal 87) für dieses Subnetz festgelegt, was sich mit dem Bereich von Subnetz 3 überschneidet.

Für Subnetz 4 muss daher eine Netzadresse gefunden werden, die mindestens drei freie Host-Bits hat: 137.18.3.88 wäre möglich. Adressen für Subnetz 4: Netz 137.18.3.88, Broadcast-Adresse 137.18.3.95, der Bereich von 89 bis 94 kann für die Rechner-Adressierung verwendet werden.

Subnetz 5 kann natürlich nicht die Netzadresse 137.18.3.96 erhalten, da diese Adresse bereits außerhalb des gesamten Bereichs liegt. Es ist jedoch noch der Bereich frei, der für Subnetz 4 nicht verwendet werden konnte: Netz 137.18.3.84 bis 137.18.3.87.

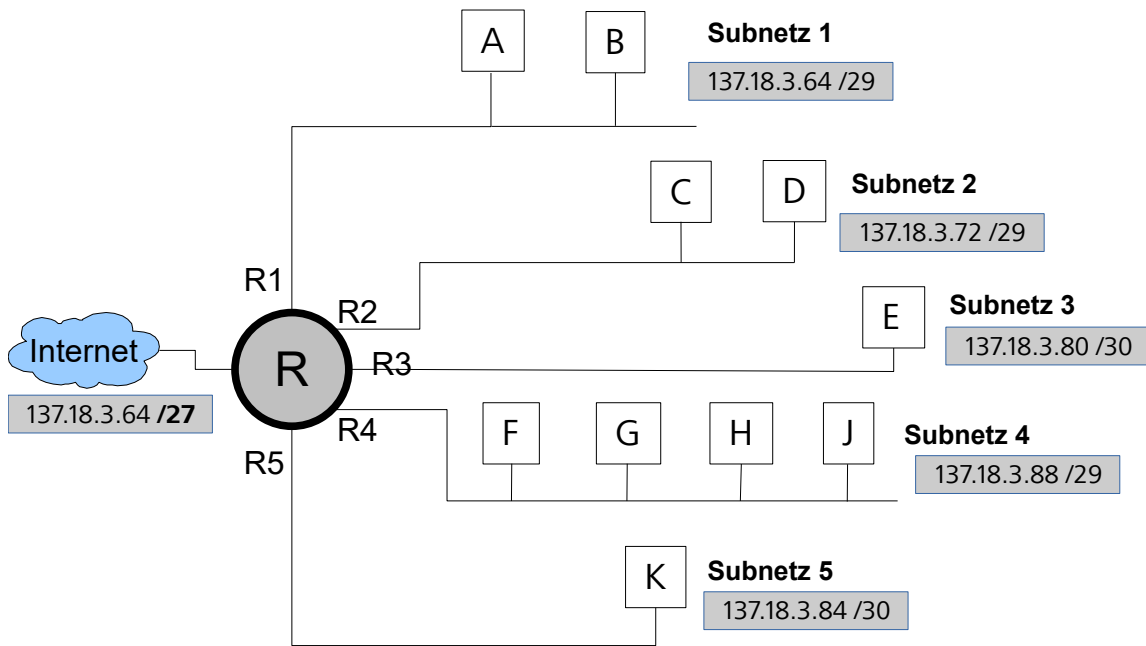


Abbildung 13

7.3 IPv6-Adressen

7.3.1 Adress-Typen und -Formate

Die Länge von IPv6-Adressen beträgt 128 Bit (zum Vergleich IPv4: 32 Bit). Die Darstellung erfolgt in acht 16-Bit-Blöcken in hexadezimaler Schreibweise, durch Doppelpunkte getrennt. Ähnlich wie bei IPv4 wird die Adresse in einen Netz- und einen Host-Anteil aufgeteilt, wobei die Größe des Netzanteils mithilfe einer Maske angegeben wird. Der Netzanteil besteht aus einem Präfix (Größe abhängig vom Adresstyp), der Global-ID und einer Subnetz-ID (sog. Site Level Aggregator SLA)

Beispiel `2001:DB8:0:1234:ABCD:EF98:5670:1/64`

In der Praxis wird eine IPv6-Adresse viele Nullen enthalten, sodass eine verkürzte Schreibweise aufeinander folgender Nullen erlaubt ist: `FF02:1:0:0:0:0:A:3` wird zu `FF02:1::A:3`

Adresszuweisung, beispielsweise

- `2001:ABC::/32` Adressbereich, den ein ISP vom RIR zugewiesen bekommt
- `2001:ABC:0A3B::/48` Netzadresse, die ein Kunde vom ISP erhält
- `2001:ABC:0A3B:0001::/64` Netzadresse mit 16 Bit Subnetzadresse (SLA)
- `2001:ABC:0A3B:0001:0:0:192.168.3.7/128` Interfaceadresse in IPv4-kompatibler Schreibweise

Unicast-Adressen dienen zur eindeutigen Identifikation eines Interfaces.

Das „Aggregatable Global Address Format“ besteht aus einem 3-Bit-Präfix, dem 45-Bit Global Routing Präfix, dem 16-Bit SLA und dem 64-Bit Interface Identifier. Die ersten drei Bit definieren dieses Format, das immer einen 64-Bit Interface Identifier hat. Dieses Format ist erkennbar am ersten Block: von `2000::/3` (binär beginnend mit 001) bis `E000::/3` (binär 111). Dieser Adressbereich wird derzeit von der IANA als `2001::/16` an die RIRs vergeben.

Die SLA-Adresse ist vergleichbar mit der Subnetzmaske von IPv4 (statt 16 Bit, d.h. 65536 Subnetze, kann vom ISP auch ein kleinerer Bereich vergeben werden). Es gibt spezielle Präfixe wie `2001:DB8::/32` für Dokumentationen oder Adressen für das Tunneling von IPv6 über IPv4.

Link-Local-Adressen werden von jedem Interface automatisch selbst erzeugt. Eine solche Adresse beginnt immer mit dem link-local-Präfix `FE80::/10` z.B. `FE80::0224:1DFF:FEB7:1E48`

Mit solchen Adressen ist eine Kommunikation im IPv6-Internet nicht möglich (sie können nicht geroutet werden), sie dienen jedoch als lokale Adresse für die automatische Konfiguration eines Rechners bzw. Interfaces mithilfe

eines Routers.

Die letzten 64 Bit werden bei Ethernet-Interfaces aus der 6-Byte MAC-Adresse der Netzwerkkarte gebildet. Beispiel: Die MAC-Adresse sei `00:24:1D:B7:1E:48`. Da jedoch 8 Byte (64 Bit) für den Interface-Identifizierer benötigt werden, wird in der Mitte der MAC-Adresse `FF:FE` eingefügt: `00:24:1D:FF:FE:B7:1E:48`. Nun wird im linken Byte das zweite Bit von rechts invertiert: im Beispiel wird aus (binär) `00000000` der Wert `00000010`, also im Ergebnis (hex): `0224:1DFF:FEB7:1E48`

Es ist auch bei IPv6 ein freier Adressbereich definiert: **Unique Local Address** vergleichbar mit 10.0.0.0 oder 192.168.0.0 bei IPv4. Es besteht damit die Möglichkeit zur Vergabe eindeutiger Adressen für lokale Netze, die aber nicht geroutet werden (d.h. ISP-unabhängige Adressierung). Der Präfix ist `FC00::/7` (d.h. beginnend mit FC oder FD). Dieser Adresstyp besteht also aus einem 7-Bit-Präfix, 41-Bit Global-ID, 16-Bit SLA und 64-Bit Interface-ID. Der FC-Bereich wird möglicherweise in der Zukunft von der IANA als global eindeutige ULAs vergeben (um z.B. Adress-Kollisionen in lokalen Netz oder beim Tunneling zwischen mehreren lokalen Netzen zu vermeiden), der FD-Bereich bleibt dann frei verfügbar.

Ein wichtiges Element bei der Adressierung sind **Multicast-Adressen**: ein Multicast-Paket wird an alle Interfaces verschickt, die mit dieser Adresse gekennzeichnet sind. Multicast-Adressen beginnen mit `FF00::/8`. Die darauf folgenden vier Bit bezeichnen die Lifetime (0 permanent, 1 temporär), die weiteren vier Bit den Gültigkeitsbereich (Scope), der definiert „wie weit“ ein Multicast-Paket geroutet werden darf: 1 (Node), 2 (Link), 5 (Site), 8 (Organization), E (Global). Alle Rechner und Router müssen so konfiguriert sein, dass sie sich in der sog. „All-node-multicast-group“ mit der Adresse `FF02::1` befinden (also Pakete mit dieser Zieladresse annehmen), Router zusätzlich auch `FF02::2`

An die Adresse `FF02::2` werden sog. Router Solicitation Messages (s.u.) geschickt, beantwortet werden sie mit einem Router Advertisement (an `FF02::1`), das zur automatischen Adresskonfiguration dient.

Für sog. Neighbor Solicitation Messages (vergleichbar mit ARP) wird eine spezielle Multicast-Adresse aus `FF02::` und einem Teil der IPv6-Unicast-Adresse oder der MAC-Adresse gebildet.

Multicast ersetzt die Broadcast-Funktionalität von IPv4.

7.3.2 Automatische Konfiguration für IPv6

Im Folgenden wird der prinzipielle Ablauf der Kommunikation zwischen Rechner und Router bzw. zwischen zwei Rechnern im lokalen Netzwerk, zur automatischen Konfiguration von IPv6-Adressen im Rechner, Mitteilung der Gateway-Adresse (also des Routers) im Netzwerks oder der Zuordnung der IPv6-Adresse(n) zur MAC-Adresse dargestellt. Für diese Kommunikation wird ausschließlich das ICMPv6-Protokoll verwendet.

Die folgenden, prinzipiellen Elemente der Kommunikation treten nicht immer alle und nicht unbedingt immer in dieser Reihenfolge auf.

1. Multicast Listener Report an die IPv6-Multicast-MAC-Adresse `33:33:00:00:00:16` bzw. IPv6-Multicast-Adresse `FF02:16`. Hiermit werden (sofern vorhanden) die verschiedenen, aktuell im Router oder Rechner bekannten Multicast-Gruppenadressen mitgeteilt
2. **Neighbor-Solicitations** (eine Anfrage, um die zur einer IPv6-Adresse gehörende MAC-Adresse eines Ziels herauszufinden) an die eigene sog. Solicited-Node-Multicast-Adresse `FF02::1:FF<rechte_drei_Byte_der_MAC-Adresse>` bzw. die rechten drei Byte der fest zugewiesenen IPv6-Adresse (nur beim Netzwerk-Start); eine Antwort sollte hierauf nicht kommen, da eine Antwort bedeuten würde, dass diese (eigene) IPv6-Adresse bereits an einen anderen Rechner vergeben wurde.
3. **Neighbor-Advertisements** mit der eigenen Link Local bzw. der fest zugewiesenen Adresse als Quelle an die Multicast-Adresse `FF02::1` (nur beim Netzwerk-Start) mit der Ziel-MAC-Adresse `33:33:00:00:00:01`, womit sich alle Rechner „angesprochen fühlen“. Damit kennen nun alle Rechner im Netz die MAC- und IPv6-Adresse dieses Nachbarn.
4. Neighbor-Solicitations an andere Rechner im lokalen Netz, um deren MAC-Adresse gezielt herauszufinden (analog ARP), als Ziel-Adresse wird die Solicited-Node-Multicast-Adresse des Ziels verwendet `FF02::1:FF<rechte_drei_Byte_der_Zieladresse>`
5. als Antwort darauf schickt der angesprochene Rechner ein Neighbor-Advertisement zurück (direkt an die Adresse des fragenden Rechners) mit seiner MAC-Adresse
6. weitere Neighbor-Solicitations und Neighbor-Advertisements zum Abgleich zwischen den verschiedenen Adresstypen (Link Local an Unique Global, Link Local an Link Local)
7. **Router-Solicitation** als Frage eines Rechners nach dem Gateway im lokalen Netzwerk; geht an die Router-Multicast-Adresse `FF02::2` unter der sich Router „angesprochen fühlen“

8. als Antwort darauf wird vom Router ein **Router-Advertisement** verschickt an die Multicast-Adresse `FF02::1` unter der sich alle Geräte im lokalen Netz „angesprochen fühlen“. Dieses Paket enthält das Adress-Präfix des Netzwerks (meistens 64 Bit bzw. 8 Byte lang); mit diesem erzeugt sich der Rechner selbstständig eine eigene dynamische Unique Global bzw. Unique Local Adresse aus dem Präfix und den rechten 8 Byte der Link Local Adresse; außerdem ist damit dem Rechner die Gateway-Adresse automatisch bekannt (sie braucht daher nicht mehr wie bei IPv4 manuell oder per zusätzlichem DHCP-Protokoll vergeben zu werden).
9. Die aus dem Präfix und der Link Local Address gebildete Unique Global Address enthält unmittelbar die MAC-Adresse des Rechners. Damit ist der Rechner im Internet eindeutig nachverfolgbar, was oftmals unerwünscht ist. Die sog. **Privacy Extensions** erlauben (basierend auf dem Präfix) die Generierung einer zusätzlichen, zufällig gewählten Global Unique Address, die je nach Voreinstellungen im Betriebssystem nur für einen begrenzten Zeitraum gültig ist und danach erneuert wird. Auf eine Neighbor Solicitation an diese eigene, zufällig erzeugte Adresse darf keine Antwort kommen (damit wird sichergestellt, dass sie im lokalen Netzwerk nicht bereits existiert).

7.3.3 Path MTU Discovery

Die Maximum Transmission Unit MTU beträgt im normalen Ethernet 1500 Byte. Da die MTU hardware-abhängig ist, kann ein Rechner direkt nur die MTU des eigenen LANs ermitteln. Wird bspw. ein TCP-Paket mit einer größeren MSS als 1440 Byte Nutzdateninhalt abgeschickt (bei MTU 1500 Byte und 40 Byte IPv6-Header) und trifft dieses Paket auf seinem Weg zum Ziel auf ein Netzwerk mit geringerer MTU, so kann es nur weitergeleitet werden, wenn es fragmentiert wird. Diese Fragmentierung durch Router auf dem Pfad ist bei IPv4-Paketen möglich, bei IPv6 jedoch nicht. Hier gibt es nur den Fragmentation Extension Header, der ausschließlich eine Fragmentierung durch den Quell-Rechner erlaubt. Daher ist IPv6 auf ein funktionierendes Path MTU Discovery angewiesen: ein sendender Rechner muss eine Information bekommen können, dass ein Netzwerk auf dem Pfad nur eine geringere MTU hat. Hierzu dient der ICMPv6-Typ 2 (Packet too big), der an den Quellrechner von demjenigen Router zurückgeschickt wird, der den Anschluss an das Netzwerk mit der geringeren MTU hat. Nun fragmentiert der Quellrechner alle folgenden Pakete mit derjenigen maximalen MTU, die in dem ICMPv6-Paket übermittelt wurde. Dieser Wert wird für ca. 10 Minuten im lokalen Kernel-Routing-Cache gespeichert.

Die minimale erlaubte MTU bei IPv6 beträgt 1280 Byte, bei kleinerer MTU ist ein Netzwerk nicht IPv6-fähig. Ein Routingprotokoll wie bspw. RIPv6 würde in diesem Fall eine Ungültig-Meldung (Metrik 16) verschicken.

7.4 Steuerprotokolle

7.4.1 ARP

Die Aufgabe des ARP-Protokolls ist die Zuordnung einer MAC-Adresse (Hardware-Adresse der Netzwerkkarte) zur IP-Adresse. Mit der IP-Adresse (Vermittlungsschicht) wird das endgültige Ziel eines Paketes benannt. Die MAC-Adresse (Sicherungsschicht) ist notwendig, um in einem lokalen Subnetz ein Paket zu einem lokalen Ziel (insbesondere dem Router) zu transportieren.

Ein Rechner bzw. ein Anwendungsprotokoll baut eine Verbindung zu einem Zielrechner i.d.R. anhand der Ziel-**IP**-Adresse auf. Diese Adressierung auf der Vermittlungsschicht (Schicht 3) ist auf der Sicherungsschicht allerdings nicht bekannt, hier werden die MAC-Adressen verwendet. Bei der Übergabe eines IP-Paketes an die Vermittlungsschicht, muss daher die **MAC**-Adresse des Ziels bekannt sein, sodass beim sendenden Rechner bereits vor dem senden eines Paketes eine Verknüpfung bzw. Zuordnung von IP- zu MAC-Adressen vorhanden sein muss. Diese Zuordnung kann manuell mittels einer statischen Adresstabelle vorgenommen werden oder dynamisch mithilfe des Address Resolution Protokolls ARP.

Es ist zunächst zu unterscheiden, ob das Ziel im eigenen Subnetz (LAN) des Senders liegt oder nicht. Ist dies der Fall, so wird die zur IP-Adresse gehörige MAC-Adresse des Ziels aus der Adresstabelle (ARP-Cache) entnommen und kann somit bei der Erstellung des Ethernet-Frame-Headers als Ziel-MAC-Adresse eingetragen werden. Liegt die IP-Adresse des Ziels jedoch außerhalb des eigenen Subnetz-Bereiches, so wird der Datenrahmen an die MAC-Adresse des Standard-Routers geschickt. Dessen IP-Adresse muss in der Netzwerk-Konfiguration eines Rechners bekannt sein (sog. Default-Gateway).

Die manuelle Pflege einer Adresstabelle ist sehr aufwändig und fehleranfällig, sodass üblicherweise das dynamische

ARP-Protokoll zum Einsatz kommt. Möchte der Sender ein IP-Paket abschicken und kennt die zugehörige MAC-Adresse (entweder Ziel-Rechner oder Router) noch nicht, wird zunächst ein **ARP-Request** an die MAC-Broadcast-Adresse geschickt. Das Format diese Anfrage ist in Abbildung 14 dargestellt.

Die Standardwerte sind: Hardware „Ethernet“, Protocol „IPv4“, Hardware-Size (hier Länge der MAC-Adresse) 6, Protocol-Size (Länge der IP-Adresse) 4, Op-Code entweder „Request“ oder „Reply“, danach folgen die beiden eigenen Adressen des Senders und die beiden Adressen des Ziels. Beim ARP-Request ist die MAC-Target-Adresse des Ziels leer (Null), also noch unbekannt. Zu beachten ist, dass bereits beim ARP-Request die eigene Adress-Kombination mitgeteilt wird, obwohl niemand danach gefragt hatte. Dies ist einerseits effizient, da praktisch immer der andere Rechner Daten zurückschicken wird und damit ebenfalls die Adress-Kombination benötigt, andererseits besteht dadurch das Risiko von ARP-Spoofing-Angriffen. Hierbei können mit gefälschten Adress-Kombinationen Datenströme im LAN umgeleitet werden.

Ein ARP-Request wird per MAC-Broadcast-Adresse an alle Rechner im LAN geschickt, woraufhin nur der Besitzer der angefragten IP-Adresse ein ARP-Reply gezielt an die MAC-Adresse des anfragenden Rechners zurückgeschickt.

Hardware-Type (2 Byte)		Protocol-Type (2)
Hardware-Size (1)	Protocol-Size (1)	Op-Code (2)
Sender MAC-Address (erste 4 Byte)		
Sender MAC-Address (letzte 2 Byte)		Sender IP-Address (erste 2 Byte)
Sender IP-Address (letzte 2 Byte)		Target MAC-Address (erste 2 Byte)
Target MAC-Address (letzte 4 Byte)		
Target IP-Address (4)		

Abbildung 14: ARP-Header

7.4.2 ICMP

In den vorangegangenen Kapitel wurde bereits auf Situationen bei der Übertragung von Paketen hingewiesen, in denen Fehlermeldungen verschickt werden. Ist beispielsweise das Zielnetz oder der Zielrechner nicht erreichbar oder setzt ein Router den TTL-Zähler im IP-Header auf Null, so wird der Absender des Paketes mittels einer Fehlermeldung über das Problem informiert. Diese Fehler- oder Informationsmeldungen werden mithilfe eines speziellen Protokolls übertragen, dem Internet Control Message Protocol ICMP.

0			31
Type	Code (Subtype)	Checksum	
zusätzliche Informationen (abh. vom Type)			
Daten, optional			

Abbildung 15: ICMP-Protokoll-Header (Grundstruktur)

Die Struktur des ICMP-Headers ist variabel und hängt vom jeweiligen Typ der Meldung ab. Es sind verschiedene Typen und Unter-Typen definiert:

- Echo Request (Typ 8), siehe Abbildung 16: Es wird ein ICMP-Paket an den Zielrechner geschickt, mit der Aufforderung auf dieses Paket zu antworten. Mit dieser Funktion (Kommando „ping“) kann überprüft werden, ob der Zielrechner (genauer: seine IP-Funktionalität) erreichbar ist. Der absendende Rechner setzt für jeden ping-Prozess einen eindeutigen „Identifier“ sowie für jedes ICMP-Paket eine Sequenznummer (wird mit jedem Paket um 1 inkrementiert); es wird eine kleine beliebige Datenmenge (32 oder 48 Byte) in das Paket gesetzt.
- Echo Reply (Typ 0): positive Antwort auf „Echo Request“; die Werte für Identifier, Sequenznummer sowie die Daten werden in das Antwortpaket übernommen.
- Destination unreachable (Typ 3): Fehlermeldung, wenn ein Paket nicht an das Ziel übermittelt werden

kann. Da es unterschiedliche Gründe dafür geben kann, wird mithilfe des Subtypes die Ursache angegeben u.a.:

- Network unreachable (Subtype 0)
- Host unreachable (Subtype 1)
- Port unreachable (Subtype 3)
- Fragmentation needed (Subtype 4): es wird zusätzlich die MTU desjenigen Netzwerks zurückgemeldet, durch das die Fragmentierung erforderlich wird.

Im Datenteil wird ein Teil des den Fehler verursachenden Paketes eingesetzt, um dem Absender die entsprechende Zuordnung zu ermöglichen.

- Time exceeded (Type 11, Subtype 0): Der TTL-Zähler wurde auf Null gesetzt, sodass das Paket nicht mehr weitergeleitet wurde.
- Redirect (Typ 5): Ein Rechner kann immer nur *ein* Default-Gateway (Standard-Router) haben; ist ein LAN jedoch an zwei oder mehr Router angeschlossen und das Zielnetzwerk des Paketes ist über einen anderen Router als das Default-Gateway erreichbar, so kann dieser Router mittels ICMP-Redirect den richtigen Router mitteilen.
- Source Quench (Typ 4): Ist ein Router überlastet (Stau-Situation) kann er mittels ICMP-Source-Quench einen Rechner auffordern, seine Datenrate zu verringern. Allerdings ist diese Art der Überlast-Kontrolle im Vergleich zu anderen Verfahren (siehe Modul Netze, Media Systems 3. Semester) ineffektiv und wird heute nicht mehr verwendet (z.B. macht ein ICMP-Source-Quench keine Aussage darüber, ob das auslösende Paket das Ziel erreicht hat oder nicht).

Die Berechnung der Checksum erfolgt wie im IP-Header: Aufsummieren aller 16-Bit-Wörter im Header (Checksum-Feld hierbei Null), sollte die Summe größer als 16 Bit sein, werden die linkesten Bytes der Summe zu den beiden rechten hinzuaddiert, danach bilden des Komplements

0			31
Type 8		Subtype 0	Checksum
Identifizier			Sequence Number
Daten			

Abbildung 16: ICMP Echo Request

7.4.3 ICMPv6

Das ICMP-Protokoll bei IPv4 ist ein Hilfsprotokoll, das eine effizientere Kommunikation ermöglicht; fehlen jedoch ICMP-Meldungen oder werden sie durch Firewalls blockiert, so wird dadurch die Kommunikation nicht grundsätzlich eingeschränkt oder verhindert. Im Gegensatz dazu ist bei IPv6 ist das ICMPv6-Protokoll ein unverzichtbarer Bestandteil der IPv6-Kommunikation.

Die Grundstruktur des ICMPv6-Headers ist dieselbe wie bei ICMP (siehe Abbildung 15). Es handelt sich jedoch um ein eigenständiges Protokoll, auf das im Feld Next-Header von IPv6 (Wert 58) verwiesen wird. Neben den bereits von ICMP bekannten Funktionen wie „Destination unreachable“, „Time exceeded“, „Echo Request/Reply“ ist eine wichtige Aufgabe von ICMPv6 das sog. „Neighbor Discovery“ NDP. Diese Funktion übernimmt u.a. die wichtige Aufgabe des ARP-Protokolls bei IPv4, also die Zuordnung der IPv6-Adresse zur MAC-Adresse.

Wie bereits in Kapitel 7.3.2 dargestellt, erfolgt die Konfiguration der Rechneradressen automatisch mithilfe des Routers; ein zusätzlicher Dienst wie DHCP bei IPv4 ist hier aber nicht mehr nötig. Die hierfür erforderliche Kommunikation im lokalen Netzwerk zwischen Rechner und Router wird mithilfe des ICMPv6-Protokolls ausgeführt. Die folgenden ICMPv6-Typen werden verwendet:

- Router Solicitation (Typ 133): Anfrage nach einem Router Advertisement
- Router Advertisement (Typ 134): Der Router macht sich im lokalen Netzwerk bekannt und liefert das IPv6-Präfix des Netzwerks, aus dem sich alle Rechner die eigene IPv6-Adresse bilden.
- Neighbor Solicitation (Typ 135): Anfrage nach Auflösung einer IPv6-Adresse in die MAC-Adresse (entsprechend ARP bei IPv4)
- Neighbor Advertisement (Typ 136): Antwort auf Neighbor Solicitations

Aufgrund der nicht zulässigen Fragmentierung von IPv6-Paketen durch Router (nur der Quell-Rechner selbst darf fragmentieren), ist für IPv6 die Funktion des „MTU Path Discovery“ sehr wichtig. Statt des entsprechenden „Destination unreachable“-Subtypes, wird bei ICMPv6 der eigenständige Typ „Packet too Big“ (Typ 2) verwendet.

7.5 Routing-Protokolle

7.5.1 Distance-Vector-Verfahren

Das Distance-Vector-Routing arbeitet nach einem einfachen Prinzip, bei dem (im Gegensatz zu Link-State-Verfahren) *keine* Berechnung der kürzesten Wege auf Basis der vollständigen Netzwerk-Topologie mithilfe eines Routing-Algorithmus erfolgt. Stattdessen wird eine Routing-Information lediglich schrittweise von einem Router an seine unmittelbaren Nachbarn weitergegeben. Zu diesem Entfernungswert (Distanz) wird der Abstand zum Nachbarn (Metrik) hinzuaddiert.

Formal muss für jeden Knoten i der kürzeste Gesamt-Weg $d(i, j)$ zu allen anderen Knoten j über die jeweils vorhandenen Nachbar-Knoten n ermittelt werden. Die Entfernung (Metrik) vom Ausgangsknoten i zu seinen direkten Nachbarn n sei $M(i, n)$ mit $d(j, j) = 0$:

$$d(i, j) = \min [M(i, n) + d(n, j)] \quad \text{für alle } n$$

Im linken Teil von Abbildung 17 ist ein Netzwerk mit den Knoten B, C und D dargestellt, das im rechten Teil der Abbildung durch Einschalten des Knotens A erweitert wird. Der ursprüngliche Wert für die Distanz B-C (linker Teil der Abbildung) ist: $d(B, C) = M(B, D) + d(D, C) = 3 + 2 = 5$

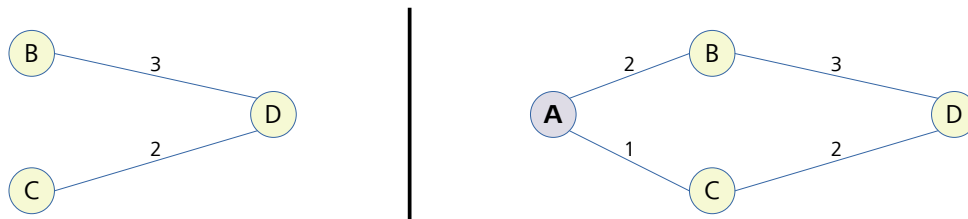


Abbildung 17: Distance-Vector-Routing

Berechnet man die kürzesten Wege vom neu hinzugekommenen Ausgangsknoten $i=A$ (rechter Teil der Abbildung) zu allen anderen Knoten j , so ergibt sich mit n aus der Menge $\{B, C\}$ und j aus der Menge $\{B, C, D\}$ die folgende Berechnung.

Ausgangsknoten $i=A$, in jedem Berechnungsschritt jeweils mit $n=B$ und $n=C$

$j=B$:

$$d(A, B) = M(A, B) + d(B, B) = 2 + 0 = 2$$

$$d(A, B) = M(A, C) + d(C, B) = 1 + 5 = 6 \rightarrow \text{Minimum } d(A, B) = 2 \text{ über Nachbar } n=B$$

$j=C$:

$$d(A, C) = M(A, B) + d(B, C) = 2 + 5 = 7$$

$$d(A, C) = M(A, C) + d(C, C) = 1 + 0 = 1 \rightarrow \text{Minimum } d(A, C) = 1 \text{ über Nachbar } n=C$$

$j=D$

$$d(A, D) = M(A, B) + d(B, D) = 2 + 3 = 5$$

$$d(A, D) = M(A, C) + d(C, D) = 1 + 2 = 3 \rightarrow \text{Minimum } d(A, D) = 3 \text{ über Nachbar } n=C$$

Damit können die so berechneten kürzesten Wege in die Routing-Tabelle von A eingetragen werden.

Die Ausgangslage hat sich durch das Einschalten von Knoten A jedoch verändert: Der kürzeste Weg von B zu C $d(B, C)$ führt nun gar nicht mehr über Nachbar D sondern über Nachbar A. Betrachtet man B als Ausgangsknoten i , so ergibt sich nun (hier nur für $j=C$ dargestellt)

$$d(B, C) = M(B, A) + d(A, C) = 2 + 1 = 3$$

$$d(B, C) = M(B, D) + d(D, C) = 3 + 2 = 5 \rightarrow \text{neues Minimum } d(B, C) = 3 \text{ über Nachbar } n=A$$

Bei der obigen Berechnung von $d(A, C)$ wurde jedoch mit $d(B, C) = 5$ gerechnet, sodass diese Berechnung verbessert werden muss:

$$d(A, C) = M(A, B) + \mathbf{d(B, C)} = 2 + \mathbf{3} = 5$$

$$d(A, C) = M(A, C) + d(C, C) = 1 + 0 = 1 \rightarrow \text{Minimum bleibt wie oben } d(A, C) = 1$$

Es entsteht ein Problem: Der Weg $d(A, C)=5$ führt von A über B wieder zurück zu A und dann weiter zu C. Dies

macht sich zunächst nicht negativ bemerkbar, da der kürzeste Weg $d(A,C)$ nach wie vor der direkte ist. Fällt jedoch die Verbindung A-C aus (formal wird damit $M(A,C)=\infty$), wird dieser (unsinnige) Weg als kürzester in die Routing-Tabelle eingetragen und es entsteht eine Routing-Schleife.

$$d(A,C) = M(A,B) + d(B,C) = 2 + 3 = 5$$

$$d(A,C) = M(A,C) + d(C,C) = 1 + \infty = \infty \rightarrow \text{Minimum } d(A,C) = 5$$

Mit der erneuten Berechnung bei Ausgangsknoten B korrigiert sich dieser Fehler jedoch wieder:

$$d(B,C) = M(B,A) + d(A,C) = 2 + 5 = 7$$

$$d(B,C) = M(B,D) + d(D,C) = 3 + 2 = 5 \rightarrow \text{Minimum } d(B,C) = 5 \text{ über Nachbar } n=D$$

In größeren Netzwerken oder ungünstigen Fällen korrigiert sich dieser Fehler jedoch nicht so schnell oder ggf. auch gar nicht: man spricht daher vom „**Count-to-Infinity-Problem**“.

Um das Konvergenz-Verhalten genauer zu untersuchen, betrachte man Abbildung 18, die ein einfaches Netzwerk zeigt, bei dem fünf Knoten aneinandergereiht sind (alle Verbindungen haben die Metrik 1). Im Anfangszustand sei Knoten A ausgeschaltet, sodass keiner der anderen Knoten eine Wege-Information zu A hat. Wird Knoten A eingeschaltet, meldet er sich bei seinem unmittelbaren Nachbarn B; somit kennt B einen Weg zu A in der Metrik 1. Mit dem nächsten Austausch-Intervall (z.B. einmal pro Minute) gibt Knoten B diese Information an seinen Nachbarn C weiter, der damit einen Weg in der Gesamt-Metrik 2 zu A über seinen Nachbarn B kennt. Nach zwei weiteren Austauschintervallen sind alle Knoten über den Weg zu A informiert. Alle Knoten kennen jeweils die Gesamt-Metrik zu A und den jeweiligen Nachbar-Knoten, über den A erreichbar ist, aber kein Knoten kennt die Topologie des gesamten Netzwerks.

	A	1	B	1	C	1	D	1	E
ein			1		-		-		-
			1		2		-		-
			1		2		3		-
			1		2		3		4

Abbildung 18: Count-to-Infinity: „gute Nachricht“ (jeweils Gesamt-Metrik zu Router A)

Dieser Austausch einer „guten Nachricht“ (d.h. es kommt ein Weg oder Netzwerk-knoten hinzu) erfolgt noch verhältnismäßig schnell in Abhängigkeit von der Länge der Austauschintervalle. Erheblich länger dauert dagegen der Austausch einer „schlechten Nachricht“: Knoten A wird ausgeschaltet. Da sich Knoten A bei seinem Nachbarn B aber nicht aktiv abmeldet (was im Falle eines plötzlichen Leitungsausfalls auch gar nicht möglich wäre), bleibt die bisherige Route in der Routingtabelle von B stehen. Knoten B bekommt von seinem ehemaligen Nachbarn A nun einfach keine Informationen mehr. Allerdings bekommt B von seinem anderen Nachbarn C die Information „ich kenne einen Weg zu A in der Metrik 2“, sodass B seine Routingtabelle aktualisiert: Er kennt nun einen Weg zu A in der Metrik 3 über C. Dieser Weg führt zwar zurück über B selbst, aber diese Information ist in diesem einfachen Protokoll nicht enthalten (in der Praxis würde diese zusätzliche Pfad-Information erhebliche Ressourcen für Übertragung und Auswertung benötigen). Im nächsten Austauschintervall bekommt Knoten C von seinen beiden Nachbarn B und D die Information, dass Knoten A in Metrik 3 zu erreichen ist, also aktualisiert C seinen Weg zu A auf Metrik 4. Dieser Verlauf geht unendlich weiter, es ergibt sich keine Konvergenz, sofern keine maximal zulässige Metrik definiert ist (bei RIP ist die maximale Metrik 15), damit wird deutlicher, was der Begriff „**Count-to-Infinity-Problem**“ meint.

	A	1	B	1	C	1	D	1	E
ein			1		2		3		4
aus			3		2		3		4
			3		4		3		4
			5		4		5		4
			5		6		5		6

Abbildung 19: Count-to-Infinity: „schlechte Nachricht“ (jeweils Metrik zu Router A)

Die Problematik des Count-to-Infinity führt in der Praxis zu einer langsamen Konvergenz des Verfahrens. Die Lösung des Problems besteht in der sehr aufwändigen Übermittlung und Verarbeitung von vollständigen Pfad-Informationen, weshalb mit der Funktion „**Split-Horizon**“ eine einfachere Möglichkeit entwickelt wurde, um die Auswirkungen des Count-to-Infinity-Problems zumindest zu begrenzen. In dem einfachen Netzwerk in Abbildung 19 entsteht die Problematik dadurch, dass Knoten C die Information bezüglich des Weges zu Knoten A, die er von seinem Nachbarn B erhalten hat, an diesen Nachbarn B wieder zurückgibt. Solange der Weg von B zu A existiert, ist dieses Verhalten unproblematisch, da B zwar zwei Informationen hat (Metrik 1 direkt zu A und Metrik 3 über C), aber nur der kürzere (richtige) Weg für die Routing-Tabelle verwendet wird. Fällt Router A jedoch aus, gibt es nur noch die Information mit dem Weg zu A über C. Indem sich Router C merkt, von welchem Nachbarn er welche Information erhalten hat, und genau diese Information *nicht* wieder zurückgibt, vermeidet man damit das Problem: Knoten B bekommt keine Information mehr von C bezüglich A und kennt damit nur noch den direkten Weg zu A. In dem hier gezeigten einfachen Netzwerk wird das Problem mit Split-Horizon sogar vollständig gelöst, in vermaschten Netzwerken können die „ungünstigen“ Routing-Informationen jedoch auf anderen Wegen trotzdem verbreitet werden. Split-Horizon kann zwar das Konvergenz-Verhalten in der Praxis verbessern, löst aber – genauso wie die folgende Erweiterung dieser Funktion – das Problem nicht.

Die Funktion „**Split-Horizon with Poisoned Reverse**“ erweitert das zuvor erläuterte Verfahren, indem Knoten C die von B gelernten Wege nicht nur (passiv) zurückhält, sondern aktiv eine Ungültig-Meldung für diese Wege zurückgibt (bei RIP hat die Metrik 16 die Bedeutung „ungültig“). Solange A eingeschaltet ist, lernen die Knoten C, D und E jeweils von ihrem linken Nachbarn den richtigen Weg zu A, geben aber diese Information mit der Metrik 16 („Weg ungültig“) an diesen Nachbarn zurück. Alle Router bekommen somit von ihrem rechten Nachbarn die Ungültig-Meldung bezüglich A, aber vom linken Nachbarn die richtige (bessere) Weg-Information. Fällt A jedoch aus, bekommt B nur noch eine Ungültig-Meldung für A vom Nachbarn C und löscht den Weg zu A sofort aus seiner Routing-Tabelle. Das gleiche passiert bei Knoten C, der nun *keine* Information bezüglich des Weges zu A vom linken Nachbarn B erhält sondern nur noch die Ungültig-Meldung vom rechten Nachbarn D und somit ebenfalls den Weg zu A aus seiner Routing-Tabelle löscht. Die Konvergenz wird also mit der Funktion Poisoned-Reverse noch schneller erreicht als nur mit Split-Horizon.

Eine weitere Verbesserung des Konvergenz-Verhaltens kann erreicht werden, wenn Meldungen nicht nur in fest definierten Intervallen verschickt werden, sondern sofort beim Auftreten von Änderungen. Wird Knoten A (wie in Abbildung 18) eingeschaltet, verbreitet sich diese „gute Nachricht“ nicht nur schrittweise mit jedem Austausch-Intervall sondern mittels „**Triggered Updates**“ unmittelbar nach Feststellung der Änderung bei Knoten B, der diese Information sofort an C weiterleitet usw.

Wichtiger Hinweis: In den zuvor gezeigten Beispielen wurde zum besseren Verständnis der Grundprinzipien immer der Weg zu einem *Netzwerknoten* bzw. einem *Router* angegeben. Routing-Protokolle wie RIP bestimmen jedoch den kürzesten Weg zu einem *Netzwerk*, sie verarbeiten also die IP-Adresse des Netzwerks und nicht die Adresse eines Routers.

7.5.2 RIP

Das Routing Information Protocol RIP war das erste dynamische Routing-Protokoll im Internet. Aufgrund seiner Einfachheit und der Beschränkung auf kleinere Netzwerke eignet es sich heute aber nur noch für den Einsatz in lokalen Netzwerken.

Das RIP-Protokoll arbeitet nach dem einfachen Prinzip des sog. **Distance-Vector-Routings**, bei dem – im Gegensatz zu Link-State-Verfahren – *keine* Berechnung der kürzesten Wege mithilfe eines Routing-Algorithmus erfolgt. Stattdessen wird eine Routing-Information lediglich schrittweise von einem Router an seine unmittelbaren Nachbarn weitergegeben. Daher ist auch RIP vom Count-to-Infinity-Problem betroffen.

Eine RIP-Information wird (auf der Transportschicht) als UDP-Paket verpackt und an UDP-Port 520 (IPv6: UDP-Port 521) versendet. Die Adressierung des RIP-Paketes kann in Abhängigkeit von der Konfiguration des Routers unterschiedlich sein: Entweder wird ein Nachbar-Router direkt adressiert (was in Mehrfachzugriffs-Netzen mit mehreren Routern unpraktisch ist) oder es wird die Multicast-Adresse 224.0.0.9 (IPv4) bzw. FF02::9 (IPv6) verwendet, unter der alle Router angesprochen werden.

0			31		
Command		Version		Routing-Domain	
Address Family			Route Tag		
Ziel-Netzadresse					
Netz-Maske					
Next Hop					
Metrik					

Abbildung 20: RIP-Header (RIPv2)

Der Protokoll-Header für RIPv2 ist in Abbildung 20 dargestellt (für den Fall IPv4; im Fall IPv6 (RIPng) sind die Felder wegen der längeren Adressen entsprechend länger, im Fall der Authentifizierung hat der Header eine andere Struktur) und enthält folgende Elemente:

- Command: Request (1) oder Response (2)
- Version: 1 (veraltet), 2 (für IPv4) oder ebenfalls 1 für IPv6; RIPng-Pakete werden von einem anderen Prozess verarbeitet – UDP-Port 521 – als RIPv1/v2-Pakete (UDP-Port 520).
- Routing-Domain (Prozess-ID): immer Null, es sei denn, es laufen gleichzeitig mehrere Routing-Prozesse
- Address Family: IP (2) oder Authentifizierung (FF:FF) mit einfachem Passwort
- Route-Tag (00:00): Unterscheidung von internen und (von anderen Routing-Protokollen) importierten Routen (z.B. die AS-Nr. von BGP)
- Netz-Adresse (Zielnetz)
- CIDR-Netzmaske
- Next Hop: IP-Adresse, an die ein Paket für obige Ziel-Netzadresse weitergeleitet werden soll, Null ist der Standardfall: Weiterleitung an den Absender dieser RIP-Nachricht
- Metrik für das Zielnetz

7.5.3 Link-State-Verfahren

Aufgrund der prinzipbedingten Schwächen von Distance-Vector-Verfahren werden die darauf basierenden Routing-Protokolle heute nicht mehr im internet-weiten Routing sondern nur noch in kleineren, einfach aufgebauten lokalen Netzwerken verwendet.

Bei den heute eingesetzten Link-State-Verfahren wird im ersten Schritt von jedem Router die Nachbarschaft erkannt (Adressen der Nachbarn und Entfernung), dann wird diese Information an *alle* anderen Router im Netzwerk übermittelt. Vereinfacht ausgedrückt, verfügt somit jeder Router über viele kleine Teilstückchen (Puzzle-Steine), bestehend aus den Nachbarschafts-Informationen aller anderen Router. Diese Puzzle-Steine können dann von jedem Router zum Gesamt-Netz zusammengesetzt werden: Jedem Router liegt damit die vollständige Topologie des Netzes vor, aus der mithilfe eines Routing-Algorithmus (siehe Kapitel 8) die kürzesten Wege zu den anderen Routern bzw. Netzwerken berechnet werden können.

Prinzipieller Ablauf:

- Aktive Erkennung der Nachbarn (und deren Adressen) mittels eines „Hello-Paketes“
- Messung der Paket-Laufzeit zu diesen Nachbarn und zurück mithilfe eines „Echo-Paketes“ und Verwendung dieses Wertes als Metrik, alternativ Verwendung der Übertragungsrate des Netzwerk-Interfaces oder eines festen Metrik-Wertes
- Generierung eines Link-State-Paketes mit der eigenen Identität, den Adressen der Nachbarn, den Metriken, ggf. weiteren Informationen wie Folge-nummern zur Erkennung veralteter Informationen
- Verteilung dieser Pakete mittels eines Flooding-Verfahrens
- Berechnung der kürzesten Wege mithilfe eines Routing-Algorithmus
- Erzeugen der aktualisierten Routing-Tabelle

Das Verfahren ist nicht nur erheblich aufwändiger als Distance-Vector-Routing, es hat ein grundlegendes Problem: Wird das Netzwerk eingeschaltet, existieren zu diesem Zeitpunkt noch gar keine Routing-Tabellen, auf deren Basis Link-State-Pakete an andere Router verschickt werden könnten. Daher muss diese Verteilung mittels eines Flooding-Verfahrens erfolgen: Es werden hierbei von jedem Router die über eine Eingangsleitung hereinkommenden Pakete an *alle* Ausgangsleitungen weitergeleitet. Die dadurch entstehende Flut an Paketen muss zum einen durch Streckenzähler oder Zeitstempel eingedämmt werden, zum anderen muss mithilfe von Folge-nummern sichergestellt werden, dass (aufgrund von „umherirrenden“ Link-State-Paketen) veraltete Routing-Informationen nicht zur Aktualisierung von Routing-Tabellen verwendet werden.

7.5.4 OSPF

Das Protokoll Open Shortest Path First OSPF ist ein hierarchisch aufgebautes und dadurch gut skalierbares Routing-Verfahren nach dem Link-State-Prinzip. Aufgrund seiner Komplexität ist es eher für größere Netzwerke geeignet, für kleinere Netzwerke stehen mit EIGRP oder IS-IS einfachere Alternativen zur Verfügung, sofern keine Distance-Vector-Verfahren eingesetzt werden sollen.

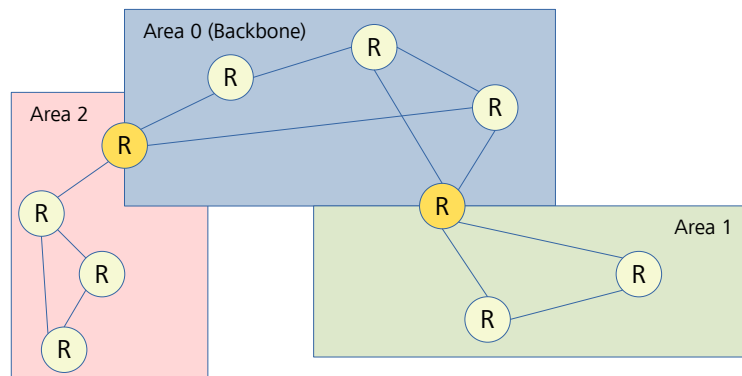


Abbildung 21: OSPF Areas

Ein Netzwerk mit dem OSPF-Routingprotokoll kann in sog. Areas unterteilt werden, mit dem Vorteil, dass in den jeweiligen Areas nur die Wege und Netzwerkknöten bzw. die Netzwerktopologie innerhalb einer Area bekannt sein müssen (kleinere Routingtabellen). Lediglich die Übergänge in die anderen Areas müssen bekannt sein (Area Border Router ABR). Innerhalb einer Area hat jeder Router die gleiche Link-State-Datenbank und kennt die ABRs. Ein Area Border Router kennt beide Datenbanken. Zusätzlich gibt es Grenz-Router, die zwischen den anderen angrenzenden sog. „autonomen Systemen“ vermitteln (AS-Boundary-Router ASBR).

Der Ablauf der Kommunikation im OSPF-Protokoll besteht u.a. aus den im Folgenden dargestellten Elementen.

OSPF-Hello

Nach dem Einschalten eines Routers bzw. dem Aktivieren des OSPF-Dienstes wird zunächst an alle vorhandenen Ausgangsleitungen ein OSPF-Hello-Paket gesendet, mit dem sich ein Router bei seinen Nachbarn bekannt macht. Die Hello-Pakete werden in regelmäßigen Abständen auch von den Nachbarn gesendet, sodass nach kurzer Zeit alle vorhandenen OSPF-Nachbarn mit ihrer Router-ID, der Area-ID, ggf. Authentifizierungsdaten und weiteren Informationen bekannt sind. Router werden zu Nachbarn, wenn die Area-ID übereinstimmt, ggf. die Authentifizierung erfolgreich ist und das Hello- und das Dead-Intervall gleich sind.

OSPF Database Description

Im Anschluss an das Hello wird das Format der sog. Link State Advertisements zwischen den Nachbarn mittels der Database-Description-Pakete vereinbart, d.h. die Elemente der späteren Routing-Datenbank (Austausch direkt zwischen den Routern). Zu Beginn handelt es sich lediglich um initiale Sequenznummern und die Struktur der Datenbank, im späteren Betrieb kann z.B. nach dem Einschalten eines neuen Routers auch die aktuelle Link State Datenbank übermittelt werden.

Link State Advertisements (LSA)

Die OSPF Database enthält die während des Betriebs von den Routern gesammelten Link State Advertisements (LSA). Diese werden von jedem Router per Flooding im gesamten Netzwerk verbreitet; sie enthalten die Informationen zu den eigenen Interface-Adressen eines Routers, zu Nachbarschaftsbeziehungen oder zu Netzwerk-Adressen und Masken. Je nach Art der Netzwerke enthalten die LSAs unterschiedliche Informationen. Aus diesen Informationen kann letztlich jeder einzelne Router selbsttätig die Topologie des gesamten Netzwerkes zusammensetzen und die kürzesten Wege mithilfe eines Routing-Algorithmus berechnen.

Es gibt verschiedene Typen von LSAs: Router-LSA (Typ 1), Network-LSA (Typ 2), Summary-LSA (Typ 3) usw.

Router LSA

Es soll zunächst die einfache Situation betrachtet werden, bei der einige Router mit Punkt-zu-Punkt-Leitungen miteinander verbunden sind.

In diesem Fall enthält ein Router-LSA den **Link-Typ 1 (Point-to-Point)** mit diesen Informationen:

- Link ID: Router-ID des Nachbarn
- Link Data: die eigene Interface-Adresse
- Metrik
- Zusätzlich wird die Netzadresse und Maske dieser Verbindung als „Stub-Netzwerk“ (s.u.) übermittelt.

Alle Router schicken ihre Router-LSAs per Flooding in das gesamte Netzwerk, sodass letztlich alle Router über die Informationen aus den jeweiligen Teilstücken verfügen und somit die Topologie des gesamten Netzwerkes zusammensetzen können (siehe Abbildung 22). Für dieses Netzwerk (mathematisch ein „Graph“) können mithilfe von Routing-Algorithmen die kürzesten Wege von einem Netzwerkknoten zu allen anderen berechnet werden.

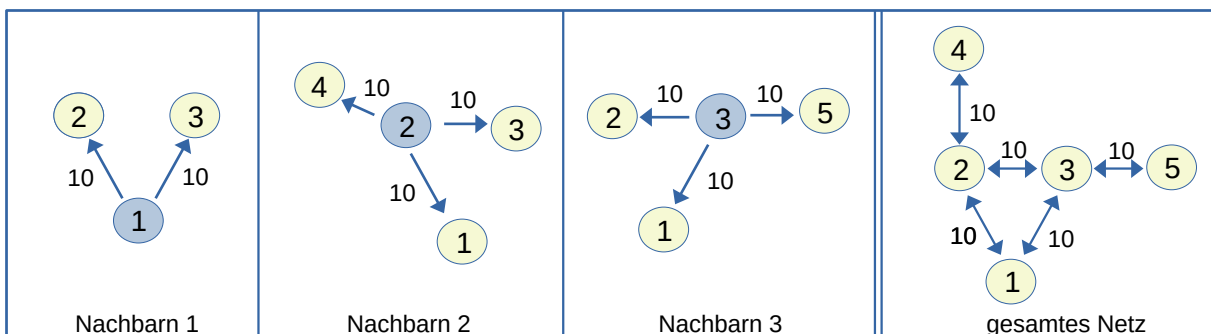


Abbildung 22

In der Realität besteht ein Netzwerk nicht nur aus Routern bzw. Knoten in einem Graphen, sondern enthält auch die lokalen Netzwerke von Unternehmen, Hochschulen oder Providern (einschließlich der privaten Heimnetzwerke). Diese Netzwerke sind Endpunkte im Gesamt-Netzwerk, die aus Sicht von OSPF keine weiteren Wege bieten: Sie werden als **Stub-Netzwerke (Link Typ 3)** bezeichnet.

Ein Router-LSA (Link Type 3) enthält diese Informationen:

- Link ID: Netzadresse
- Link Data: Netzmaske
- Metrik

Im Beispiel (Abbildung 23) sendet Router 4 ein Router LSA (mit dem Link-Typ 3) mit der Adresse 192.168.48.0 und der Maske 255.255.254.0, womit dieses Stub-Netzwerk allen anderen Routern bekannt gemacht wird.

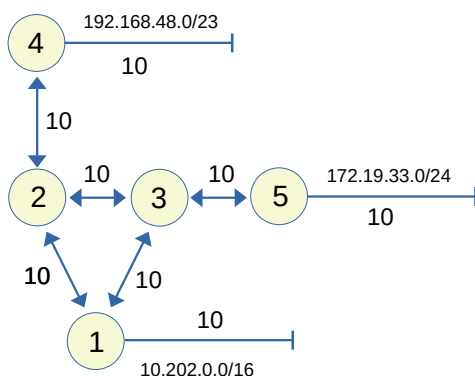


Abbildung 23: Stubnetzwerke

Router können nicht nur über Punkt-zu-Punkt-Leitungen verbunden sein (diese Techniken sind in der heutigen Praxis selten geworden), sondern können an Mehrfachzugriffsnetze (Multiple Access) mit oder ohne Broadcast-Funktion angeschlossen sein. Dies ermöglicht nicht nur den Anschluss von zwei, sondern auch von mehreren

Routern an ein einziges Netzwerk (linker Teil aus Abbildung 24). Aus der formalen Sicht von OSPF auf die Netzwerk-Topologie ist hierbei jeder Router mit jedem anderen direkt verbunden. Damit entstehen bei n Routern insgesamt innerhalb eines Mehrfachzugriffsnetzes $n(n-1)/2$ Nachbarschaftsbeziehungen (mittlerer Teil aus Abbildung 24).

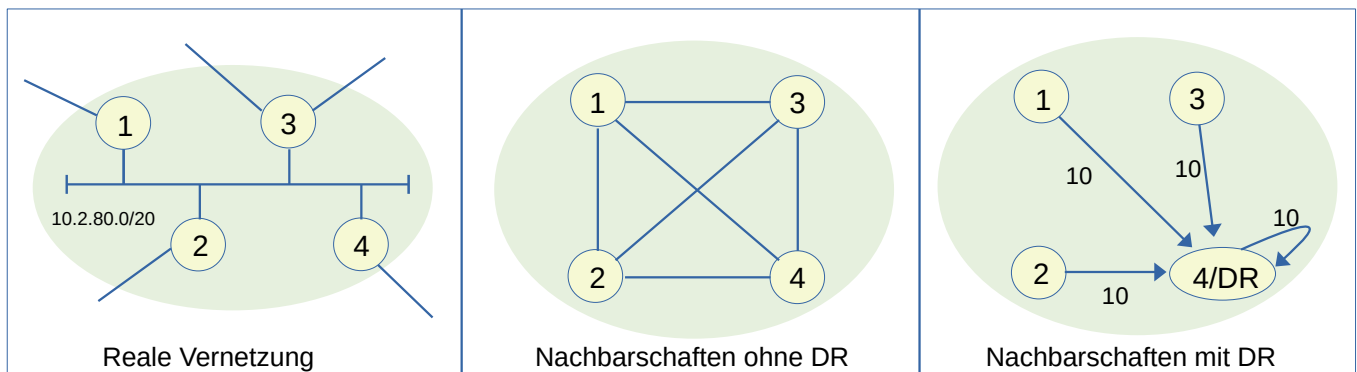


Abbildung 24

Um diese Situation bezüglich der Berechnung der Routingtabellen zu vereinfachen, wurde für Mehrfachzugriffs-Netze das Konzept des „Designated Router“ (DR) und „Backup Designated Router“ (BDR) eingeführt. Die automatische Aushandlung des DR bewertet zunächst die im Hello-Paket mitgeteilten Prioritätswerte (default 1). Sind diese Werte gleich, so gewinnt derjenige Router mit der größten Router-ID. Alle anderen Router sind nun (aus Sicht von OSPF) nur noch mit dem DR benachbart (rechter Teil aus Abbildung 24). Weiterer Vorteil: Kommt ein neuer Router hinzu, so bekommt er die bisherige Database Description gesammelt vom DR zugeschickt und muss nicht mit allen Routern einzeln diese Information austauschen.

Im Router LSA wird von allen Routern eines Mehrfachzugriffs-Netzes ein **Link-Typ 2 (Transit-Netzwerk)** an alle anderen Router (im gesamten Netz) per Flooding versendet, in dem folgende Informationen enthalten sind:

- Link ID: IP-Adresse des DR
- Link Data: eigene Interface-Adresse
- Metrik

Es entsteht jedoch ein Problem: Das Mehrfachzugriffsnetz selbst wird mit diesen „Transit-Informationen“ nicht bekannt gemacht, obwohl es als eigenständiges Netzwerk existiert und in den Routingtabellen enthalten sein sollte. Denn anders als bei den Stub-Netzwerken wird im Link-Typ 2 (Transit-Netzwerk) keine Information zur Netzwerkadresse und Maske geliefert. Zudem entsteht ein falsches Bild der Realität: Ein Datenpaket, das über Router 1 ankommt, wird direkt an den richtigen Ausgangs-Router (z.B. Router 3) gesendet und läuft nicht über den DR. Bei speziellen Netzwerk-Konfigurationen (VLAN) kann u.U. die Situation entstehen, dass zwar alle Router den DR kennen, aber untereinander gar nicht direkt kommunizieren können.

Daher benötigen Mehrfachzugriffsnetze ein zusätzliches LSA: das Network-LSA.

Network LSA

Ein Network LSA wird ausschließlich vom dem DR eines Mehrfachzugriffsnetzes per Flooding gesendet. Im Network LSA werden alle IDs der tatsächlich im Mehrfachzugriffsnetz befindlichen Router zusammengefasst und die Netzwerkmaske mitgeteilt (die IP-Adresse ist bereits im Router-LSA des DR enthalten). Damit entsteht die in Abbildung 25 dargestellte virtuelle Ansicht auf das Mehrfachzugriffsnetz, mit dem LAN selbst als Pseudo-Node.

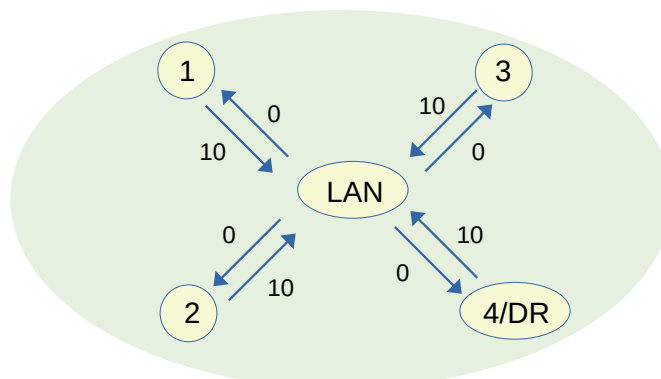


Abbildung 25

Summary LSA

Die Area Border Router (ABR) teilen der jeweils anderen Area die existierenden Netzwerk-Adressen einer Area mit (Inter-Area-Routes), einschließlich der Metrik (vom ABR aus betrachtet); es wird zwischen Routen innerhalb eines Autonomen Systems (AS) und Routen zu ASBRs unterschieden.

AS External LSA:

Autonomous System Boundary Router (ASBR) informieren über externe Netzwerkadressen (außerhalb des eigenen von OSPF verwalteten Netzwerks).

Damit verfügt jeder Router über folgende Informationen, anhand derer die Berechnung der Routing-Tabelle per Shortest Path Algorithmus erfolgt:

- Alle Router mit ID und Interface-Adressen aus der eigenen Area, einschließlich Metrik, es wird zwischen Punkt-zu-Punkt-, Transit- und Stub-Netzwerken unterschieden.
- Alle Netzwerke der eigenen Area (bei Mehrfachzugriffsnetzen identifiziert über den jeweiligen DR und Netzmaske, mit den dort angeschlossenen Routern).
- Alle Netzwerke der anderen Areas (mitgeteilt von den ABRs, einschließlich Metrik ab dem ABR, ggf. zusammengefasste Adressen).
- Externe Netzwerke (über ASBRs)

Damit verfügt jeder Router über eine vollständige Sicht auf das Netzwerk. Hinweis: trotz schlechterer Metrik werden Intra-Area-Routen gegenüber Inter-Area-Routen bevorzugt. Die Lastverteilung auf mehrere Wege (Multipathing) ist möglich, aber falls Hin- und Rückweg bzgl. der Kosten von den beteiligten Routern nicht gleich bewertet werden, kann es zu asymmetrischem Routing kommen; dies wird von Firewalls oder Routing-Software z.T. nicht akzeptiert.

7.5.5 BGP

Die bisher vorgestellten Protokolle ermöglichen nur die Kommunikation bzw. das Routing in begrenzten Bereichen, den sog. Autonomous Systems (AS), z.B. Provider-Netze. Das globale Routing kann jedoch nicht nur nach den Prinzipien eines internen Gateway Protocols (IGP) funktionieren, da es unnötig ist, dass jeder Router innerhalb eines AS alle weltweiten Routen kennt. Nur Router, die Autonome Systeme miteinander verbinden, müssen alle Netze kennen. Zudem gibt es nicht-technische Routing-Kriterien, z.B. Leitungskosten oder politisch unerwünschte Verbindungen. Hierzu werden externe Gateway Protokolle (EGP) eingesetzt: das aktuelle Protokoll für das „Interdomain Routing“ im Internet ist das Border Gateway Protocol BGP.

BGP ist ein Distance Vector Protokoll, das jedoch die Nachteile von z.B. RIP vermeidet: die BGP-Routing-Tabelle enthält nicht nur Nachbar-Informationen sondern den vollständigen Pfad. Dieser Pfad besteht aber *nur* aus den AS-Nummern, interne Strukturen oder Details innerhalb eines AS werden nicht berücksichtigt (hierfür ist das jeweilige IGP des AS zuständig), siehe Abbildung 26. Die AS-Nummern müssen weltweit eindeutig sein und werden daher von der IANA vergeben (AS-Nummern-Bereich: 0 – 65535, wobei der private Bereich 64512 - 65535 nicht zum globalen Routing verwendet wird).

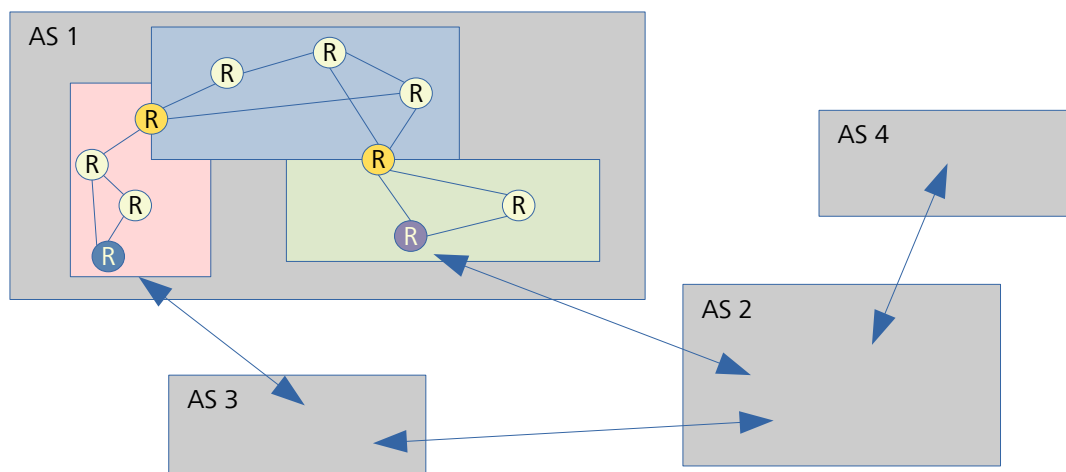


Abbildung 26: Routing in Autonomen Systemen

8 Routing-Algorithmen

8.1 Dijkstra-Algorithmus

Der Dijkstra-Algorithmus berechnet (ausgehend von einem der Knoten) die kürzesten Pfade in einem ungerichteten Graphen zu allen anderen Knoten. Die Kanten des Graphen können unterschiedliche Gewichtungen (Metriken) haben, die jedoch nicht negativ sein dürfen.

Ausgehend vom Startknoten werden **alle** direkten Wege zu den Nachbar-Knoten betrachtet und die Metriken (Kosten) vorläufig in einem Array gespeichert. Da der direkte Weg nicht zwangsläufig der kürzeste sein muss, können diese Zwischenergebnisse im Laufe des Algorithmus noch verbessert werden. Aus den zunächst vorliegenden Ergebnissen wird der aktuell kürzeste Pfad ausgewählt. Dieser Pfad ist bereits der kürzest mögliche zu diesem Nachbarn, da alle anderen Alternativpfade betrachtet wurden und somit auf jeden Fall länger sind, als der direkte Weg. Dieser Pfad kann daher als „endgültig berechnet“ markiert werden und braucht bei den weiteren Berechnungen nicht mehr berücksichtigt zu werden.

Abbildung 27 zeigt einen ungerichteten Graphen, in dem ausgehend von Knoten A die kürzesten Wege zu allen anderen Knoten berechnet werden sollen.

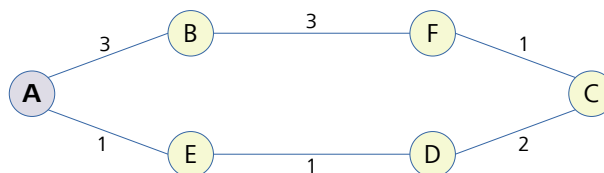


Abbildung 27: Dijkstra-Algorithmus, ungerichteter Graph

Der erste Schritt des Algorithmus beginnt bei Knoten A: es werden die beiden Wege zu den Nachbarn B und E betrachtet und die Metriken in einem vorläufigen Array gespeichert, hier also Weg zu B in Metrik 3 über Vorgänger A und Weg zu E in Metrik 1 über Vorgänger A:

Beginn bei A: B(3,A) und E(1,A)

Da alle Wege ausgehend von A betrachtet wurden, ist der Weg zu E in der Metrik 1 der kürzeste zu E, denn die einzige Alternative über B ist auf jeden Fall länger; der Weg zu E in der Metrik 1 kann daher als „endgültig berechnet“ markiert werden. Der Weg zu B kann noch nicht endgültig beurteilt werden, da der Alternativ-Pfad über E eine kürzere Gesamt-Metrik ergeben könnte.

Im zweiten Schritt wird derjenige Knoten als Ausgangspunkt gewählt, der aktuell den kürzesten Pfad besitzt, in diesem Fall Knoten E (einziger Nachbar außer dem Vorgänger A ist Knoten D, der in der Gesamt-Metrik $1+1=2$ erreicht wird):

2. Schritt ab E: D(2,E)

Nun betrachtet man alle Zwischenergebnisse, die noch nicht „endgültig berechnet“ sind (bzw. noch nicht „endgültig gestrichen“ sind). Dies betrifft hier die Ergebnisse B(3,A) und D(2,E), sodass Knoten D als Ausgangspunkt für den 3. Schritt verwendet wird; D(2,E) kann aus dem zuvor genannten Grund als „endgültig berechnet“ markiert werden.

3. Schritt ab D: C(4,D)

Wieder werden alle noch nicht endgültig berechneten Wege betrachtet (hier B(3,A) und C(4,D)), dann der kürzere ausgewählt, als „endgültig berechnet“ markiert und als Ausgangspunkt für den 4. Schritt verwendet.

4. Schritt ab B: F(6,B)

Aus den verbliebenen Wegen C(4,D) und F(6,B) ist der erstere der kürzere, sodass sich für den 5. Schritt ergibt:

5. Schritt ab C: F(5,C)

Es verbleiben die beiden Wege zu F: F(6,B) und F(5,C). Der Weg über C aus dem 5. Schritt kann als „endgültig berechnet“ markiert werden und der Weg aus dem 4. Schritt über B kann gestrichen werden, da er länger ist als der Weg über C. Damit sind alle Zwischenergebnisse entweder als „endgültig berechnet“ oder als „endgültig

gestrichen“ markiert, womit der Algorithmus sein Ende erreicht hat.

Man erkennt an diesem Beispiel das Grundprinzip des Dijkstra-Algorithmus: Man folgt einem Pfad solange, bis man erkennt, dass er länger wird als ein zuvor beschrittener Pfad. In diesem Beispiel folgt man $A \rightarrow E \rightarrow D$, erkennt dann, dass der folgende Weg zu C länger wäre als der zuvor berechnete Pfad $A \rightarrow B$. Man springt zurück und folgt nun dem Pfad $A \rightarrow B \rightarrow F$.

8.2 Bellman-Ford-Algorithmus

Der Bellman-Ford-Algorithmus kann im Gegensatz zum Dijkstra-Algorithmus auch kürzeste Wege in einem *gerichteten* Graphen berechnen, bei dem die Kanten in beiden Richtungen unterschiedliche Metriken (Gewichtungen) haben können, und außerdem können auch negative Gewichtungen vorhanden sein (es dürfen jedoch keine negativen Schleifen entstehen s.u.). Bellman-Ford berechnet ausgehend von jeder Kante des Graphen die Distanz vom Anfangsknoten zum Endknoten dieser Kante. Im obigen Graphen (Abbildung 27) wird beispielsweise die Kante von B zu F genommen und die Distanz zu F berechnet, indem die aktuell berechnete Distanz zu B (ausgehend vom Startknoten A) zur Metrik der Kante hinzuaddiert wird. Bei dieser Vorgehensweise ist es fast unvermeidlich, dass sich im Laufe der Berechnungen eine bereits berechnete Distanz verbessert, sodass solche Zwischenrechnungen wiederholt werden müssen, die zuvor mit einem schlechteren Wert berechnet wurden. Auch kann es vorkommen, dass ein zu addierender Distanzwert noch gar nicht bestimmt wurde und daher noch seinen initialen Wert (unendlich) hat.

Abbildung 28 zeigt anhand eines Pseudo-Code den Ablauf des Algorithmus. Er läuft über mehrere „Runden“ (die äußere for-Schleife über i), da am Ende jeder Runde die Situation aufgetreten sein kann, dass sich ein zuvor verwendeter Distanzwert im Laufe der Runde verbessert hat. Alle Berechnungen, die zuvor mit dem zu großen Distanzwert ausgeführt wurden, müssen also wiederholt werden. Im Idealfall („best case“) hat der Algorithmus schon nach der ersten Runde das endgültige Ergebnis bestimmt, im worst case müssen so viele Runden gerechnet werden wie Knoten vorhanden sind (minus Eins). Die Zahl der erforderlichen Runden ist abhängig von der gegebenen Reihenfolge der Kanten; man könnte also die Kanten auf die beste mögliche Weise sortieren und würde dann immer nur eine Runde im Bellman-Ford-Algorithmus benötigen. Allerdings ist die „geeignete Vorsortierung“ des Kanten-Arrays aufwändiger, als den Algorithmus vollständig mit allen Runden zu berechnen.

Das Array M der Kanten besteht aus den Gewichtungen (Metriken) aller Kanten $M(u, v)$, wobei u den Anfangsknoten und v den Endknoten einer Kante bezeichnet. Der Distanzwert $d(v)$ zum Endpunkt der Kante berechnet sich aus der Metrik $M(u, v)$ der Kante und dem Distanzwert des Anfangsknotens $d(u)$

$$d(v) = d(u) + M(u, v)$$

Die Distanzwerte werden zu Beginn des Algorithmus mit unendlich initialisiert, mit Ausnahme des Start-Knotens, von dem aus die kürzesten Wege zu allen anderen Knoten berechnet werden sollen, er hat zu sich selbst die Distanz Null: $d(\text{Start}) = 0$ und $d(u) = \infty$ für alle anderen Distanzen.

Die erste for-each-Schleife nimmt alle Metrikwerte $M(u, v)$ aus dem Array M in der gegebenen Reihenfolge und berechnet jeweils den Distanzwert $d(v)$ des Endknotens; sofern dieser Wert kleiner ist als der bisherige Wert von $d(v)$, so wird $d(v)$ mit diesem Wert überschrieben. Als „Vorgänger“ wird der Anfangsknoten u der Kante $M(u, v)$ gespeichert. Die for-Schleife kann vorzeitig beendet werden, wenn sich keine Verbesserungen im Vergleich zu vorherigen Runde mehr ergeben. Zum Schluss (zweite for-each-Schleife) muss noch geprüft werden, ob sich negative Schleifen ergeben: u.U. kann es Situationen in einem Graphen geben, in denen Distanzwerte aufgrund negativer Metriken von Runde zu Runde immer kleiner werden. Da der Algorithmus maximal so viele Runden durchlaufen kann wie Knoten (minus Eins) vorhanden sind, muss *nach der letzten* möglichen Runde nochmals getestet werden, ob sich erneut kleinere Distanzwerte ergeben. Ist dies der Fall, so liegt eine negative Schleife vor und der Algorithmus muss ohne Ergebnis abgebrochen werden.

```

for 1 ≤ i ≤ (Anzahl der Knoten) - 1
    for each M(u,v) do
        if distanz(u) + M(u,v) < distanz(v)
            then
                distanz(v) = distanz(u) + M(u,v)
                vorgänger(v) = u

for each M(u,v) do    wenn nach der maximalen Zahl an Runden i immer noch eine Verbesserung
    if distanz(u) + M(u,v) < distanz(v)    festgestellt wird, handelt es sich um eine negative
        then „Fehler: negative Schleife“    Schleife und der Algorithmus funktioniert nicht

return distanz, vorgänger

```

Abbildung 28: Bellman-Ford, Pseudo Code

Ein erstes Beispiel (Abbildung 29) zum Bellman-Ford-Algorithmus zeigt zur Vereinfachung zunächst nochmals den ungerichteten Graphen aus dem vorherigen Kapitel

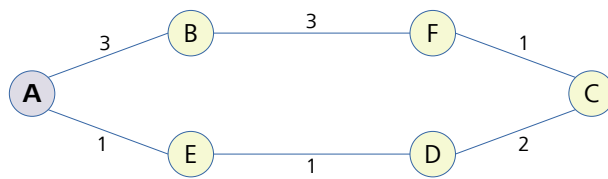


Abbildung 29: Bellman-Ford, ungerichteter Graph

M(u,v):	Distanzwerte:
M(A,B) = 3	d(A) = 0
M(A,E) = 1	d(v) = ∞ für alle
M(B,A) = 3	anderen Knoten v
M(B,F) = 3	
M(C,D) = 2	
M(C,F) = 1	
M(D,C) = 2	
M(D,E) = 1	
M(E,A) = 1	
M(E,D) = 1	
M(F,B) = 3	
M(F,C) = 1	

Ausgehend von Knoten A, soll wieder der kürzeste Weg zu allen anderen Knoten berechnet werden. Die Reihenfolge von M ist hier alphabetisch nach den Anfangsknoten u sortiert, sodass in diesem Beispiel zunächst alle Kanten von Knoten A betrachtet werden, dann alle Kanten von Knoten B usw. Bei einer anderen (beliebigen) Reihenfolge von M können sich mehr oder weniger Runden ergeben.

Runde 1	Runde 2	Runde 3
Alle Kanten von A: $u=A$, $v=B$ und $v=E$		
$d(B) = d(A) + M(A,B) = 0 + 3 = \mathbf{3}$		
$d(E) = d(A) + M(A,E) = 0 + 1 = \mathbf{1}$		
Alle Kanten von B: $u=B$, $v=A$ und $v=F$		
$d(A) = d(B) + M(B,A) = 3 + 3 = \mathbf{6}$ (gestrichen)		
$d(F) = d(B) + M(B,F) = 3 + 3 = \mathbf{6}$		
Alle Kanten von C: $u=C$, $v=D$ und $v=F$	Alle Kanten von C:	Alle Kanten von C:
$d(D) = d(C) + M(C,D) = \infty + 2 = \infty$	$d(D) = d(C) + M(C,D) = 7 + 2 = \mathbf{9}$ (gestrichen)	$d(D) = d(C) + M(C,D) = 4 + 2 = \mathbf{6}$ (gestrichen)
$d(F) = d(C) + M(C,F) = \infty + 1 = \infty$ (gestrichen)	$d(F) = d(C) + M(C,F) = 7 + 1 = \mathbf{8}$ (gestrichen)	$d(F) = d(C) + M(C,F) = 5 + 1 = \mathbf{5}$ (besser)
Alle Kanten von D: $u=D$, $v=C$ und $v=E$	Alle Kanten von D:	
$d(C) = d(D) + M(D,C) = \infty + 2 = \infty$	$d(C) = d(D) + M(D,C) = 2 + 2 = \mathbf{4}$ (besser)	
$d(E) = d(D) + M(D,E) = \infty + 1 = \infty$	$d(E) = d(D) + M(D,E) = 2 + 1 = \mathbf{3}$ (gestrichen)	
Alle Kanten von E: $u=E$, $v=A$ und $v=D$		
$d(A) = d(E) + M(E,A) = 1 + 1 = \mathbf{2}$ (gestrichen)		
$d(D) = d(E) + M(E,D) = 1 + 1 = \mathbf{2}$ (besser)		
Alle Kanten von F: $u=F$, $v=B$ und $v=C$		Alle Kanten von F:
$d(B) = d(F) + M(F,B) = 6 + 3 = \mathbf{9}$ (gestrichen)		$d(B) = d(F) + M(F,B) = 5 + 3 = \mathbf{8}$ (gestrichen)
$d(C) = d(F) + M(F,C) = 6 + 1 = \mathbf{7}$ (besser)		$d(C) = d(F) + M(F,C) = 5 + 1 = \mathbf{6}$ (gestrichen)

Aus Gründen der besseren Übersichtlichkeit wurde hier auf die Darstellung der Vorgänger verzichtet und auch die letzte for-each-Schleife nicht berücksichtigt, da keine negativen Metriken existieren.

Man erkennt, dass in Runde 1 zunächst mit Werten $d(C)=\infty$ und $d(D)=\infty$ gerechnet wurde, sich *danach* aber noch bessere Werte $d(C)=7$ und $d(D)=2$ ergaben. Daher muss in einer zweiten Runde mit diesen verbesserten Werten neu gerechnet werden (auch hier sind aus Gründen der Übersichtlichkeit nur die relevanten Berechnungen dargestellt). Für die Distanz $d(C)$ ergibt sich mit $d(C)=4$ erneut ein besserer Wert als der, mit dem zuvor gerechnet wurde, sodass eine dritte Runde erforderlich ist: aufgrund des verbesserten Wertes von $d(C)$ ergibt sich ein besserer Wert für $d(F)=5$. Da dieser Wert aber in derselben Runde erst später berücksichtigt wird, braucht keine vierte Runde berechnet zu werden (da sich keine weiteren Verbesserungen ergeben).

Das Endergebnis steht damit fest: $d(A)=0$, $d(B)=3$, $d(C)=4$, $d(D)=2$, $d(E)=1$, $d(F)=5$

9 Algorithmen und Datenstrukturen

9.1 Feld/Array und verkettete Liste

In einem Array (Feld) werden Elemente (Variablen) eines bestimmten Typs gespeichert. Anzahl und Typ der Elemente und die Gesamtgröße des Feldes werden z.B. beim Initialisieren eines Programms festgelegt, soweit möglich wird ein zusammenhängender Speicherbereich dafür reserviert. In einer solchen Datenstruktur ist z.B. das Vertauschen von Elementen durch einfaches Umkopieren der Speicherplätze realisierbar, auch das Einfügen neuer Elemente ist einfach, indem an die letzte freie Stelle das neue Element eingesetzt wird. Beim Löschen wird der betreffende Speicherplatz geleert und durch das letzte Element ersetzt, damit keine Lücken entstehen, die beim sequentiellen Durchsuchen störend wären bzw. die Suche länger machen als nötig. Außerdem ist jedes Element $A(i)$ direkt über seinen Index im Speicherbereich ansprechbar.

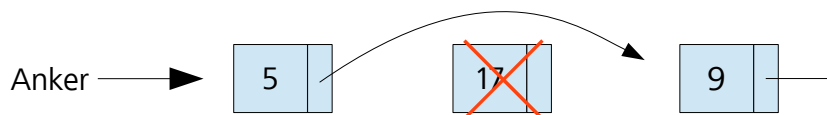
Von Nachteil ist jedoch, dass die Datenstruktur „Array/Feld“ sehr starr ist und zudem die Elemente beim Löschen oder Sortieren von einem Speicherplatz auf einen anderen kopiert werden müssen. Ist ein Array vollständig belegt und soll erweitert werden, so muss zunächst der physikalische Speicherbereich vergrößert werden oder falls dies nicht möglich ist ein neuer zusammenhängender Bereich gefunden werden; in diesem Fall müssen alle Elemente vom alten Speicherplatz in den neuen umkopiert werden (sog. dynamische Datenstruktur).

Eine alternative Datenstruktur stellen sog. verkettete Listen dar. Hierbei wird das erste Element an irgendeiner beliebigen Speicherposition abgelegt (die genügend Platz bietet); außer den eigentlichen Daten enthält das Element zusätzlich einen Zeiger, der auf den Speicherplatz des zweiten Elements verweist. Um das erste Element zu finden, benötigt die verkettete Liste einen Anker, der nur aus dem Zeiger auf das erste Element besteht. Daher wird für die Liste kein zusammenhängender Bereich im Speicher benötigt; auch in einem gut gefüllten und stark fragmentierten Speicher lassen sich verkettete Listen noch problemlos anlegen. Der insgesamt benötigte Speicherplatz ist im Vergleich zum Array größer, da zusätzlich die Information des Zeigers in jedem Element enthalten sein muss.



Diese Liste besteht aus einem Anker, der einen Zeiger auf das erste Element mit dem Schlüssel 5 enthält. Dieses Element hat einen Zeiger, der auf das zweite Element mit dem Schlüssel 17 verweist usw. Das letzte Element enthält formal auch einen Zeiger, in diesem Fall einen Null-Zeiger.

Das Einfügen und Löschen ist einfach realisierbar, indem nur entsprechende Zeiger verändert werden; es muss nichts umkopiert werden. Beim Löschen wird das zu löschende Element gesucht und der Zeiger dieses Elementes wird an die Stelle des Zeigers des Vorgängers gesetzt:



Beim Einfügen wird das neue Element an den Anfang gesetzt, sodass lediglich der bisherige Zeiger des Ankers als Zeiger für das neue Element verwendet werden muss und der Anker auf das neue Element verweist.



Der bloße Aufruf eines Elementes i in einer verketteten Liste ist jedoch aufwändig, da immer nur schrittweise (iterativ) von einem Element zum nächsten gesprungen werden kann. Ein direkter Zugriff auf ein Element $A(i)$ wie beim Array ist prinzipbedingt nicht möglich. Die Laufzeit für den Aufruf eines Elementes ist im ungünstigsten Fall (worst case) $O(n)$ bei einer Liste mit n Elementen. Man springt vom Anker zum ersten Element, prüft ob der Schlüssel i mit dem gesuchten übereinstimmt (Vergleichsoperation), wenn ja: Rückgabe der Daten des Speicherplatzes, wenn nein: weiter zum nächsten Speicherplatz mithilfe des Zeigers.

Man erkennt, dass die verkettete Liste nur einen wesentlichen Vorteil im Vergleich zum Array bietet: die hohe Flexibilität (leicht erweiterbar und einfaches Einfügen, Löschen oder Umsortieren nur durch Änderung der Verzeigerung, ohne Umkopieren der Daten).

9.2 Laufzeitabschätzung

In der Informatik spielt die Abschätzung der Laufzeit bei Algorithmen eine wichtige Rolle. Hierbei kommt es nicht auf das exakte Abzählen von Berechnungsschritten an, was bei komplexeren Algorithmen bei unterschiedlichen Startparametern ohnehin kaum möglich wäre (z.B. kann die Zahl von Schleifendurchläufen in einem Programm von vielen schwer abschätzbaren Variablen abhängen); außerdem ließe sich selbst dann kaum eine exakte Aussage über die tatsächliche Laufzeit eines Programms treffen (die abhängig von einer realen Hardware ist). Wichtiger ist stattdessen eine Aussage über die sog. asymptotische Laufzeitkomplexität bzgl. einer zunehmenden Problemgröße n . Die Frage ist: Wird ein Algorithmus/Programm in einer begrenzten, in der Praxis brauchbaren Zeit beendet sein, auch wenn die Problemgröße zunimmt? Oder umgekehrt: Welche Größenordnung der Laufzeit ist (im Best Case) mindestens zu erwarten? Für diese Abschätzungen bzw. die Einordnung in Funktionsklassen dienen die sog.

Landau-Symbole O (Groß-Oh), Ω (Omega) und Θ (Theta).

Wächst eine Funktion $f(n)$ nicht wesentlich schneller als eine Funktion $g(n)$, so schreibt man:

$$f(n) \in O(g(n)) \quad \text{mit} \quad f(n) \leq c \cdot g(n) \quad \text{mit } c \text{ als konstantem Faktor } c \in \mathbb{R}$$

Man kann $g(n)$ als eine grobe Abschätzung nach oben betrachten. Umgekehrt als Abschätzung nach unten:

$$f(n) \in \Omega(g(n)) \quad \text{mit} \quad f(n) \geq c \cdot g(n)$$

und als Schnittmenge von beiden:

$$\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$$

Als „Vergleichsfunktion $g(n)$ “ werden häufig folgende Funktionen verwendet: 2^n , n^2 , $n \cdot \log n$, n , $\sqrt{n}$

Ein Algorithmus mit exponentieller Laufzeitkomplexität ist bzgl. seiner praktischen Verwendbarkeit problematisch, ein (nur langsam) logarithmisch mit der Problemgröße wachsender Algorithmus ist sicherlich wünschenswert.

Die Symbole O und Ω sind komplementär zueinander, d.h. wenn $f(n) \in O(g(n))$ so ist $g(n) \in \Omega(f(n))$

Und: $f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$

In einer Abschätzung nach oben kann in einer Summe die schwächere Funktion weggelassen werden, bzw. in einer Abschätzung nach unten die stärkere:

$$f(n) \in O(g(n)) \implies O(f(n)+g(n)) = O(g(n))$$

$$f(n) \in \Omega(g(n)) \implies \Omega(f(n)+g(n)) = \Omega(g(n))$$

z.B. $f(n) = n$ und $g(n) = n \cdot \log n$ so gilt $O(n + n \cdot \log n) = O(n \cdot \log n)$

9.3 Sortieren

9.3.1 Sortieren durch Auswahl

Oftmals ist es sinnvoll, die Elemente in einem Array oder einer Liste nach bestimmten Kriterien zu sortieren, um die Suche nach einem Schlüssel zu beschleunigen. Es ist von der jeweiligen Anwendung abhängig, welche Kriterien hierfür sinnvoll verwendet werden können (z.B. ist ein Telefonbuch nach dem Schlüssel „Nachnamen“ sortiert, jedoch nicht nach Telefonnummern).

Gegeben sei ein Array A mit N Elementen $A(1), A(2), \dots, A(N)$. Diese Elemente haben einen Schlüssel und einen oder mehrere Werte (z.B. Matrikel-Nummer und einen Datensatz mit den Daten des/der Studierenden). Das Array A soll nun mittels des „Sortierens durch Auswahl“ nach aufsteigenden Schlüsseln sortiert werden. Man suche hierfür in $A(1)$ bis $A(N)$ nach demjenigen Element $A(i)$ mit dem kleinsten Schlüssel und vertausche $A(1)$ mit $A(i)$. Dann suche man in $A(2)$ bis $A(N)$ nach dem Element $A(j)$ mit dem kleinsten Schlüssel und vertausche $A(2)$ mit $A(j)$ und setze fort bis alle Elemente an der gewünschten Position stehen.

Beispiel:

Position k	1	2	3	4	5	6
$A(k)$, Schlüssel	35	11	4	21	3	17
1. Schritt	3	11	4	21	35	17
2. Schritt	3	4	11	21	35	17
3. Schritt	3	4	11	21	35	17
4. Schritt	3	4	11	17	35	21
5. Schritt	3	4	11	17	21	35

Im Ausgangs-Array (grau hinterlegt) wird das kleinste Element (hier $A(5)=3$) gesucht und mit dem ersten Element $A(1)$ (hier 35) vertauscht; das Ergebnis dieses ersten Schrittes steht in der Zeile „1. Schritt“. In dieser Zeile wird nun ab dem zweiten Element $A(2)=11$ wiederum nach dem kleinsten Schlüssel gesucht (hier $A(3)=4$) und $A(2)$ mit $A(3)$ vertauscht (Ergebnis in der Zeile „2. Schritt“). Und weiter: Suche in dieser Zeile ab $A(3)$ nach dem kleinsten Element, dieses ist $A(3)$ selbst, sodass keine Vertauschung erforderlich ist. Setze den Algorithmus fort, bis das letzte Element erreicht ist.

Die Laufzeit dieses Sortier-Algorithmus: Im ersten Durchlauf müssen offensichtlich $n-1$ Vergleiche durchgeführt werden, im zweiten Durchlauf einer weniger, im dritten noch ein Vergleich weniger usw. In der Summe:

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \in O(n^2)$$

Der Algorithmus hat also eine Laufzeit in $O(n^2)$

9.3.2 Sortieren durch Einfügen

Beim „Sortieren durch Einfügen“ wird ein Element $A(i)$ beginnend mit $i=1$ mit den Schlüsseln eines zuvor bereits (vorläufig) sortierten Bereiches verglichen, an der richtigen Position eingefügt und alle weiteren Elemente des vorläufig sortierten Bereiches verschoben.

Beispiel (Ausgangs-Array wie zuvor)

Position k	1	2	3	4	5	6
$A(k)$, Schlüssel	35	11	4	21	3	17
1. Schritt	11	35	4	21	3	17
2. Schritt	4	11	35	21	3	17
3. Schritt	4	11	21	35	3	17
4. Schritt	3	4	11	21	35	17
5. Schritt	3	4	11	17	21	35

Im Ausgangs-Array wird das Element $A(1)=35$ als vorläufig sortierter Bereich betrachtet. Nun wird das Element $A(2)=11$ mit dem einzigen Element dieses vorläufig sortierten Bereiches verglichen und an die richtige Stelle einsortiert (wegen $11 < 35$ vor das Element $A(1)=35$). Das Element $A(1)=35$ muss nun um eine Position nach rechts verschoben werden, alle anderen Elemente verbleiben zunächst an ihren Plätzen. Es muss hierfür ein Vergleich und eine Verschiebung durchgeführt werden. Jetzt wird in der Zeile „1. Schritt“ $A(3)=4$ eingelesen und schrittweise mit den Elementen des vorläufig sortierten Bereiches (links, grün hinterlegt) verglichen, so lange bis ein größerer Schlüssel gefunden ist. In diesem Schritt ist dies bereits beim linken Element $A(1)=11$ der Fall, sodass das Element $A(3)=4$ nun an die Stelle des Elements $A(1)=11$ gesetzt werden kann. Alle anderen Elemente rechts davon (also hier die 11 und die 35) werden um eine Position nach rechts verschoben; das Ergebnis ist die Zeile „2. Schritt“. Der Aufwand beträgt 1 Vergleich und 2 Verschiebungen. Weiter geht es mit $A(4)=21$ (Zeile „2. Schritt“): Es werden 3 Vergleiche benötigt bis $21 < 35$ gefunden ist, dann 1 Verschiebung (nur die 35), Ergebnis ist die Zeile „3. Schritt“. Das Ergebnis in der Zeile „4. Schritt“ erfordert 1 Vergleich und 4 Verschiebungen und die letzte Zeile erhält man mit 4 Vergleichen und 2 Verschiebungen.

Laufzeit: Im ungünstigsten Fall (Worst Case) muss ein eingelesenes Element $A(i)$ mit allen Schlüsseln des vorläufig sortierten Bereiches verglichen werden. Die Laufzeit ist also $O(n^2)$, genauso wie beim „Sortieren durch Auswahl“. Im Best Case ist $A(i)$ immer der kleinste Schlüssel und es muss lediglich ein einziger Vergleich mit dem ersten Element des vorläufig sortierten Bereiches durchgeführt werden, also Laufzeit $O(n)$.

9.3.3 Bubble-Sort

Ein weiterer einfacher Sortier-Algorithmus ist Bubble-Sort, der sowohl in Arrays als auch Listen gleich schnell funktioniert, da ohnehin alle Elemente der Reihenfolge nach abgearbeitet werden müssen. Bubble-Sort vergleicht jeweils benachbarte Elemente und vertauscht die Elemente, wenn der Schlüssel des linken Elementes größer als der des rechten Elementes ist. Elemente mit größeren Schlüsseln werden also von links nach rechts „durchgeschoben“.

Beispiel:

Array/Liste	8	21	47	32	17	1	25	7	14
1. Durchlauf	8	21	32	17	1	25	7	14	47
2. Durchlauf	8	21	17	1	25	7	14	32	47
3. Durchlauf	8	17	1	21	7	14	25	32	47
4. Durchlauf	8	1	17	7	14	21	25	32	47
5. Durchlauf	1	8	7	14	17	21	25	32	47
6. Durchlauf	1	7	8	14	17	21	25	32	47

Aktionen im Ausgangs-Array:

- $8 > 21$, false: Elemente bleiben an ihren Positionen
- $21 > 47$, false: Elemente bleiben an ihren Positionen
- $47 > 32$, true: Elemente vertauschen, jetzt steht die 47 auf der Position der 32, sodass nun die 47 mit der 17 benachbart ist
- $47 > 17$, true: Elemente vertauschen, die 47 jetzt auf der Position der 17, nächster Vergleich daher
- $47 > 1$, true: Elemente vertauschen usw.
- Am Ende steht als Ergebnis die Zeile „1. Durchlauf“

Aktionen in der Zeile „1. Durchlauf“:

- $8 > 21$, false: Elemente bleiben an ihren Positionen
- $21 > 32$, false: Elemente bleiben an ihren Positionen
- $32 > 17$, true: Elemente vertauschen, ab jetzt wird die 32 bis zur vorletzten Position „durchgereicht“

Aktionen in der Zeile „2. Durchlauf“, zusammengefasst: die 21 wird bis zur 25 durchgeschoben und dann die 25 bis zur drittletzten Position. Allerdings können die Vergleiche an dieser Stelle nicht einfach beendet werden, da man sich sonst ja merken müsste, ab welcher Position im vorherigen Durchlauf die Liste bereits sortiert ist.

Der Algorithmus ist erfolgreich beendet, wenn keine Vertauschungen mehr erforderlich sind.

Betrachtet man (im Hinblick auf die Laufzeit) den einfachsten Fall, bei dem die Liste zu Beginn des Algorithmus bereits sortiert vorliegt, so müssen alle n Elemente einmal „angefasst“ und verglichen werden, Schreibzugriffe (also das Kopieren) sind nicht erforderlich. Der Algorithmus ist nach n Vergleichen beendet (eigentlich $n-1$, aber man muss noch testen, ob das Ende des Feldes erreicht ist). Im ungünstigsten Fall ist die Liste umgekehrt sortiert, sodass in jedem Durchlauf (n Vergleiche) alle Elemente verglichen werden, dann aber auch n Durchläufe benötigt werden bis alles richtig sortiert ist. Die Laufzeit ist also in $O(n^2)$.

4	3	2	1
---	---	---	---

Umgekehrt sortiertes Feld

3	2	1	4
---	---	---	---

4 Vergleiche, 6 Schreibzugriffe

2	1	3	4
---	---	---	---

4 Vergleiche, 4 Schreibzugriffe

1	2	3	4
---	---	---	---

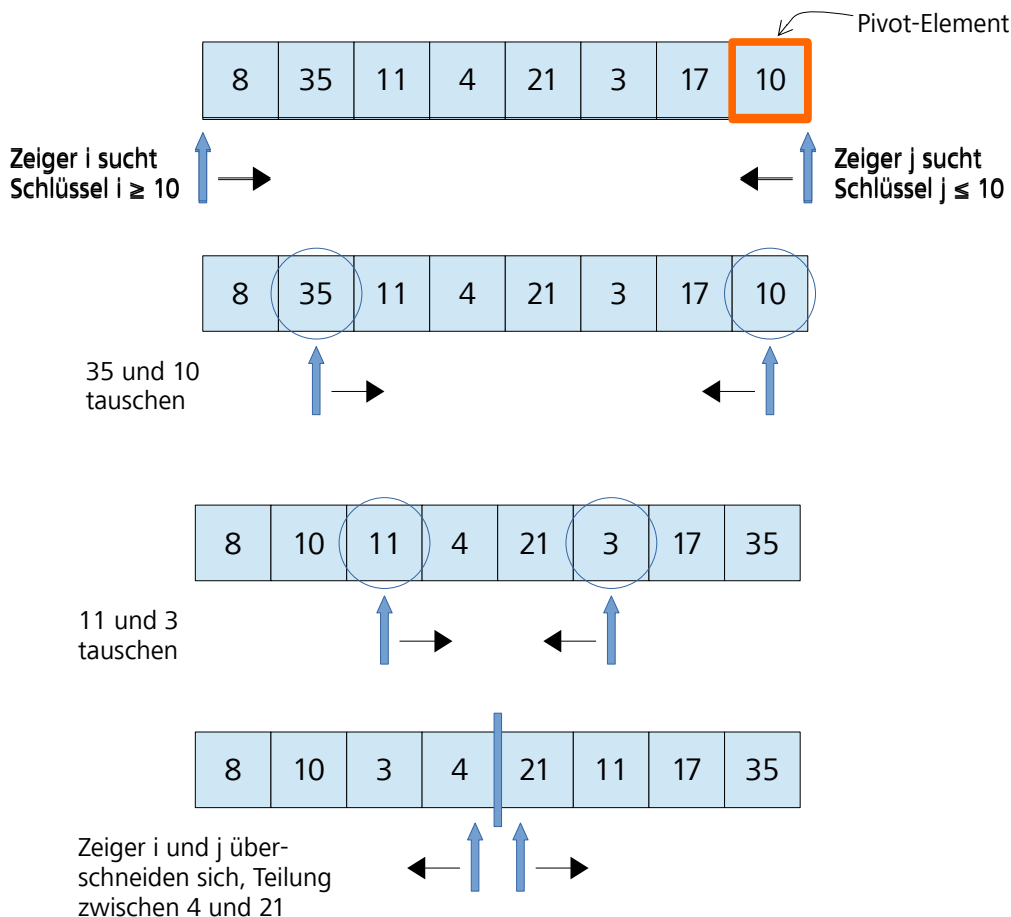
4 Vergleiche, 2 Schreibzugriffe, zum Abschluss nochmals 4 Vergleiche

9.3.4 Quicksort (Sortieren durch rekursives Teilen)

Die grundsätzliche Idee beim „Quick-Sort“ ist: Man nehme aus einer Folge $A(1)$ bis $A(N)$ ein beliebiges Element heraus (dies wird als „Pivot-Element“ bezeichnet) und teile anhand dieses Schlüssels die Folge in zwei vorsortierte Teilfolgen. In diesen Teilfolgen wird wieder jeweils ein beliebiges Element als Pivot-Element herausgegriffen, anhand dessen diese Teilfolgen wiederum in vorsortierte Teilfolgen geteilt werden usw.

Im folgenden Beispiel wird das rechte Element $A(8)=10$ als Pivot-Element verwendet. Nun geht man von rechts

nach links mit einem Zeiger j über alle Elemente und stoppe beim ersten Element, dessen Schlüssel kleiner oder gleich dem Pivot-Element ist (hier $j \leq 10$). Entsprechend gehe man von links aus nach rechts mit einem Zeiger i über alle Elemente, so lange bis ein Element größer oder gleich dem Pivot-Element gefunden ist (hier $i \geq 10$). Die beiden Elemente, bei denen die Zeiger zuerst stoppen werden miteinander vertauscht. Dann bewegt man die Zeiger weiter bis wieder zwei Element kleiner/gleich bzw. größer/gleich 10 gefunden und getauscht sind. Dies mache man so lange weiter bis sich die Zeiger überschneiden. An dieser Stelle wird nun eine Teilung der Folge vorgenommen. Beide Teilfolgen sind zwar in sich immer noch unsortiert, aber die linke Teilfolge enthält nun nur noch Schlüssel, die kleiner als die der rechten Teilfolge sind. Für beide Teilfolgen beginnt nun das genannte Verfahren von vorne (daher auch die Bezeichnung „rekursives Teilen“).



Der weitere Verlauf des Algorithmus „in Worten“:

Linke Teilfolge (8, 10, 3, 4): Man nehme bspw. die 4 als Pivot-Element, Zeiger j stoppt bei der 4, Zeiger i stoppt bei der 8, beide Werte tauschen, entsprechend die 10 und die 3 tauschen; Zeiger überschneiden sich zwischen der 3 und der 10, sodass die Teilfolge in zwei weitere Teilfolgen (4, 3) und (10, 8) geteilt wird. Man nehme wieder die jeweils rechten Werte 3 bzw. 8 als Pivot-Elemente, die Zeiger bleiben bei 4 und 3 bzw. bei 10 und 8 stehen, die Elemente werden getauscht, der Algorithmus ist bzgl. der linken Hälfte der obigen Gesamtfolge beendet.

Rechte Teilfolge (21, 11, 17, 35): Man nehme bspw. wieder das rechte Element (35) als Pivot-Element; beide Zeiger treffen sich nun bei der 35, es wird nichts getauscht, dieses Element gehört zu keiner Teilfolge sondern bleibt für sich an dieser Position stehen. Es bleibt die Teilfolge (21, 11, 17) zurück. Wieder rechtes Element (17) als Pivot-Element, 17 und 21 werden getauscht, dann bleibt Zeiger j bei 11 stehen und Zeiger i überholt ihn „auf dem Weg zur 21“, daher wird zwischen 11 und 21 geteilt. Nun noch die letzte verbliebene Teilfolge (17, 11) entsprechend bearbeiten.

Ein Beispiel, bei dem das Pivot-Element an seiner ursprünglichen Position stehen bleibt:

Folge (2, 1, 3, 17, 5) man nehme die 3 als Pivot-Element, die Zeiger treffen sich bei der 3 ohne zuvor Elemente getauscht zu haben, die 3 bleibt stehen und gehört nun zu keiner Teilfolge mehr, es bleiben die beiden Teilfolgen (2, 1) und (17, 5), die wie zuvor mittels der Pivot-Elemente 1 bzw. 5 sortiert werden können.

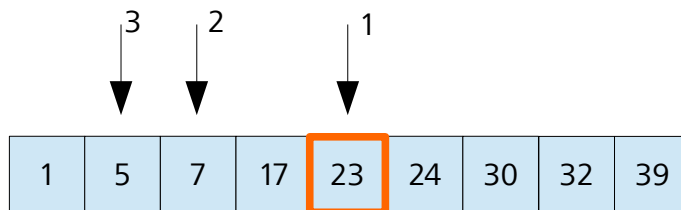
Die Laufzeit ist in (ohne Herleitung) $O(n \cdot \log n)$

9.4 Baum-Strukturen

9.4.1 Binäres Suchen

Um in einem unsortierten Array ein beliebiges Element zu finden, kann man das Array nur von Anfang bis Ende durchsuchen und jeden Schlüssel schrittweise mit dem Such-Schlüssel vergleichen, bis entweder das gewünschte Element gefunden oder das Ende erreicht ist. Ist das Array jedoch sortiert, so kann ein Suchvorgang mittels einer einfachen binären Suche schneller ablaufen.

Die Suche beginnt in der Mitte des Arrays beim Element $A(N/2)$ mit $N/2 \in \mathbb{N}$. Ist der Schlüssel dieses Elements größer als der gesuchte Schlüssel, so wird im linken Bereich des Arrays weitergesucht (andernfalls im rechten) und in diesem Bereich wieder das mittlere Element aufgerufen usw.



Gesucht sei der Schlüssel 5. Die Mitte ist in diesem Beispiel das 5. Element $A(5)$ (Schlüssel 23), das Such-Element ist kleiner, also neue Mitte im linken Bereich (gerundet) das 3. Element (Schlüssel 7). Wieder ist das Such-Element kleiner, somit neue Mitte beim (gerundet) 2. Element und damit ist das gesuchte Element gefunden.

Der Pseudo-Algorithmus:

```

vorne = 1
hinten = N
while vorne ≤ hinten
    mitte = (vorne+hinten)/2 (gerundet)
    case A(mitte).schlüssel = suchschlüssel → gefunden
    case A(mitte).schlüssel > suchschlüssel
        hinten = mitte-1
    case A(mitte).schlüssel < suchschlüssel
        vorne = mitte+1

```

Die Kosten (Laufzeit T in Abhängigkeit von der Anzahl n der Elemente) für diesen Algorithmus berechnen sich wie folgt: Die Kosten ergeben sich aus einem Vergleich im ersten Schritt zzgl. der Kosten für die verbleibende Hälfte des Arrays im zweiten Schritt, zzgl. der Kosten für die davon verbleibende Hälfte im dritten Schritt usw. Die Gesamtkosten lassen sich also rekursiv formulieren:

$$T(n) = T(n/2) + 1 \quad \text{wobei} \quad T(n/2) = T(n/4) + 1 \quad \text{also} \quad T(n) = T(n/4) + 2 \quad \text{usw.}$$

$$\text{letztlich } T(n) = T(n/2^i) + i$$

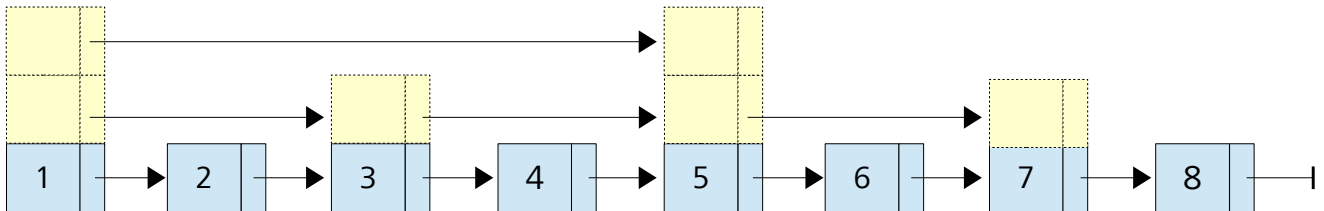
Im Worst Case muss so lange geteilt werden, bis nur noch ein Element verbleibt und damit das letzte $T(n/2^i) = 1$ sein muss. Dies ist mit $i = \log_2 n$ gegeben: $2^{\log_2 n} = n$, $T(n/n) = T(1) = 1$.

Damit ergibt sich als Laufzeit im Worst Case $T(n) = \log_2 n + 1$ und somit $O(\log n)$ (Hinweis: da die genaue Art des Logarithmus für die Laufzeitkomplexität nicht von Bedeutung ist, schreibt man nur $O(\log n)$).

9.4.2 Skiplisten

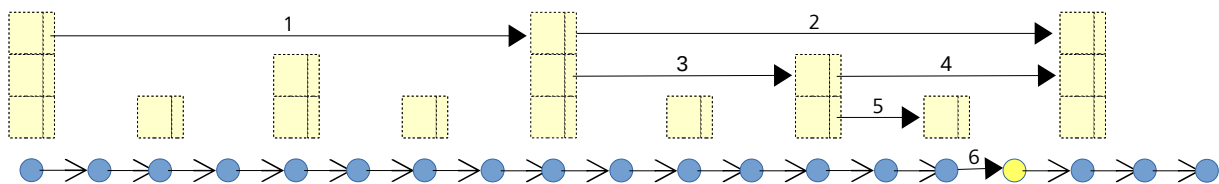
Für eine verkettete Liste bietet das Sortieren keinen besonderen Vorteil bzgl. der Suche, da kein wahlfreier Zugriff auf die Elemente erfolgen kann. Lediglich das Suchen nach einem gar nicht existierenden Schlüssel i kann in der sortierten Liste schneller erfolgen, da die Suche abbricht sobald ein Schlüssel mit einem Wert größer als i erreicht ist. Dies ist insbesondere beim Einfügen von Elementen in eine bereits sortierte Liste von Vorteil.

Versieht man eine sortierte Liste jedoch mit einer zusätzlichen „drübergestülpten“ Zugriffsstruktur, so können Suchvorgänge schneller erfolgen, indem größere Sprünge zwischen den Elementen ermöglicht werden.



Es müssen zwar nun noch mehr Zusatzinformationen für die Zeiger gespeichert werden, aber die Sprünge sind vergleichbar mit der binären Suche und damit wesentlich effektiver. Technisch betrachtet ist der Zeiger bei einer „Skip-Liste“ kein einzelner Wert mehr sondern ein Feld aus mehreren Zeigern. Eine Suche beginnt hierbei auf der obersten Zeigerebene: Man sucht dort das Element, das gerade noch kleiner ist als das gesuchte (beispielsweise die Suche nach Schlüssel 4): Sprung zur 5 (ist größer) also zurück zum Element mit Schlüssel 1. Dann wird auf der zweithöchsten Zeigerebene weitergesucht (in diesem Beispiel von der 1 zur 3 und weiter zur 5, also wieder zurück zur 3) und dann auf der dritthöchsten Zeigerebene (hier bereits die unterste) bis zum gesuchten Element (oder Abbruch falls auf der untersten Zeigerebene ein größerer Schlüssel als der gesuchte gefunden wurde).

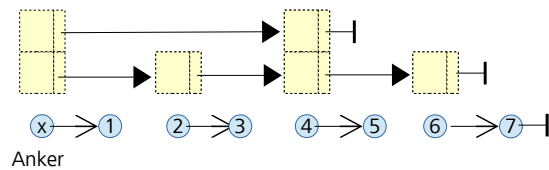
Folgendes Ideal-Beispiel entspricht der binären Suche in einem Array (der gelbe Punkt sei das gesuchte Element):



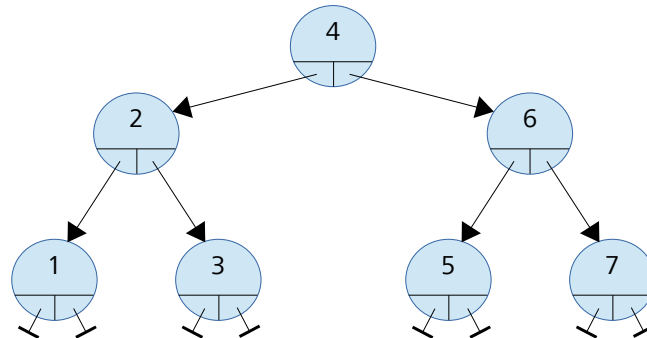
Wie man sieht, sollte sich optimalerweise die Anzahl der Zeiger-Knoten von einer Ebene zur nächst tieferen ungefähr verdoppeln, dann ergibt sich genau dieselbe Laufzeit $O(\log_2 n)$ wie bei der binären Suche. In der Praxis ist eine solche Optimalverteilung nur mit großem Aufwand herstellbar (man müsste „irgendwie gleichmäßig abzählen“ und dann den Elementen die Zeigerhöhe zuweisen). Daher werden Wahrscheinlichkeiten berechnet, wieviele Elemente die höchste Zeigerhöhe erhalten sollen, wieviele die zweithöchste usw. und dann werden die zur Verfügung stehenden Zeigertypen zufällig verteilt. In obigem Beispiel werden 3 Ebenen ($i=1, 2, 3$) vorgegeben, sodass bei „binärem Verteilen“ die Wahrscheinlichkeit $1/2^i$ ist. Ebene $i=3$ wird im Durchschnitt jedem achten Knoten zugewiesen: $1/2^3 = 1/8$, Ebene $i=2$ ($1/4$) jedem vierten Element und Ebene $i=1$ ($1/2$) jedem zweiten Element (im Durchschnitt). Die Abstände zwischen den jeweiligen Ebenen sind dann zwar nicht perfekt gleichmäßig, aber die Laufzeit wird davon nicht wesentlich beeinflusst. Beim Einfügen eines Elements in eine sortierte Skipliste wird zunächst dem neuen Element zufällig eine Zeigerhöhe zugewiesen, dann müssen die jeweiligen Vorgänger auf jeder Ebene ermittelt und die Verzeigerung angepasst werden.

9.4.3 Binäre Suchbäume

Betrachtet man den Idealfall bei einer sortierten Skipliste mit optimaler Verteilung der Ebenen, so fällt auf, dass man auf die vollständige Verzeigerung auf der untersten Ebene verzichten kann:



Zeichnet man dies etwas anders auf und ändert die Verzeigerungen und ihre Richtungen, so ergibt sich die Struktur eines Baumes:



Definition „Binärer Baum“:

Ein binärer Baum ist ein gerichteter Graph, der (sofern er nicht leer ist) aus einem Knoten K mit Verweisen auf zwei binäre Unterbäume (linker und rechter Unterbaum) besteht. Knoten mit Verweisen auf zwei leere Unterbäume werden als „Blätter“ bezeichnet. Der Eingangsknoten ist die „Wurzel“ des Baumes. Die „Tiefe eines Knotens K “ im Baum B ist: $\text{Tiefe}(K, B) = 1 + \text{Anzahl der Kanten zwischen } K \text{ und der Wurzel}$

Die „Baumtiefe von B “ entspricht dem Maximum der Knotentiefen (es wird statt „Tiefe“ auch analog von „Höhe“ gesprochen).

Als „Binärer Suchbaum“ wird ein binärer Baum bezeichnet, in dem alle Knoten im rechten Unterbaum einen größeren Schlüssel haben als der Wurzelknoten und im linken Unterbaum einen kleineren. Das Suchen in einem binären Suchbaum erfolgt genauso wie in einer idealen Skipliste: Ausgehend von der Wurzel wird ein Vergleich durchgeführt (gesuchter Schlüssel größer oder kleiner als der Wurzelschlüssel); entsprechend des Ergebnisses wird der linke oder rechte Unterknoten aufgerufen und der Vergleich mit diesem Knotenschlüssel durchgeführt, so lange bis der Schlüssel gefunden ist oder ein leerer Verweis (Null-Zeiger) erreicht ist.

Auch das Einfügen ist nicht schwierig: man folgt (entsprechend der Suche) dem Baum so lange bis ein Nullzeiger gefunden ist und ersetzt ihn durch einen Zeiger auf das neue Element 9.

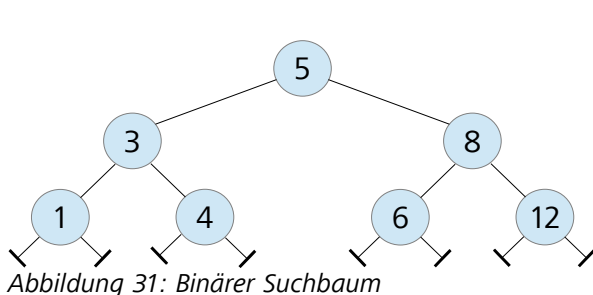


Abbildung 31: Binärer Suchbaum

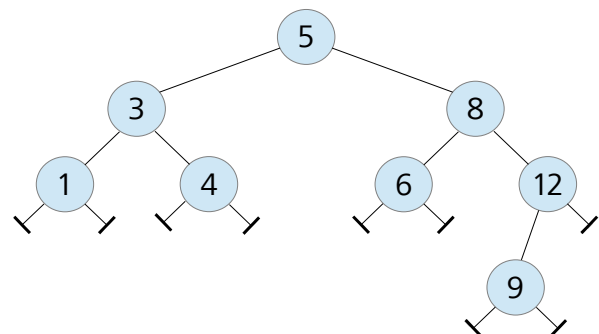


Abbildung 30: Einfügen eines Elements (9)

Fügt man in diesem Beispiel im Weiteren noch die Schlüssel 10 und 11 ein, so ergibt sich ein „nicht besonders schöner“ Baum: der Baum wird unausgeglich und kann im Extremfall zu einer sortierten verketteten Liste degradieren.

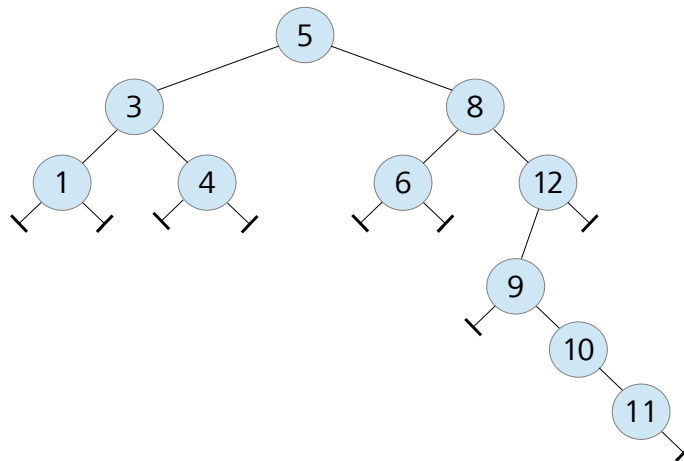


Abbildung 32: Einfügen von Elementen (10 und 11)

Pseudo-Code für das Einfügen im binären Suchbaum (rekursiver Algorithmus):

EINFÜGEN_REK

Aufruf mit „neuer Schlüssel“, „neue Daten“, Knoten K (zu Beginn der Wurzelknoten)

```

case K = null
    allokieren neues Element K.neuerSchlüssel
    return K

case neuerSchlüssel = K.schlüssel
    error „Element existiert bereits“

case neuerSchlüssel < K.schlüssel
    K.links → rekursiver Aufruf von EINFÜGEN_REK(neuerSchlüssel,
                                                neue Daten, K.links)
    return K

case neuerSchlüssel > K.schlüssel
    K.rechts → rekursiver Aufruf von EINFÜGEN_REK(neuerSchlüssel,
                                                  neue Daten, K.rechts)
    return K
  
```

Etwas schwieriger ist das Löschen eines Knotens, da die Baum-Eigenschaften erhalten bleiben müssen. Ganz einfach ist das Löschen eines Blattes, da nur der entsprechende Zeiger des darüber liegenden Knotens durch einen Null-Zeiger ersetzt werden muss. Auch das Löschen eines Knotens mit nur *einem* Nicht-Null-Zeiger ist nicht kompliziert: Hier muss der Zeiger des übergeordneten Knotens, der auf den zu löschenden verweist, lediglich durch den Nicht-Null-Zeiger des zu löschenden ersetzt werden. In obigem Beispiel (Abb. 32) soll das Element 9 gelöscht werden: es wird der linke Zeiger von 12 (zeigt ursprünglich auf 9) ersetzt durch den Zeiger von 9 auf 10.

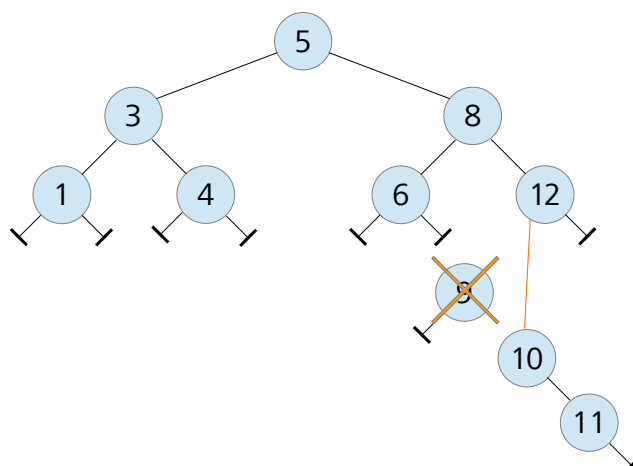


Abbildung 33: Löschen (Element mit einem Null-Zeiger)

Schwieriger wird es, wenn ein Knoten mit *zwei* Nicht-Null-Zeigern gelöscht werden soll, da in diesem Fall der zu löschende Knoten durch einen anderen ersetzt werden muss und zwar so, dass die Baumeigenschaften erhalten bleiben. Um diese Eigenschaften nicht zu zerstören, darf ein Knoten offenbar nur durch denjenigen Knoten ersetzt werden, der den nächst-kleineren oder den nächst-größeren Schlüssel hat, da dieser keinen rechten bzw. linken Nachfolger haben kann. Dieser Knoten muss also zunächst gesucht werden. Ausgehend von dem Baum in Abb. 32 soll die 8 gelöscht werden und kann entweder durch die 6 oder die 9 ersetzt werden. Nimmt man die 9 (den sog. Inorder-Nachfolger), so müssen folgende Aktionen durchgeführt werden:

- Suche das Element mit Schlüssel 9 (den Inorder-Nachfolger der 8)
- Verfahre wie beim Löschen von 9, indem der Zeiger von 12 auf 9 durch denjenigen von 9 auf 10 ersetzt wird
- Ersetze die Zeiger von 9 durch die von 8
- 8 kann gelöscht werden
- Zeiger $5 \rightarrow 8$ muss durch $5 \rightarrow 9$ ersetzt werden

Das Suchen des Inorder-Nachfolgers geschieht einfach dadurch, dass man ausgehend vom Lösch-Knoten einmal nach rechts geht und anschließend im Unterbaum so lange nach links bis ein linker Null-Zeiger gefunden ist (entsprechend der Inorder-Vorgänger: einmal nach links, dann immer rechts).

Pseudo-Code für das Löschen im binären Baum (rekursiver Algorithmus)

LÖSCHEN_REK (Übergabeparameter: LöschSchlüssel, Knoten K)

```

case K=null
    error „Element existiert nicht“

case LöschSchlüssel < K.schlüssel
    K.links → rekursiver Aufruf von LÖSCHEN_REK(LöschSchlüssel,
                                                K.links)
    return K

case LöschSchlüssel > K.schlüssel
    K.rechts → rekursiver Aufruf von LÖSCHEN_REK(LöschSchlüssel,
                                                K.rechts)
    return K

case LöschSchlüssel = K.schlüssel
    if K.links = null
        then return K.rechts
    else
        if K.rechts = null
            then return K.links
        else
            ersatzK → Aufruf von NACHFOLGER_SUCHE(K)
            ersatzK.links = K.links
            ersatzK.rechts = K.rechts
            return ersatzK

```

Pseudo-Code zur Nachfolger-Suche (Inorder-Nachfolger „ersatz“) für das Löschen von K im binären Baum (iterativer Algorithmus):

NACHFOLGER_SUCHE(K)

```

if linker Zeiger von K.rechts = null
    then ersatz = K.rechts
    K.rechts = rechter Zeiger von K.rechts
else K = K.rechts
    while linker Zeiger von K.links ≠ null
        K = K.links
    ersatz = K.links
    K.links = rechter Zeiger von K.links
return ersatz

```


Beschreibung:

- Die if-Abfrage prüft, ob der rechte Knoten bereits selbst der gesuchte Nachfolger ist, wenn ja so wird dieser Knoten zum Ersatzknoten (Achtung: in LÖSCHEN_REK wird $\text{ersatz.rechts} = K.\text{rechts}$, was bedeutet, dass in diesem Fall der rechte Zeiger des Ersatzknotens auf sich selbst zeigen würde; daher ist es erforderlich den rechten Zeiger vom zu löschenden K auf den rechten Zeiger des Ersatzknotens zu setzen, damit in LÖSCHEN_REK der Ersatzknoten wieder seinen ursprünglichen rechten Knoten zugewiesen bekommt)
- hinter dem „else“ (einmalig nach rechts gehen): der neue Knoten K ist der rechte Knoten des alten K
- in der while-Schleife wird geprüft, ob der linke Zeiger von $K.\text{links}$ nicht Null ist, wenn ja so wird immer weiter nach links gegangen bis dies der Fall ist und die Schleife ist beendet
- dann kann dieser Knoten (also der mit dem linken Null-Zeiger) zum Ersatzknoten erklärt werden
- und zum Schluss muss dieser Ersatzknoten in seinem bisherigen Unterbaum noch ausgehängt werden, indem sein rechter Knoten links an seinen Eltern-Knoten angehängt wird

9.4.4 Balancierte Bäume (AVL-Bäume)

Der binäre Suchbaum kann im Extremfall zu einer verketteten Liste degradieren, sodass die Laufzeit zu $O(n)$ werden kann statt $O(\log_2 n)$ im ausbalancierten Zustand. Im ideal ausbalancierten Zustand verdoppelt sich die Anzahl der Knoten (bzw. Blätter) mit jeder zusätzlichen Ebene.

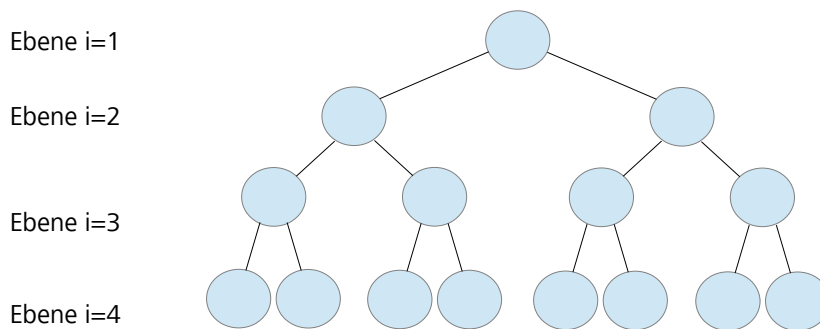


Abbildung 34: idealer AVL-Baum

Gesamtzahl der Knoten/Blätter: $2^0 + 2^1 + \dots + 2^{i-1} = \sum_{k=0}^{i-1} 2^k = 2^i - 1$ (geometrische Reihe)

Wenn ein Baum 2^i Knoten hat, folgt umgekehrt daraus, dass es offenbar eine Tiefe des Baumes von $\Omega(\log_2 i)$ gibt, woraus eine Laufzeit von $O(\log_2 i)$ für alle Operationen auf dem Baum folgt. Um diese Laufzeit zu erhalten, darf also ein Baum mit $2^i - 1$ Knoten/Blättern maximal die Tiefe i haben. Und weiter: Um diese maximale Tiefe zu erreichen, dürfen sich offenbar alle Unterbäume eines Knotens K nur um eine Ebene unterscheiden.

Definition: Ein binärer Suchbaum wird als AVL-Baum bezeichnet, wenn für die Unterbäume jedes Knotens K gilt:
 $\text{Tiefe}(K.\text{rechts}) - \text{Tiefe}(K.\text{links}) \in \{-1, 0, 1\}$

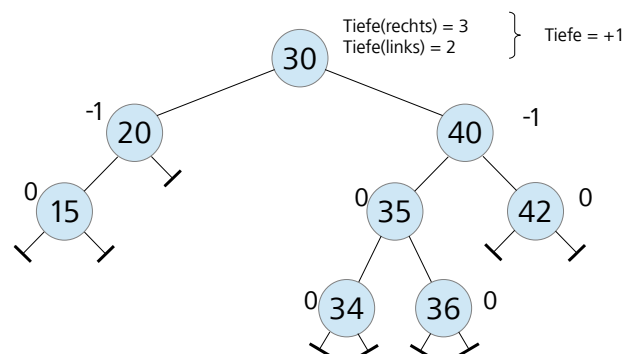


Abbildung 35: AVL-Baum mit Tiefenangaben

Für den Baum in Abb. 35 ist die Bedingung für einen AVL-Baum erfüllt.

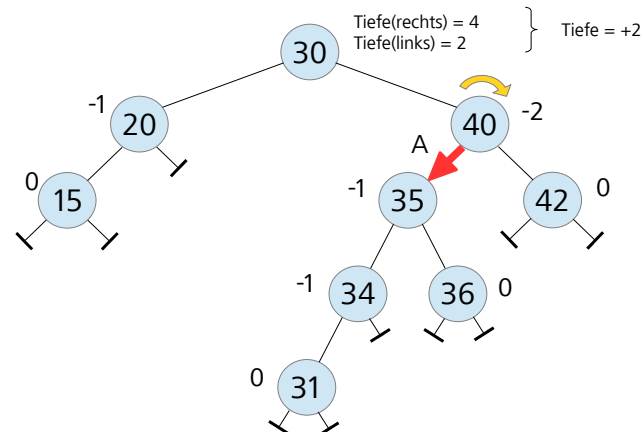


Abbildung 36: nicht ausbalancierter Baum

Dieser Baum (Abb. 36) ist nach dem Einfügen von Schlüssel 31 nicht mehr ausbalanciert. Der Baum muss daher umgestellt werden und zwar so, dass er zu einem AVL-Baum wird aber auch weiterhin die Eigenschaften des binären Suchbaumes erfüllt.

Man erreicht dies mithilfe einer sog. „Rotation“ im Baum (Links- oder Rechts-Rotation). Prinzipiell wird bei der Rotation ein tieferer Unterbaum um eine Ebene höher geschoben und ein weiterer Unterbaum umgehängt, um die Eigenschaften des binären Suchbaumes zu erhalten.

Beispiel: Ausgangs-Baum sei Abb. 36, es soll der linke Unterbaum von Knoten 40 „hochgezogen“ werden, indem Knoten 40 nach rechts rotiert wird.

Knoten 40 wird nach rechts rotiert, indem:

- die 35 (K2) an die Stelle der 40 (K1) verschoben wird,
- die Richtung von Kante A umgedreht wird, d.h. die 40 hängt jetzt rechts unter der 35,
- der rechte Unterbaum der 35 (hier nur die 36) links an die 40 gehängt wird.

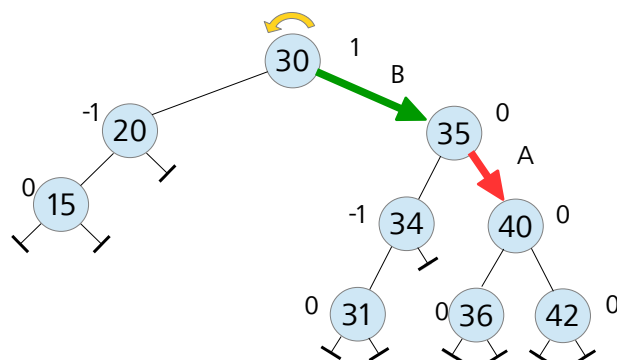


Abbildung 37: nach der Rechts-Rotation der 40

Jetzt ist der Baum ausgeglichen und die „links-kleiner-“ und „rechts-größer“-Eigenschaft des binären Suchbaumes ist wieder gegeben.

Auch wenn es bzgl. der Ausgeglichenheit des Baumes keine weitere Verbesserung mehr bringt: Links-Rotation von Knoten 30 (nur als prinzipielles Beispiel für eine Links-Rotation):

- die 35 an die Stelle der 30 schieben,
- die Richtung von Kante B umdrehen, d.h. die 30 hängt jetzt links unter der 35,
- den linken Unterbaum der 35 (hier ab 34) rechts an die 30 hängen.

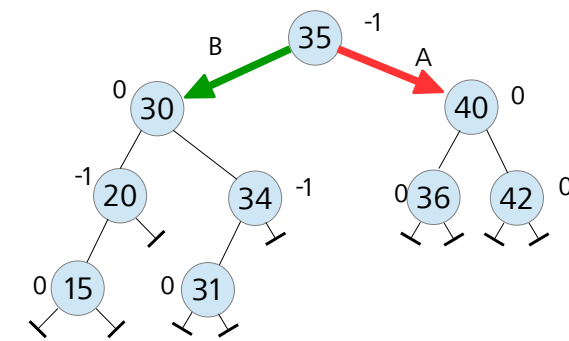


Abbildung 38: nach Links-Rotation der 30

Algorithmus für diese Rotation:

```

LINKS-ROTATION (K)
  x = K.rechts
  K.rechts = x.links
  x.links = K
  return x

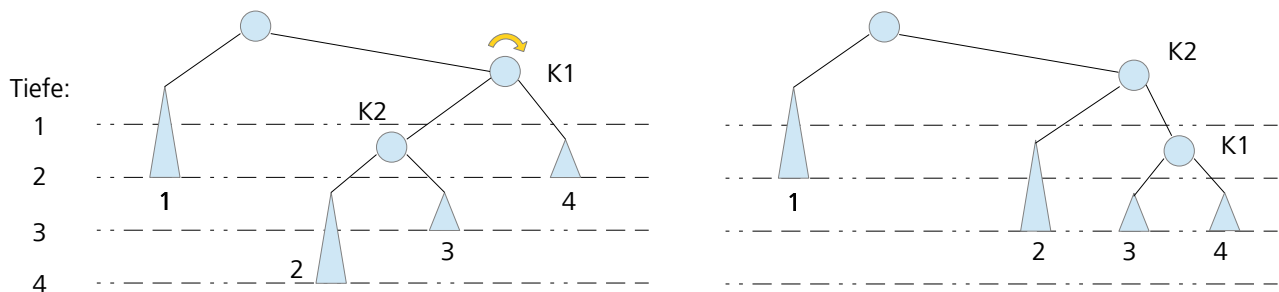
```

Bezogen auf die Links-Rotation von Knoten $K=30$ (in Worten): Zwischenspeicherung des hochzuschiebenden Knotens $30.rechts=35$ als x . Dann wird der linke Unterbaum von $x=35$ ($35.links=34$) rechts an die $K=30$ gehängt (das neue $30.rechts=34$). Und schließlich wird $35.links$ zu 30 .

„**Links**-Rotation“ von Knoten $K1$ formal: Tausche Knoten $K1$ mit $K2$ (mit $K1 < K2$), drehe die Kante zwischen $K1$ und $K2$ um und hänge den linken Unterbaum von $K2$ rechts an $K1$ an.

„**Rechts**-Rotation“ von Knoten $K1$ formal: Tausche Knoten $K1$ mit $K2$ (mit $K1 > K2$), drehe die Kante zwischen $K1$ und $K2$ um und hänge den rechten Unterbaum von $K2$ links an $K1$ an.

Nochmals anschaulich die Rechts-Rotation (in obigem Beispiel $K1=40$)



Offensichtlich kann beim Einfügen eines neuen Elements in den AVL-Baum die Balance verloren gehen. Es muss also während des Einfügens die Tiefe „gemessen“ werden und falls die Tiefe $\notin \{-1, 0, 1\}$ muss eine geeignete Rotation durchgeführt werden. Zunächst wird entsprechend des rekursiven Einfüge-Codes (s.o.) die richtige, neue Stelle gefunden. Dann kann einfach im Laufe der Rückkehr aus der Einfüge-Rekursion jeweils geprüft werden, ob sich die Balance verändert hat (hierzu ist eine Speicherung des vorherigen Balance-Wertes $K.balance$ erforderlich). Hat man z.B. im linken Unterbaum eines Knotens einen neuen Wert eingehängt und dieser Knoten hatte bereits einen Balance-Wert von -1 , so würde der neue Balance-Wert -2 sein und eine Rechts-Rotation dieses Knotens wäre erforderlich.

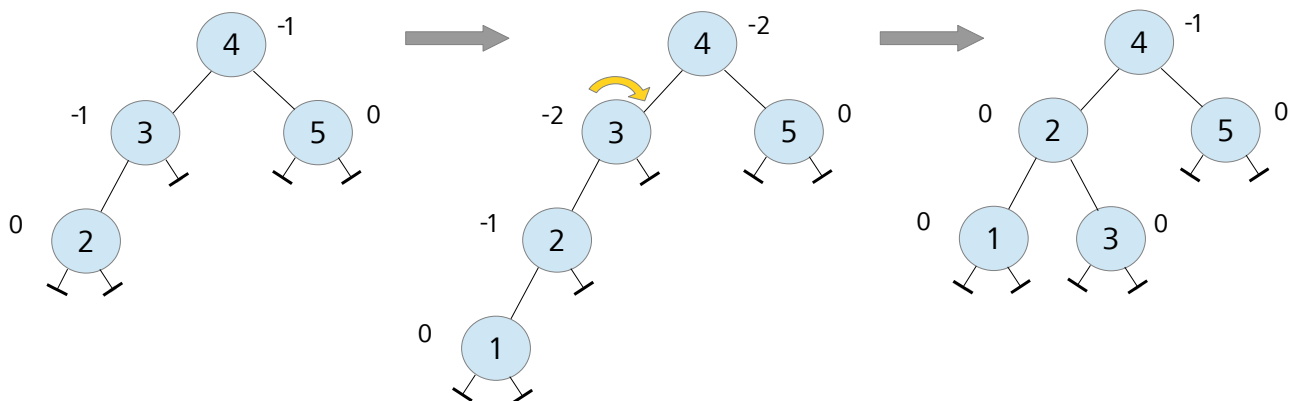


Abbildung 39: Ausbalancieren nach Einfügen der 1

Die (rekursiven) Algorithmen hierzu sind nicht schwierig, aber in der Darstellung „etwas unübersichtlich“ (daher soll hier darauf verzichtet werden), das Prinzip ist jedoch leicht nachvollziehbar: Da man auf dem Rückweg aus der Einfüge-Rekursion an jedem Knoten weiß, ob man auf dem Hinweg links oder rechts weitergegangen ist, kann der alte Balance-Wert entweder erhöht (rechts) oder verringert (links) werden. Ist der neue Balance-Wert $+2$ so muss nun an dieser Stelle zunächst eine Links-Rotation durchgeführt werden, entsprechend bei -2 eine Rechts-Rotation; erst danach wird der Rückweg aus der Einfüge-Rekursion fortgesetzt. Abb. 39 zeigt das Vorgehen nach Einfügen des neuen Elements 1.

Problem der „**Doppel-Rotation**“: Betrachtet man Abb. 40 so erkennt man den prinzipiellen Ablauf einer Rechts-Rotation bzgl. eines Unterbaumes 1, an den ein neues Element angehängt wird; der Baum ist nicht ausbalanciert. Die Rechts-Rotation ersetzt K1 durch K2, hängt K1 rechts an K2 an und der ehemalige rechte Unterbaum 2 von K2 wird links an K1 angehängt: die Balance ist wiederhergestellt.

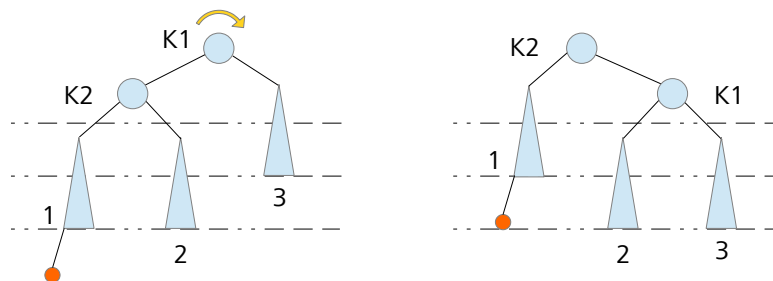


Abbildung 40: Einfügen eines neuen Elements: ausbalancierter Baum nach Rotation

Hängt man jedoch das neue Element an den Unterbaum 2 an (Abb. 41), so bringt die Rechts-Rotation nicht das gewünschte Ergebnis: der Baum ist weiterhin nicht in Balance. Damit die Rotation erfolgreich ist, wird ein Trick angewendet: Im Beispiel muss derjenige Unterbaum, an dem das neue Element hängt (der also die größere Tiefe hat) nach „links außen“ verschoben werden (denn dann funktioniert ja die Rotation wie erwünscht).

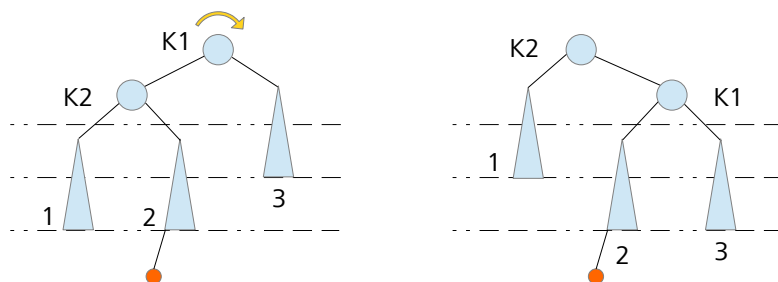


Abbildung 41: Einfügen eines neuen Elements: Trotz Rotation ist der Baum nicht ausbalanciert

Dies wird durch eine „vorbereitende Links-Rotation“ von K2 erreicht, obwohl diese Aktion bzgl. der Balance zunächst gar nichts bringt (entsprechend in einem rechten Unterbaum: es muss der Unterbaum ganz rechts außen die größte Tiefe bekommen s. Abb. 42). Da hierfür in den Pseudo-Codes diverse Fälle unterschieden werden müssen, sollen diese Codes hier nicht weiter aufgeführt werden.

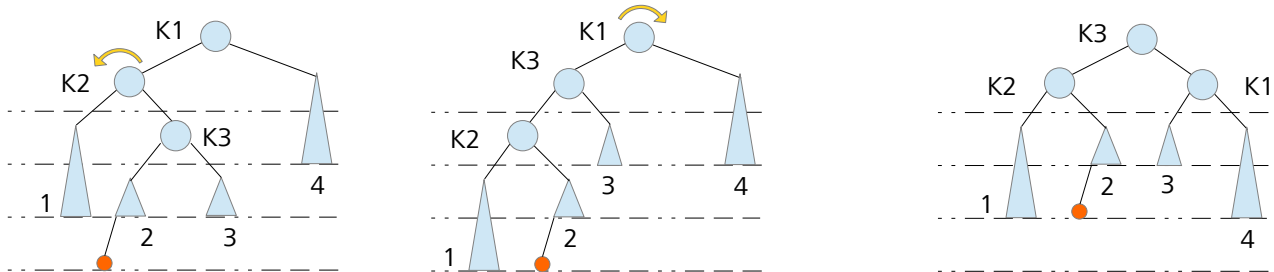


Abbildung 42: Doppelrotation

Ob eine Doppelrotation erforderlich ist oder nicht, kann wie folgt festgestellt werden: Man stellt bei der Rückkehr aus der Einfüge-Rekursion fest, dass $K1.balance = -2$ also muss eine Rechts-Rotation von K1 durchgeführt werden. Ist der Kind-Knoten K2 „links-lastig“ (Abb. 40) ist offenbar keine Doppelrotation nötig, ist K2 dagegen „rechts-lastig“ (Abb. 41), so muss die Doppelrotation durchgeführt werden. Formal: Bei erforderlicher Rechts-Rotation von K1 prüfen ob $K1.links.balance = K2.balance = 1$, dann Doppelrotation. Bei erforderlicher Links-Rotation eines Knotens K1, prüfen ob $K1.rechts.balance = K2.balance = -1$, dann Doppelrotation.

9.4.5 B-Bäume (Bäume mit extern gespeicherten Daten)

Bisher wurde bei der Betrachtung von Feldern, Listen oder Bäumen davon ausgegangen, dass sich alle Daten im Hauptspeicher des Rechners befinden. Der Zugriff auf einzelne Elemente erfolgt sehr schnell und ist für alle Elemente gleich schnell. Der größte „Kostenfaktor“ bzgl. der Laufzeit ist immer der Vergleich (also der Test, ob der Suchschlüssel gleich, kleiner oder größer als der Schlüssel des aktuellen Elements ist); die Zeit zum Laden des Elements ist im Vergleich dazu vernachlässigbar.

Ist ein Element jedoch in einem langsamen externen Speicher (Festplatte, SSD) abgelegt, so gelten diese Annahmen nicht mehr uneingeschränkt. Solche Medien sind an sich bereits langsamer im Zugriff, zudem sind die Daten in größeren Blöcken (i.d.R. 4 kB) organisiert, sodass auch für kleine Datenmengen jeweils ein ganzer Block gelesen bzw. geschrieben werden muss. Die eigentlich vorteilhafte Funktionsweise von AVL-Bäumen kann mit externen Speichermedien sehr problematisch werden: Bei Änderungen im AVL-Baum werden insbesondere bei Rotationen viele (kleine) Elemente „angefasst“, um die Verzeigerungen zu ändern; Kopiervorgänge werden dadurch vermieden. Wenn man aber ohnehin gezwungen ist, Daten von einem externen Speicher zu lesen/schreiben, so sollte ihr Informationsgehalt etwas mehr Möglichkeiten bieten als die einfache „Rechts-/Links-Entscheidung“. Ziel beim Aufbau eines B-Baumes ist es, eine möglichst geringe Tiefe zu erhalten; es sollten auch bei großen Datenmengen nur sehr wenige Knoten gelesen werden müssen.

Beispiel für einen solchen Baum, der die Entscheidungen „kleiner als der linke Schlüssel“, „größer als der rechte Schlüssel“ oder „zwischen beiden Schlüssel“ ermöglicht:

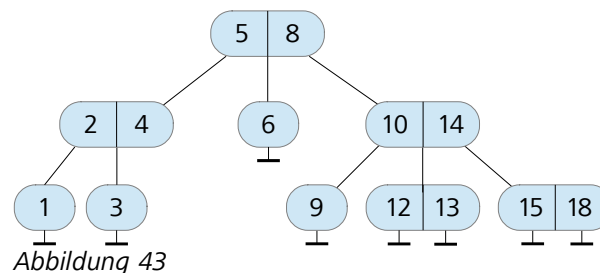


Abbildung 43

Definition: Ein „B-Baum der Ordnung m “ ist ein verallgemeinerter Suchbaum mit den Eigenschaften, dass jeder Knoten maximal $2 \cdot m$ Schlüssel enthält und – außer der Wurzel – mindestens m Schlüssel. Alle Nullzeiger liegen in derselben Tiefe.

Der Baum in Abb. 43 erfüllt diese Eigenschaft nicht vollständig (Element 6 liegt nicht in derselben Tiefe wie die anderen Blätter)..

Beispiel für einen B-Baum der Ordnung $m=2$:

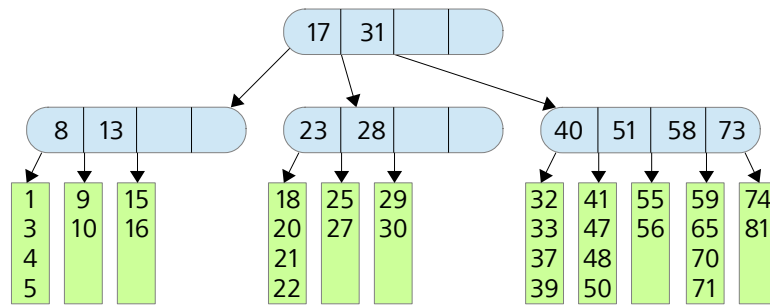


Abbildung 44: B-Baum der Ordnung $m=2$

Alle Bedingungen des B-Baumes sind erfüllt. Sucht man z.B. Schlüssel 59, so vergleicht man zunächst alle Schlüssel der Wurzel bis ein größerer Schlüssel oder das Ende der Liste gefunden ist und folgt dem entsprechenden Zeiger zum nächsten Knoten. Das Einfügen im B-Baum erfolgt grundsätzlich auf der untersten Ebene, also in den Blättern. Sofern für einen einzufügenden Schlüssel ein Platz im passenden Blatt frei ist, bleibt die Grundstruktur des Baumes erhalten. Schwieriger wird es, wenn dies nicht der Fall ist, also das passende Blatt vollständig gefüllt ist. In diesem Fall müssen Blätter aufgeteilt und einzelne Schlüssel vom Blatt in den nächsthöheren Knoten verschoben werden. Sollte dieser Knoten jedoch bereits belegt sein, so muss auch er geteilt und einer seiner Schlüssel nach oben verlagert werden. Im Extremfall muss die Wurzel des Baumes erweitert werden. Ist auch dies nicht mehr möglich, so wird die Wurzel geteilt und ein neuer, noch höherer Wurzelknoten geschaffen (daher darf der Wurzelknoten der einzige mit weniger als m Schlüsseln sein).

Beispiel oben: Einfügen eines neuen Schlüssels 57 ist einfach in dem entsprechenden Blatt möglich. Einfügen eines neuen Schlüssels 19 ist schwieriger, da das entsprechende Blatt vollständig belegt ist. Vorgehensweise: Man fügt trotzdem zunächst das neue Element in das Blatt ein und teilt es dann. Die kleinsten m Schlüssel werden zu einem neuen Blatt, der mittlere Wert (den es ja bei $2*m+1$ immer gibt) wird in den Eltern-Knoten verschoben und dort entsprechend einsortiert, und die größten m Schlüssel bilden wieder ein neues Blatt. Die beiden neu geschaffenen Blätter werden vor bzw. hinter dem neuen Schlüssel des Eltern-Knotens verzeigert.

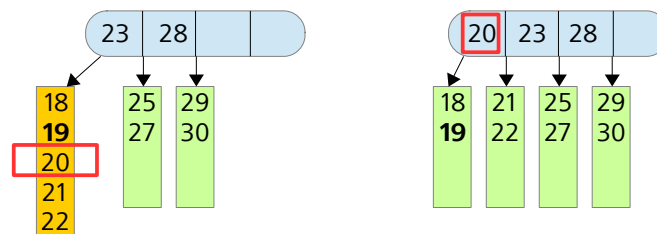


Abbildung 45: Einfügen im B-Baum

Das Einfügen des neuen Schlüssels 62 in dem Baum in Abb. 84 wird noch schwieriger, da in diesem Fall auch der Eltern-Knoten bereits vollständig belegt ist.

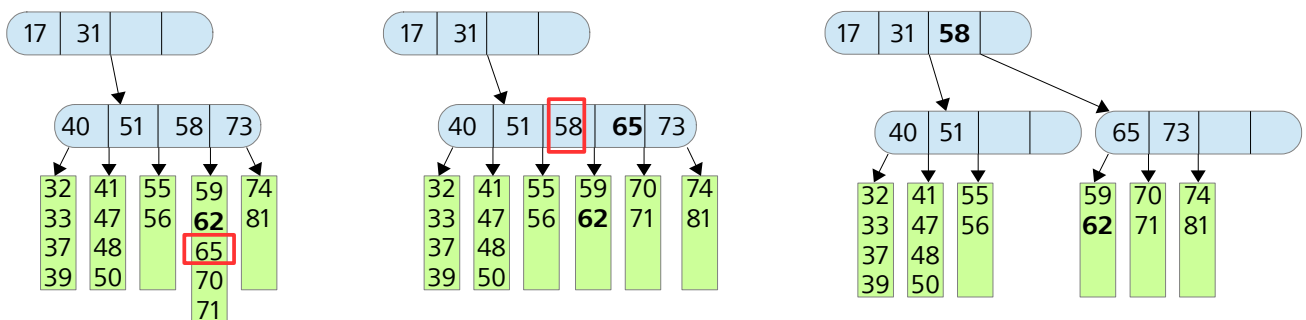


Abbildung 46: Einfügen im B-Baum

Vorgehensweise: Einfügen der 62 in das entsprechende Blatt, teilen dieses Blattes in der Mitte an der 65 (linker Teil von Abb. 46), die 65 in den Elternknoten einfügen (mittlerer Teil der Abbildung), auch diesen in der Mitte an der 58 teilen, die 58 in ihren Elternknoten einfügen (rechts).

Beim Löschen (soll hier nicht weiter behandelt werden) müssen diverse Fälle unterschieden werden. Solange die Eigenschaften des B-Baumes (max. $2 \cdot m$, mind. m Elemente pro Blatt) erhalten bleiben, kann im Blatt einfach ein Schlüssel gelöscht werden. In allen anderen Fällen muss geeignet umsortiert werden, z.B. wenn ein Blatt weniger als m Elemente besitzt oder es muss beim Löschen eines Knoten-Schlüssels analog zum Binärbaum ein Inorder-Nachfolger gesucht werden, der den zu löschenden Schlüssel ersetzt.

Laufzeit (ohne Herleitung): Ein B-Baum mit n Elementen und der Ordnung m ist in $O(\log_{m+1} n)$.

Beispielsweise hat ein B-Baum mit 100 Mio. Elementen und der Ordnung $m=100$ lediglich eine Tiefe 5 (!), sodass also nur wenige Datenblöcke eingelesen werden müssen, auch wenn hierbei pro Knoten sehr viele Vergleiche erfolgen müssen.

Eine Variante des B-Baumes ist der **B⁺-Baum**, bei dem nur die Blätter Verweise auf die tatsächlichen Daten haben dürfen. Die höheren Knoten spannen also nur eine Art „Index-Baum“ auf.

Literatur

Martin Dietzfelbinger et al.: Algorithmen und Datenstrukturen, Springer Vieweg 2014

Hartmut Ernst et al.: Grundkurs Informatik, Springer Vieweg 2016

Jan Goyvaerts, Steven Levithan: Reguläre Ausdrücke, O'Reilly 2010

Michael Kofler: Linux – Das umfassende Handbuch, Rheinwerk 2016

Heinrich Müller, Frank Weichert: Vorkurs Informatik, Springer Vieweg 2015

Thomas Ottmann, Peter Widmayer: Algorithmen und Datenstrukturen, Spektrum Akademischer Verlag 2012

Larry L. Peterson, Bruce S. Davie: Computer Networks, Elsevier 2012

Andrew S. Tanenbaum, Computernetzwerke, Pearson 2012

Karsten Weicker, Nicole Weiker: Algorithmen und Datenstrukturen, Springer Vieweg 2013

Jürgen Wolf, Stefan Kania: Shell-Programmierung – Das umfassende Handbuch; Rheinwerk Computing 2016

www.self-linux.org/selflinux/html/shellprogrammierung.html

www.linux-praxis.de/linux1/shell2.html

The Linux Documentation Project: Advanced Bash-Scripting Guide tldp.org/guides.html

The Linux Documentation Project: Bash Guide for Beginners: tldp.org/guides.html

<https://www.gnu.org/software/sed/manual/sed.html>